Học phần Thiết kế Web

GIÁO TRÌNH GIẢNG DẠY · HỌC PHẦN 111101

Mục tiêu trọng tâm

- **Biết đọc hơn biết viết:** Sinh viên không cần thuộc lòng code, nhưng nhìn vào phải hiểu AI đang viết gì.
- **Làm chủ Prompt:** Thành thạo cách ra lệnh cho AI để tạo giao diện, fix lỗi và tối ưu web.
- **Sản phẩm thực tế:** Kết thúc 15 buổi, mỗi nhóm phải có một Website hoàn chỉnh "made by AI & Human".

Buổi	Chủ đề chính	Nội dung thực hành (Hands-on)	Kỹ năng AI cần đạt
1	**Mì ăn liền với HTML**	Tạo một trang "Profile cá nhân" đơn giản gồm: Ảnh, Họ tên, Kỹ năng.	Dùng AI tạo khung HTML từ một mô tả bằng tiếng Việt.
2	**Trang trí với CSS**	Đổi màu chữ, font chữ, tạo khung (Border) cho trang Profile ở buổi 1.	Yêu cầu AI "phối màu chuyên nghiệp" và "chọn font chữ hiện đại".
3	**Layout: Dàn trang nhanh**	Chia cột cho Website (Cột trái: Menu, Cột phải: Nội dung).	Học cách mô tả Layout bằng lời để AI viết CSS Flexbox/Grid.
4	**Thực chiến: Landing Page (P1)**	Dựng Header và Banner lớn (Hero Section) cho một trang bán hàng.	Copy link một ảnh đẹp và bảo AI: "Viết code HTML/CSS giống kiểu này".
5	**Thực chiến: Landing Page (P2)**	Dựng các khối: Danh sách sản phẩm, Ý kiến khách hàng, Footer.	Dùng AI tạo nội dung giả (Lorem Ipsum) và ảnh minh họa tự động.
6	**Web "Co giãn" (Responsive)**	Làm cho trang Landing Page xem được trên iPhone/iPad không bị vỡ.	Prompt: "Làm đoạn code này tương thích với màn hình điện thoại".

Buổi	Chủ đề chính	Nội dung thực hành (Hands-on)	Kỹ năng AI cần đạt
7	**Bootstrap/Tailwind**	Sử dụng thư viện có sẵn để web đẹp lung linh trong 5 phút.	Ra lệnh: "Chuyển code CSS thuần này sang dạng Bootstrap 5".
8	**Kiểm tra giữa kỳ**	**Đề bài:** Dùng AI thiết kế 1 trang Web giới thiệu về Trường Đại học Lạc Hồng.	Đánh giá khả năng "điều khiển AI" để ra kết quả nhanh nhất.
9	**Javascript: Những cú chạm đầu**	Tạo nút bấm: Click vào thì hiện thông báo hoặc đổi màu nền.	Dùng AI giải thích ý nghĩa các câu lệnh: function, onclick.
10	**Form & Xử lý dữ liệu**	Tạo Form liên hệ. Kiểm tra nếu bỏ trống thì báo đỏ.	Prompt: "Viết script kiểm tra email trong form phải có đuôi @lhu.edu.vn".
11	**Hiệu ứng & Chuyển động**	Làm Slide ảnh chạy (Carousel) và hiệu ứng hiện dần khi cuộn trang.	Dùng AI tìm và tích hợp các thư viện hiệu ứng có sẵn (AOS, Swiper).
12	**Kết nối dữ liệu (API)**	Lấy danh sách tin nhắn hoặc thời tiết từ Internet đổ lên Web.	Prompt: "Viết code Fetch API lấy dữ liệu từ [Link] và hiển thị dạng bảng".
13	**AI Debugging (Sửa lỗi)**	Cố tình làm sai code và dùng AI để tìm lỗi - sửa lỗi.	Kỹ năng: Copy thông báo lỗi từ Console dán vào AI để hỏi cách sửa.
14	**Hoàn thiện đồ án nhóm**	Các nhóm ghép nối các phần lại thành website hoàn chỉnh.	Dùng AI tối ưu lại toàn bộ code (Clean Code) và viết chú thích.

Buổi	Chủ đề chính	Nội dung thực hành (Hands-on)	Kỹ năng AI cần đạt
15	**Showcase (Báo cáo)**	Demo Website. Giải thích cách các em đã "dạy" AI làm việc như thế nào.	Kỹ năng thuyết trình và chứng minh sự sáng tạo cá nhân.

Phương pháp đánh giá (Nhẹ nhàng - Thực tế)

- **Sản phẩm (70%):** Website chạy tốt, giao diện ưa nhìn, không lỗi.
- **Nhật ký Prompt (20%):** Một file Word lưu lại các câu lệnh (Prompt) mà sinh viên đã dùng để hỏi AI. Đây là bằng chứng của việc tư duy.
- **Thái độ & Cải tiến (10%):** Khả năng tùy biến lại những gì AI viết (không copy 100% một cách mù quáng).

Công cụ bắt buộc

1. **Trình duyệt:** Chrome / Edge.
2. **Soạn thảo:** VS Code (Cài thêm extension **GitHub Copilot** hoặc **Codeium** - đây là những AI "nằm vùng" ngay trong chỗ viết code).
3. **Trợ lý:** ChatGPT / Claude (Dùng để hỏi đáp và sinh mã nguồn).

MỤC LỤC

Mục tiêu trọng tâm 1
Phương pháp đánh giá (Nhẹ nhàng - Thực tế) 3
Công cụ bắt buộc 3
Outline 14
KHUNG NỘI DUNG 12 TUẦN (PHƯƠNG PHÁP LỚP HỌC ĐẢO NGƯỢC) 14
GIAI ĐOẠN 1: THIẾT LẬP NỀN TẢNG & CÔNG CỤ (Tuần 1 - 2) 14
GIAI ĐOẠN 2: HỆ THỐNG GIAO DIỆN & RESPONSIVE (Tuần 3 - 6) 14
TUẦN 7: ĐÁNH GIÁ GIỮA KỲ (KIỂM TRA NĂNG LỰC) 14
GIAI ĐOẠN 3: LOGIC & TƯƠNG TÁC THỰC CHIẾN (Tuần 8 - 12) 14
Tuần 1: Nhập môn & Hệ sinh thái AI Web Development 16
PHẦN 1 - MỤC TIÊU VÀ CHUẨN ĐẦU RA BÀI HỌC 16
1. Mục tiêu bài học (Lesson Objectives) 16
2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO) 16
3. Ma trận ánh xạ Chuẩn đầu ra (Mapping) 16
PHẦN 2: LÝ THUYẾT 17
1. Hệ sinh thái Công cụ (The Modern Toolstack) 17
2. Tư duy Lập trình thời đại AI (AI-First Mindset) 17
3. Nền tảng Quản lý mã nguồn nhóm (Git Workflow) 17
4. Khái niệm cơ bản về cấu trúc Web 17
5. Văn hóa làm việc nhóm (Agile/Scrum cơ bản) 17
PHẦN 3 - THỰC HÀNH - KÍCH HOẠT HỆ SINH THÁI VIBE CODE 18
Hoạt động 1: Kiểm tra & Đồng bộ môi trường (20 phút) 18
Hoạt động 2: Thử thách "Git nhanh" - Xây biệt thự Web (30 phút) 18
Hoạt động 3: Hình thành dự án xuyên suốt (45 phút) 18
Hoạt động 4: Buổi "Vibe Prompting" đầu tiên (30 phút) 18
Check-list hoàn thành buổi thực hành (Nộp bài) 19
TUẦN 2: TƯ DUY SEMANTIC HTML5 & CẤU TRÚC WEB 20
TUẦN 2: TƯ DUY SEMANTIC HTML5 & CẤU TRÚC WEB HỆ THỐNG 20
1. Mục tiêu bài học (Lesson Objectives) 20
2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO) 20
3. Nội dung chuẩn bị (Ở nhà - Flipped Classroom) 20
BẢNG TỔNG HỢP CÁC THẺ HTML SEMANTIC 20
Lời khuyên cho sinh viên khi làm bài Tuần 2: 22
4. Phần thực hành tại lớp (In-class Activities) 22
5. Đánh giá & Checklist hoàn thành 23

THÁCH THỨC TẠI NHÀ: "KIẾN TRÚC SƯ SEMANTIC" 24
THÁCH THỨC TẠI NHÀ: "KIẾN TRÚC SƯ SEMANTIC" (TUẦN 2) 24
1. Mục tiêu của thách thức 24
2. Yêu cầu cụ thể 24
3. Hướng dẫn nộp bài 24
Gợi ý thực hiện (Tips) 25
TUẦN 3: CSS LAYOUT SYSTEM (FLEXBOX & GRID) 26
1. Mục tiêu bài học (Lesson Objectives) 26
2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO) 26
3. Nội dung chuẩn bị (Ở nhà - Flipped Classroom) 26
4. Phần thực hành tại lớp (In-class Activities) 26
HƯỚNG DẪN THỰC HÀNH CHI TIẾT 28
HƯỚNG DẪN THỰC HÀNH CHI TIẾT (STEP-BY-STEP) 28
HOẠT ĐỘNG 1: "FLEX & GRID BATTLE" (CĂN CHỈNH NHANH) 28
1. Phân tích yêu cầu (Analysis) 28
2. Viết Prompt thực hiện (Prompting) 28
3. Đánh giá & Kiểm tra (Checklist) 28
HOẠT ĐỘNG 2: XÂY DỰNG HỆ THỐNG LƯỚI (PORTFOLIO CÁ NHÂN) 28
1. Phân tích yêu cầu (Analysis) 28
2. Viết Prompt thực hiện (Prompting) 28
3. Đánh giá kết quả (Evaluation) 28
HOẠT ĐỘNG 3: DEBUG & REFACTOR (TỐI ƯU MÃ NGUỒN) 29
1. Phân tích yêu cầu (Analysis) 29
2. Viết Prompt thực hiện (Prompting) 29
CÁCH KIỂM TRA CÔNG VIỆC (Final Checklist) 29
PHẦN MỞ RỘNG 29
BÀI TẬP 1: THỬ THÁCH "HOLY GRAIL LAYOUT" (GRID MASTER) 29
BÀI TẬP 2: THIẾT KẾ "BẢNG GIÁ DỊCH VỤ" (FLEXBOX CARDS) 30
BÀI TẬP 3: LƯỚI ẢNH TỰ DO (CSS GRID MASONRY-LIKE) 30
BÀI TẬP 4: THANH ĐIỀU HƯỚNG PHỨC HỢP (COMPLEX NAV BAR) 30
BÀI TẬP 5: TRANG CHI TIẾT SẢN PHẨM (PRODUCT PAGE LAYOUT) 30
BÀI TẬP 6: DASHBOARD QUẢN TRỊ (ADMIN PANEL UI) 30
YÊU CẦU NỘP BÀI (DÀNH CHO TẤT CẢ CÁC BÀI) 31
THÁCH THỨC TẠI NHÀ (NHÓM): "THE GRID MASTER" 32
TUẦN 4: DESIGN SYSTEM & TƯ DUY RESPONSIVE 33
TUẦN 4: DESIGN SYSTEM & TƯ DUY RESPONSIVE (MOBILE FIRST) 33
1. Mục tiêu bài học (Lesson Objectives) 33

2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO) 33
3. Nội dung chuẩn bị (Ở nhà - Flipped Classroom) 33
4. Phần thực hành tại lớp (In-class Activities) 34
Hoạt động 3: "Responsive Breakpoint Challenge" (30 phút) 35
Hoạt động 4: "Design System Refactoring" (30 phút) 35
Bài tập về nhà tuần 4 36
BÀI TẬP TỰ RÈN LUYỆN TẠI NHÀ (HOMEWORK) 36
Bài 1: Responsive Typography Scale 36
Bài 2: Adaptive Card Layout 36
Bài 3: Smart Navigation Bar 36
Bài 4: Dark/Light Mode Theme System 36
Bài 5: Responsive Image Gallery with CSS Grid 36
Bài 6: Mobile-First Contact Form 36
📥 YÊU CẦU NỘP BÀI 36
Thực hành tuần 4 - Multi-Device News Portal 37
Chủ đề bài tập: "Multi-Device News Portal" 37
1. Mục tiêu bài học 37
2. Mô tả yêu cầu Layout (Wireframe) 37
3. Các bước thực hiện chi tiết (Step-by-Step) 37
4. Danh sách kiểm tra hoàn thành (To-do List cho sinh viên) 38
5. Lỗi Prompt cần tránh trong Responsive Design 38
6. Thử thách nâng cao (Bonus) 38
RESPONSIVE CSS CHEAT SHEET (2026 Edition) 38
1. Thẻ Meta bắt buộc (Trong file HTML) 38
2. Các điểm ngắt (Breakpoints) tiêu chuẩn 39
3. Công thức "Vàng" cho Layout linh hoạt 39
4. Bảng đối chiếu "Code Tốt" vs "Code Rác" 40
5. Mẹo Prompt để AI xuất code chuẩn theo Cheat Sheet này: 40
Thách thức tự rèn luyện tại nhà 41
Thách thức tự rèn luyện tại nhà (Hand-on Challenges) 41
Thử thách 1: Dark Mode Switcher (Sơ khai) 41
📝 Checklist hoàn thành Tuần 4 41
TUẦN 5: HIỆU ỨNG CHUYỂN ĐỘNG & THƯ VIỆN HIỆU ỨNG 42
TUẦN 5: HIỆU ỨNG CHUYỂN ĐỘNG & THƯ VIỆN HIỆU ỨNG (CSS ANIMATIONS) 42
1. Mục tiêu bài học (Lesson Objectives) 42
2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO) 42
3. Nội dung lý thuyết (Flipped Classroom) 42

4. Hoạt động thực hành trên lớp (In-class Activities) 42
5. 6 Bài tập tự rèn luyện tại nhà (Homework) 43
YÊU CẦU NỘP BÀI 43
BÀI TẬP TỰ RÈN LUYỆN - TUẦN 5 44
BÀI TẬP TỰ RÈN LUYỆN (HOMEWORK) - TUẦN 5 44
Bài 1: Floating Action Button (FAB) - Nút tương tác nổi 44
Bài 2: Card Flip Effect - Thẻ nhân sự lật mặt 44
Bài 3: Typing Effect - Hiệu ứng máy đánh chữ 44
Bài 4: Parallax Background - Ảnh nền chuyển động sâu 44
Bài 5: Modern Hamburger Menu - Biến đổi biểu tượng 45
Bài 6: Skill Bar Animation - Thanh kỹ năng tự động chạy 45
Checklist nộp bài dành cho sinh viên: 45
THÁCH THỨC TẠI NHÀ: "THE INTERACTIVE SHOWCASE" 46
THÁCH THỨC TẠI NHÀ: "THE INTERACTIVE SHOWCASE" (TUẦN 5) 46
1. Mục tiêu của thách thức 46
2. Yêu cầu cụ thể 46
3. Hướng dẫn và Yêu cầu nộp bài 46
4. Tiêu chí chấm điểm (Rubric) 47
TUẦN 6: HOÀN THIỆN GIAO DIỆN & TỐI ƯU HÓA SẢN PHẨM 48
TUẦN 6: HOÀN THIỆN GIAO DIỆN & TỐI ƯU HÓA SẢN PHẨM (REFACTORING & UI/UX FINALIZATION) 48
1. Mục tiêu bài học (Lesson Objectives) 48
2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO) 48
3. Nội dung lý thuyết tự học (Flipped Classroom) 48
4. Hoạt động thực hành trên lớp (In-class Activities) 48
6. Danh mục Checklist hoàn thiện Tuần 6 49
HƯỚNG DẪN HOẠT ĐỘNG TẠI LỚP (IN-CLASS) 50
Hoạt động 1: "The Bug Hunter" (Săn lỗi giao diện) 50
Hoạt động 2: "Speed Up!" (Tối ưu hóa hiệu năng) 50
Hoạt động 3: Workshop "Showcase & Peer Review" 50
CHUỖI BÀI TẬP TỰ RÈN LUYỆN (HOMEWORK) 50
Bài 1: Xây dựng Trang mục lục (Index Hub) 50
Bài 2: Tối ưu hóa chuẩn W3C (Validation) 50
Bài 3: Responsive Navigation hoàn chỉnh 51
Bài 4: Tích hợp thư viện AOS chuyên sâu 51
Bài 5: Nâng cấp Footer chuyên nghiệp 51
Bài 6: Viết README.md chuyên nghiệp 51

📋 CHECKLIST HOÀN THIỆN TRƯỚC TUẦN 7 51
Thách thức tổng kết giai đoạn 52
Thách thức tổng kết giai đoạn (Hackathon Mini tại nhà) 52
TUẦN 7: ĐÁNH GIÁ GIỮA KỲ 53
TUẦN 7: ĐÁNH GIÁ GIỮA KỲ - "FRONT-END WARRIOR HACKATHON" 53
1. Mục tiêu buổi đánh giá (Assessment Objectives) 53
2. Chuẩn đầu ra đánh giá (Assessment Learning Outcomes - ALO) 53
3. Cấu trúc bài thi thực hành (Thời lượng: 150 phút) 53
4. Tiêu chí chấm điểm (Rubric) 53
🚀 HƯỚNG DẪN ÔN TẬP CHO SINH VIÊN (DÀNH CHO TUẦN 7) 54
📥 HÀNH ĐỘNG DÀNH CHO GIẢNG VIÊN 54
Hỗ trợ giáo viên 55
DANH SÁCH ĐỀ THI 55
Đề 1: Trang Giới thiệu Dự án Công nghệ (Tech Launchpad) 55
Đề 2: Trang Portfolio Nghệ thuật (Creative Studio) 55
Đề 3: Trang Đặt chỗ Nhà hàng/Quán Cà phê (Smart Bistro) 55
🛠️ 2. ĐOẠN MÃ MẪU (STARTER TEMPLATE) 57
Cấu trúc thư mục (Folder Structure): 57
Mã mẫu cơ bản trong style.css: 57
🔍 3. HƯỚNG DẪN QUAN SÁT & ĐÁNH GIÁ TƯ DUY (OBSERVATION GUIDE) 58
📝 4. BIỂU MẪU GIẢI TRÌNH (REPORT.MD TEMPLATE) 58
TUẦN 8: JAVASCRIPT FUNDAMENTALS & TƯ DUY 59
TUẦN 8: JAVASCRIPT FUNDAMENTALS & TƯ DUY LOGIC AI-FIRST 59
1. Mục tiêu bài học (Lesson Objectives) 59
2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO) 59
3. Nội dung lý thuyết tự học (Flipped Classroom) 59
4. Hoạt động thực hành trên lớp (In-class Activities) 59
5. 6 Bài tập tự rèn luyện tại nhà (Homework) 60
🎯 THÁCH THỨC TUẦN 8: "THE LOGIC PUZZLE" (NHÓM) 61
🎯 THÁCH THỨC TUẦN 8: "THE LOGIC PUZZLE" (NHÓM) 61
HƯỚNG DẪN THỰC HÀNH CHI TIẾT (STEP-BY-STEP) 62
HƯỚNG DẪN THỰC HÀNH CHI TIẾT (STEP-BY-STEP) - TUẦN 8 62
HOẠT ĐỘNG 1: "HELLO JS CONSOLE" - ĐỐI THOẠI VỚI TRÌNH DUYỆT 62
1. Phân tích yêu cầu 62
2. Viết Prompt thực hiện (Thực hành cùng AI) 62
3. Đánh giá & Kiểm tra (Checklist) 62

HOẠT ĐỘNG 2: "LOGIC BUILDER" - MÁY TÍNH XẾP LOẠI MINI 62
1. Phân tích yêu cầu 62
2. Viết Prompt thực hiện (Thực hành cùng AI) 62
3. Đánh giá & Kiểm tra (Checklist) 62
HOẠT ĐỘNG 3: "AI LOGIC EXPLAINER" - GIẢI MÃ MÃ NGUỒN 62
1. Phân tích yêu cầu 62
2. Viết Prompt thực hiện (Thực hành cùng AI) 63
3. Cách kiểm tra công việc (Final Checklist) 63
Gợi ý cho giảng viên: 63
TUẦN 9: TỰ ĐỘNG HÓA VỚI VÒNG LẶP 64
TUẦN 9: TỰ ĐỘNG HÓA VỚI VÒNG LẶP & TƯ DUY ĐÓNG GÓI VỚI HÀM 64
1. Mục tiêu bài học (Lesson Objectives) 64
2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO) 64
3. Nội dung lý thuyết tự học (Flipped Classroom) 64
4. Hoạt động thực hành trên lớp (In-class Activities) 64
5. 6 Bài tập tự rèn luyện tại nhà (Homework) 65
THÁCH THỨC TUẦN 9: "THE SMART INTERFACE" (NHÓM) 65
TUẦN 10: THAO TÁC VỚI DOM & XỬ LÝ SỰ KIỆN 67
TUẦN 10: THAO TÁC VỚI DOM & XỬ LÝ SỰ KIỆN (DOM MANIPULATION) 67
1. Mục tiêu bài học 67
2. Chuẩn đầu ra bài học (LLO) 67
3. Nội dung lý thuyết tự học (Flipped Classroom) 67
4. Hoạt động thực hành trên lớp (In-class Activities) 67
5. 6 Bài tập tự rèn luyện tại nhà (Homework) 68
THÁCH THỨC TUẦN 10: "THE SMART PROJECT" (NHÓM) 69
TUẦN 11: XỬ LÝ FORM & KIỂM SOÁT DỮ LIỆU 70
TUẦN 11: XỬ LÝ FORM & KIỂM SOÁT DỮ LIỆU (FORM HANDLING & VALIDATION) 70
1. Mục tiêu bài học 70
2. Chuẩn đầu ra bài học (LLO) 70
3. Nội dung lý thuyết tự học (Flipped Classroom) 70
4. Hoạt động thực hành trên lớp (In-class Activities) 70
5. 6 Bài tập tự rèn luyện tại nhà (Homework) 71
THÁCH THỨC TUẦN 11: "THE SECURE FORM" (NHÓM) 71
TUẦN 12: QUẢN LÝ DỮ LIỆU VỚI MẢNG NÂNG CAO 72
TUẦN 12: QUẢN LÝ DỮ LIỆU VỚI MẢNG NÂNG CAO & ĐỐI TƯỢNG (DATA MANAGEMENT) 72
1. Mục tiêu bài học 72

2. Chuẩn đầu ra bài học (LLO) 72
3. Nội dung lý thuyết tự học (Flipped Classroom) 72
4. Hoạt động thực hành trên lớp (In-class Activities) 72
5. 6 Bài tập tự rèn luyện tại nhà (Homework) 73
THÁCH THỨC TUẦN 12: "THE SMART CATALOG" (NHÓM) 73
PBL - Agile (Scrum) 74
TUẦN 13: KHỞI ĐỘNG DỰ ÁN CUỐI KHÓA (PBL - PHASE 1: PLANNING & PROTOTYPING) 74
1. Mục tiêu bài học (Lesson Objectives) 74
2. Chuẩn đầu ra bài học (LLO) 74
3. Quy trình triển khai PBL (Project Lifecycle) 74
4. Hoạt động thực hành trên lớp (In-class Activities) 74
5. Thách thức dành cho dự án (Project Challenges) 75
6. Danh mục Checklist cho sinh viên (PBL Milestone 1) 75
Lời khuyên cho thầy (The Mentor's Tip): 75
TUẦN 13: KHỞI ĐỘNG DỰ ÁN & SPRINT 1 (PLANNING & CORE UI) 75
1. Lý thuyết (45 phút): Agile Scrum & Vibe Coding Workflow 75
2. Thực hành PBL - Giai đoạn "Khám phá & Thiết kế" 76
TUẦN 14: SPRINT 2 - LOGIC THÔNG MINH & KIỂM THỬ (FINAL PUSH) 76
1. Lý thuyết (30 phút): Agile Review & Refactoring 76
2. Thực hành PBL - Giai đoạn "Thực thi & Tinh chỉnh" 76
TIÊU CHÍ "VIBE CODE" DÀNH CHO GIẢNG VIÊN (MENTOR) 76
Phương pháp giảng dạy Project base Learning 78
PHƯƠNG PHÁP GIẢNG DẠY 78
PROJECT BASE LEARNING 78
1. DÀNH CHO GIÁO VIÊN (VAI TRÒ MENTOR & PRODUCT OWNER) 78
1.1. Quy trình chuẩn bị dự án 78
1.2. Các giai đoạn hướng dẫn trong PBL 78
1.3. Tích hợp Agile/Scrum 78
2. DÀNH CHO SINH VIÊN (VAI TRÒ WEB DEVELOPER & SCRUM TEAM) 78
2.1. 7 Yếu tố cốt lõi của một dự án PBL thành công 78
2.2. Quy trình "Vibe Coding" trong Agile 78
3. SỰ KẾT HỢP GIỮA PBL VÀ QUY TRÌNH AGILE (HÀNH ĐỘNG CỤ THỂ) 79
Agile & Scrum Framework 79
1. AGILE LÀ GÌ? (TƯ DUY LINH HOẠT) 79
2. SCRUM FRAMEWORK (KHUNG LÀM VIỆC NHÓM) 79
2.1. 3 Vai trò chính (Roles) 79

2.2. 3 Tạo vật quan trọng (Artifacts) 80
2.3. C4 Sự kiện chính (Events) 80
3. TẠI SAO PBL NÊN KẾT HỢP VỚI SCRUM? 80
TÓM TẮT CHO GIẢNG VIÊN & SINH VIÊN 80
Dự án mẫu 81
1. SPRINT 1 – THIẾT LẬP CẤU TRÚC & GIAO DIỆN CỐT LÕI 81
1.1. Vai trò trong nhóm (Scrum Roles): 81
1.2. 2. Hoạt động Vibe Coding (Làm việc với AI): 81
2. SPRINT 2 – JAVASCRIPT LOGIC & TƯƠNG TÁC THÔNG MINH 81
2.1. Quy trình Agile (Daily Stand-up): 81
2.2. Hoạt động Vibe Coding (Làm việc với AI): 81
2.3. Kết quả 81
3. HACKATHON – CÔNG BỐ & TRÌNH DIỄN (DEMO DAY) 81
3.1. Hoạt động chính: 81
3.2. Tiêu chí đánh giá "Vibe Code": 81
BẢNG PHÂN CHIA CÔNG VIỆC VÍ DỤ (BACKLOG) 82
Phân tích chi tiết theo SCRUM 83
DỰ ÁN: LHU STUDENT PORTAL (HỆ THỐNG QUẢN LÝ HỌC VỤ) 83
1. SPRINT 1 – THIẾT LẬP CẤU TRÚC & GIAO DIỆN CỐT LÕI 83
1.1. Vai trò trong nhóm (Scrum Roles): 83
1.2. Hoạt động Vibe Coding (Làm việc với AI): 83
1.3. Kết quả cuối tuần 13: Giao diện tĩnh (HTML/CSS) hoàn thiện, cấu trúc dữ liệu JSON sẵn sàng 83
2. SPRINT 2 – JAVASCRIPT LOGIC & TƯƠNG TÁC THÔNG MINH 83
2.1. Quy trình Agile (Daily Stand-up): 83
2.2. Hoạt động Vibe Coding (Làm việc với AI): 83
2.3. Kết quả cuối tuần 14: Website đã có thể tính điểm tự động, tìm kiếm môn học và hiển thị thông báo "Cảnh báo học vụ" nếu điểm GPA < 2.0 (sử dụng cấu trúc If-Else). 83
3. HACKATHON – CÔNG BỐ & TRÌNH DIỄN (DEMO DAY) 83
3.1. Hoạt động chính: 83
3.2. Tiêu chí đánh giá "Vibe Code": 84
BẢNG PHÂN CHIA CÔNG VIỆC VÍ DỤ (BACKLOG) 84
Phương phap phân chia vai trò 85
PHƯƠNG PHÁP PHÂN CHIA VAI TRÒ TRONG NHÓM SCRUM 85
1. CHIẾN LƯỢC CHIA VIỆC ĐỂ TRÁNH CHỒNG CHÉO 85
1.1. Thống nhất "Hợp đồng dữ liệu" (Data Contract) - Quan trọng nhất 85
1.2. Chia theo lớp (Layers) và Module 85
2. VÍ DỤ CỤ THỂ: DỰ ÁN QUẢN LÝ HỌC VỤ 85

2.1. Bước 1: Thiết lập cấu trúc "Xương cá" (Tuần 13 - Sprint 1) 85
2.2. Bước 2: Phân chia Task theo "Luồng AI" (Tuần 14 - Sprint 2) 85
● Sinh viên A (Làm Giao diện tĩnh): 85
3. QUY TẮC "VÀNG" ĐỂ GIÁO VIÊN HƯỚNG DẪN SINH VIÊN 86
4. BẢNG PHÂN CHIA CÔNG VIỆC VÍ DỤ (BACKLOG) 86
PHỤ LỤC 86
Phụ lục 1 - Danh sách đề tài 87
PHỤ LỤC 1 87
DANH SÁCH CHỦ ĐỀ 87
1. THÔNG TIN CHUNG VỀ CHỦ ĐỀ 87
1.1. Yêu cầu chung 87
1.2. Các nhóm đề tài gợi ý 87
1.3. Yêu cầu kỹ thuật (Sản phẩm cuối cùng) 87
1.4. Các trang tối thiểu cần có trong dự án 87
1.5. Hồ sơ nộp bài (Checklist) 88
2. DANH SÁCH CÁC CHỦ ĐỀ 88
2.1. DANH SÁCH 5 CHỦ ĐỀ: HỆ THỐNG QUẢN LÝ HỌC VỤ (STUDENT PORTAL) 88
2.2. DANH SÁCH 5 CHỦ ĐỀ: THƯƠNG MẠI ĐIỆN TỬ (E-COMMERCE) 89
2.3. DANH SÁCH 5 CHỦ ĐỀ: QUẢN TRỊ DOANH NGHIỆP (ERP) 91
2.4. DANH SÁCH 5 CHỦ ĐỀ: CÁ NHÂN HÓA ĐÀO TẠO (EDTECH) 93
2.5. DANH SÁCH 5 CHỦ ĐỀ: PHÁT TRIỂN BỀN VỮNG 94
Phụ lục 2 - Nhật ký đồng hành cùng AI 97
PHỤ LỤC 2 97
NHẬT KÝ ĐỒNG HÀNH CÙNG AI (PROMPT LOG) 97
1. Thông tin công cụ AI sử dụng 97
2. Nhật ký quá trình thực hiện (Các Prompt quan trọng) 97
3. Những bài học kinh nghiệm khi dùng AI 97
4. Cam kết của nhóm 98
5. Lưu ý 98
Phụ lục 3 - Cấu trúc kho lưu trữ tiêu chuẩn 99
PHỤ LỤC 3 99
CÁCH TỔ CHỨC CẤU TRÚC LƯU TRỮ TIÊU CHUẨN TRÊN GITHUB 99
1. CẤU TRÚC KHO LƯU TRỮ (REPOSITORY) TIÊU CHUẨN 99
Sơ đồ thư mục đề xuất: 99
2. QUY TRÌNH PHỐI HỢP (GIT WORKFLOW) 99
● Bước 1: Phân nhánh (Branching) 99
● Bước 2: Viết Code & Commit 99

- Bước 3: Pull Request (PR) & Review 99

3. SỬ DỤNG GITHUB PROJECTS ĐỂ QUẢN LÝ AGILE 100

4. NỘI DUNG FILE README.MD (QUAN TRỌNG) 100

Phụ lục 4 Rubric chấm điểm học phần 101

PHỤ LỤC 4 101

RUBRIC CHẤM ĐIỂM CHI TIẾT (THANG ĐIỂM 10) 101

Cách tính điểm định lượng cho từng tiêu chí: 102

Outline

KHUNG NỘI DUNG 12 TUẦN (PHƯƠNG PHÁP LỚP HỌC ĐẢO NGƯỢC)

GIAI ĐOẠN 1: THIẾT LẬP NỀN TẢNG & CÔNG CỤ (Tuần 1 - 2)

- **Tuần 1: Nhập môn & Hệ sinh thái AI Web Development**
  - **Nội dung chuẩn bị (Ở nhà):** Xem video hướng dẫn cài đặt VS Code, Vibe Code, Git và đăng ký tài khoản GitHub.
  - **Hoạt động trên lớp (In-class):** Thực hiện thử thách "Git nhanh" trong 15 phút; Chia nhóm, bốc thăm 1 trong 20 chủ đề dự án xuyên suốt.
- **Tuần 2: Tư duy Semantic HTML5 & Cấu trúc Web**
  - **Nội dung chuẩn bị (Ở nhà):** Tìm hiểu các thẻ Semantic (header, footer, main, section) và khái niệm SEO cơ bản.
  - **Hoạt động trên lớp (In-class):** Dùng AI phân tích hình ảnh giao diện thực tế thành cấu trúc DOM; Thực hành Bài 1: Cấu trúc hóa bản sắc cá nhân.

GIAI ĐOẠN 2: HỆ THỐNG GIAO DIỆN & RESPONSIVE (Tuần 3 - 6)

- **Tuần 3: CSS Layout System (Flexbox & Grid)**
  - **Nội dung chuẩn bị (Ở nhà):** Xem video về mô hình Box Model, Flexbox và CSS Grid.
  - **Hoạt động trên lớp (In-class):** Thực hành Bài 2: Xây dựng hệ thống lưới cho mục "Kỹ năng" và "Dự án" bằng AI; Phản biện mã CSS AI vừa sinh ra.
- **Tuần 4: Design System & CSS Variables**
  - **Nội dung chuẩn bị (Ở nhà):** Tìm hiểu cách sử dụng biến CSS (`--primary-color`) và xây dựng bảng màu, typography.
  - **Hoạt động trên lớp (In-class):** Xây dựng hệ thống UI đồng nhất; Thực hành thay đổi 1 biến để cập nhật toàn bộ giao diện (Tư duy hệ thống).
- **Tuần 5: Responsive Design (Mobile First)**
  - **Nội dung chuẩn bị (Ở nhà):** Nghiên cứu về Media Queries và kỹ thuật ẩn hiện thành phần trên di động.
  - **Hoạt động trên lớp (In-class):** Thực hành Bài 3: Biến đổi Menu ngang thành Menu Hamburger bằng Vibe Code; Kiểm tra giao diện trên nhiều thiết bị.
- **Tuần 6: Thực hành tổng hợp Giai đoạn 1**
  - **Hoạt động trên lớp (In-class):** Hoàn thiện giao diện Landing Page tĩnh; Chuẩn bị Portfolio để giảng viên review chéo.

TUẦN 7: ĐÁNH GIÁ GIỮA KỲ (KIỂM TRA NĂNG LỰC)

- **Hình thức:** Thực hành 120 phút với chủ đề "AI Coffee Shop".
- **Trọng tâm:** Kiểm tra kỹ năng Prompting, tư duy Responsive và cách quản lý file `style.css` bằng biến.

GIAI ĐOẠN 3: LOGIC & TƯƠNG TÁC THỰC CHIẾN (Tuần 8 - 12)

- **Tuần 8: JavaScript ES6 & DOM cơ bản**
  - **Nội dung chuẩn bị (Ở nhà):** Tìm hiểu cú pháp JS cơ bản (biến, hàm, mảng) và cách truy cập phần tử DOM.
  - **Hoạt động trên lớp (In-class):** Dùng AI viết hàm xử lý sự kiện click; Thực hành sơ đồ luồng (Flowchart) cho các hàm AI vừa sinh.

- **Tuần 9: Theme Engine & Real-time Interaction**
  - **Nội dung chuẩn bị (Ở nhà):** Tìm hiểu cách JS thay đổi thuộc tính CSS và Class của HTML.
  - **Hoạt động trên lớp (In-class):** Thực hành Bài 4: Xây dựng trình chỉnh sửa giao diện Dark/Light mode thời gian thực.
- **Tuần 10: Data Interaction (Google Sheets/Excel)**
  - **Nội dung chuẩn bị (Ở nhà):** Tìm hiểu cách cấu trúc dữ liệu JSON và cách lấy dữ liệu từ một link công khai.
  - **Hoạt động trên lớp (In-class):** Thực hành Bài 5: Hiển thị danh sách dự án lấy từ Google Sheets thay vì viết cứng vào HTML.
- **Tuần 11: Animation & Smart Components**
  - **Nội dung chuẩn bị (Ở nhà):** Tìm hiểu về thư viện hiệu ứng (GSAP/Anime.js) và kỹ thuật lọc dữ liệu (Filter).
  - **Hoạt động trên lớp (In-class):** Thực hành Bài 6: Tích hợp hiệu ứng cuộn trang và bộ lọc dự án theo danh mục (Web/Mobile).
- **Tuần 12: Tư duy phản biện & Tối ưu hóa Logic**
  - **Nội dung chuẩn bị (Ở nhà):** Nghiên cứu cách tối ưu hóa code và xử lý lỗi (Debug) cơ bản.
  - **Hoạt động trên lớp (In-class):** So sánh code AI vs Code tối ưu; Kiểm tra validation form thông minh cho sản phẩm cá nhân.

Tuần 1: Nhập môn & Hệ sinh thái AI Web Development

PHẦN 1 - MỤC TIÊU VÀ CHUẨN ĐẦU RA BÀI HỌC

1. Mục tiêu bài học (Lesson Objectives)

Sau khi kết thúc buổi học này, sinh viên sẽ:

- **Về kiến thức:** Hiểu rõ lộ trình phát triển của chuỗi 4 học phần phát triển ứng dụng Web và vai trò của AI trong quy trình lập trình hiện đại.
- **Về kỹ năng:** Thiết lập được môi trường làm việc chuyên nghiệp (VS Code, Vibe Code, Git) và thực hiện được các thao tác quản lý mã nguồn nhóm cơ bản trên GitHub.
- **Về thái độ:** Hình thành tư duy làm chủ công nghệ, có trách nhiệm với mã nguồn do AI sinh ra và sẵn sàng cộng tác trong môi trường dự án nhóm.

2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO)

Sau tuần học này, sinh viên có khả năng:

- **LLO 1.1.1:** Giải thích được vai trò và sự phối hợp giữa các công cụ trong hệ sinh thái AI Web Development (VS Code, AI Assistant, Git, GitHub).
- **LLO 1.1.2:** Cấu hình thành thạo môi trường lập trình và kết nối thành công tài khoản GitHub cá nhân với trình soạn thảo mã nguồn.
- **LLO 1.1.3:** Thực hiện chính xác quy trình Git Flow cơ bản bao gồm: Clone, Branch, Add, Commit, Push và xử lý xung đột (Conflict) ở mức độ đơn giản trong nhóm.
- **LLO 1.1.4:** Vận dụng tư duy phân rã bài toán để viết được các Prompt (câu lệnh) đầu tiên yêu cầu AI thiết lập cấu trúc dự án.
- **LLO 1.1.5:** Thể hiện kỹ năng làm việc nhóm thông qua việc bốc thăm chủ đề, phân chia vai trò (Leader/Member) và thiết lập bảng tiến độ công việc trên GitHub Projects.

3. Ma trận ánh xạ Chuẩn đầu ra (Mapping)

Chuẩn đầu ra bài học (LLO)	Ánh xạ CLO	Hoạt động giảng dạy	Thành phần đánh giá
LLO 1.1.1	CLO1	Giảng giải & Thảo luận hệ sinh thái công nghệ.	Vấn đáp trên lớp.
LLO 1.1.2	CLO1	Thực hành hướng dẫn cài đặt trực tiếp.	Kiểm tra môi trường máy tính.
LLO 1.1.3	CLO1, CLO3	Thử thách "Git nhanh" trong 15 phút.	Nhật ký Commit trên GitHub.
LLO 1.1.4	CLO1, CLO4	Thực hành viết Prompt khởi tạo dự án.	File prompt.txt.
LLO 1.1.5	CLO3	Hoạt động bốc thăm và lập nhóm dự án.	Danh sách nhóm & GitHub Repo.

PHẦN 2: LÝ THUYẾT

1. Hệ sinh thái Công cụ (The Modern Toolstack)

Sinh viên cần hiểu vai trò của từng công cụ trong quy trình phát triển Web hiện đại:

- **VS Code (Editor):** Tại sao đây là lựa chọn hàng đầu cho lập trình viên Web? Tìm hiểu sơ lược về các tính năng như IntelliSense và Extension.
- **Vibe Code/Cursor (AI Coding Assistant):** Khái niệm về lập trình có sự hỗ trợ của AI (AI-Assisted Development). AI đóng vai trò là "trợ lý" thực thi, còn con người đóng vai trò là "kiến trúc sư" ra lệnh.
- **Git & GitHub (Version Control):** Phân biệt giữa Git (công cụ quản lý trên máy cục bộ) và GitHub (nền tảng lưu trữ đám mây và cộng tác nhóm).

2. Tư duy Lập trình thời đại AI (AI-First Mindset)

Thay vì học thuộc cú pháp, sinh viên cần tìm hiểu về:

- **Prompt Engineering cho Lập trình:** Cách mô tả một yêu cầu kỹ thuật bằng ngôn ngữ tự nhiên sao cho AI hiểu và sinh mã chính xác.
- **Quy trình Phân rã bài toán (Decomposition):** Cách chia nhỏ một giao diện lớn thành các phần nhỏ (Header, Footer, Content) để ra lệnh cho AI hiệu quả hơn.

3. Nền tảng Quản lý mã nguồn nhóm (Git Workflow)

Để thực hiện được thử thách "Git nhanh", sinh viên phải nắm vững các khái niệm:

- **Repository (Kho lưu trữ):** Nơi chứa toàn bộ mã nguồn của dự án.
- **Commit:** Cách ghi lại một "phiên bản" hoặc một dấu mốc thay đổi của mã nguồn kèm thông điệp mô tả.
- **Branch (Nhánh):** Hiểu tại sao phải chia nhánh (Main, Develop, Feature) để nhiều người có thể làm việc cùng lúc mà không ghi đè lên nhau.
- **Pull Request (PR) & Merge:** Quy trình gửi mã nguồn đã hoàn thiện để trưởng nhóm kiểm tra và gộp vào dự án chung.

4. Khái niệm cơ bản về cấu trúc Web

- **Sự phối hợp giữa HTML - CSS - JavaScript:** Hiểu sơ lược về vai trò của từng ngôn ngữ (HTML xây khung, CSS làm đẹp, JS tạo tương tác) để biết mình đang yêu cầu AI làm gì.
- **Semantic HTML:** Tại sao việc sử dụng các thẻ đúng ý nghĩa (như `<header>`, `<nav>`, `<article>`) lại quan trọng hơn việc chỉ dùng thẻ `<div>` đơn thuần.

5. Văn hóa làm việc nhóm (Agile/Scrum cơ bản)

- **Phân chia Task:** Cách sử dụng GitHub Projects hoặc các bảng Kanban đơn giản để quản lý công việc.
- **Đạo đức sử dụng AI:** Hiểu rằng sinh viên là người chịu trách nhiệm cuối cùng cho mã nguồn, AI chỉ là công cụ hỗ trợ.

PHẦN 3 - THỰC HÀNH - KÍCH HOẠT HỆ SINH THÁI VIBE CODE

Thời lượng: 3 tiết (khoảng 135 phút)

Hoạt động 1: Kiểm tra & Đồng bộ môi trường (20 phút)

> - **Nhiệm vụ:** Đảm bảo mọi máy tính của sinh viên đều sẵn sàng "tác chiến".
- **Chi tiết:**
  - Giảng viên kiểm tra nhanh việc cài đặt **VS Code**, **Git**, và các Extension AI (**Vibe Code**: Gemini Assist code| Github Copilot chatbot, Google Anti-Gravity).
  - Hướng dẫn sinh viên đăng nhập GitHub trên VS Code và cấu hình `user.name`, `user.email`.
- **Sản phẩm:** Một đoạn mã đơn giản (Hello World) được sinh bởi AI và hiển thị trên trình duyệt.

Hoạt động 2: Thử thách "Git nhanh" - Xây biệt thự Web (30 phút)

Đây là hoạt động trọng tâm để đạt được **LLO 1.1.3**.

- **Kịch bản:** Nhóm 3 người (Leader, Member A, Member B) cùng xây dựng một Landing Page cực kỳ đơn giản.
- **Nội dung thực hiện:**
  1. **Leader:** Khởi tạo Repo, Push khung HTML lên `main`.
  2. **Member A & B:** Clone về, mỗi người tạo một nhánh `feature/` riêng để thêm nội dung khác nhau (ví dụ: sân trước, sân sau).
  3. **Tạo xung đột:** Member B cố tình sửa đè vào phần code của Member A.
  4. **Xử lý:** Leader hướng dẫn nhóm giải quyết **Conflict** ngay trên VS Code để hiểu về tư duy quản lý phiên bản.

Hoạt động 3: Hình thành dự án xuyên suốt (45 phút)

> Hoạt động này giúp đạt được **LLO 1.1.5** và chuẩn bị cho chuỗi học phần dài hạn.

- **Bốc thăm chủ đề:** Trưởng nhóm bốc thăm 1 trong **20 chủ đề** đã chuẩn bị (ví dụ: Web đặc sản Đồng Nai, Quản lý chi tiêu...).
- **Thiết lập dự án nhóm:**
  - Tạo Organization/Repository chung trên GitHub.
  - Sử dụng **GitHub Projects** (Kanban Board) để tạo các cột: *To Do, In Progress, Review, Done*.
  - Phân chia ít nhất 3 Task đầu tiên cho các thành viên.
- **Sản phẩm:** Link Repository nhóm và bảng quản lý công việc đã có các Task đầu tiên.

Hoạt động 4: Buổi "Vibe Prompting" đầu tiên (30 phút)

Ứng dụng **LLO 1.1.4** để khởi tạo dự án thực tế.

> - **Nhiệm vụ:** Sử dụng AI để sinh ra cấu trúc thư mục và file README chuẩn cho dự án vừa bốc thăm.
- **Yêu cầu Prompt:**
  - Sinh viên phải viết câu lệnh mô tả dự án, yêu cầu AI đề xuất cấu trúc thư mục (css, js, assets, index.html).
  - Tạo file `README.md` mô tả: Tên dự án, thành viên nhóm, và các công nghệ sử dụng.

- **Lưu trữ:** Sinh viên lưu lại câu lệnh này vào file `prompt.txt` để nộp bài.

📋 Check-list hoàn thành buổi thực hành (Nộp bài)

Cuối buổi, mỗi nhóm phải gửi link GitHub chứa:

1. Cấu trúc thư mục dự án hoàn chỉnh (đã Push lên nhánh `develop`).
2. File `README.md` giới thiệu dự án đầy đủ.
3. Lịch sử Commit thể hiện sự tham gia của tất cả thành viên.
4. File `prompt.txt` chứa các câu lệnh AI đã sử dụng.

TUẦN 2: TƯ DUY SEMANTIC HTML5 & CẤU TRÚC WEB

TUẦN 2: TƯ DUY SEMANTIC HTML5 & CẤU TRÚC WEB HỆ THỐNG

1. Mục tiêu bài học (Lesson Objectives)

Sau khi kết thúc tuần này, sinh viên sẽ:

- **Về kiến thức:** Hiểu rõ tầm quan trọng của cấu trúc trang web theo chuẩn Semantic HTML5 và tác động của nó đến SEO và khả năng tiếp cận (Accessibility).
- **Về kỹ năng:** Sử dụng AI để phân tích và chuyển đổi một bản vẽ giao diện (Wireframe/Mockup) thành mã nguồn HTML5 có cấu trúc chặt chẽ.
- **Về thái độ:** Nhận thức được tầm quan trọng của việc viết mã "sạch" và có ý nghĩa thay vì lạm dụng các thẻ vô nghĩa như `<div>`.

2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO)

Sau tuần học này, sinh viên có khả năng:

- **LLO 1.2.1:** Phân loại và sử dụng đúng các thẻ Semantic (header, nav, main, section, article, aside, footer) cho từng thành phần giao diện.
- **LLO 1.2.2:** Vận dụng kỹ năng Prompting để yêu cầu AI sinh mã HTML5 theo cấu trúc lồng nhau (nesting) chính xác và hợp lệ.
- **LLO 1.2.3:** Phân tích một ảnh chụp giao diện thực tế và phác thảo được sơ đồ cấu trúc DOM (Document Object Model) tương ứng.
- **LLO 1.2.4:** Thực hiện việc đẩy mã nguồn (Push) các thành phần giao diện lên các nhánh tính năng (Feature branch) theo quy trình Git nhóm đã học.

3. Nội dung chuẩn bị (Ở nhà - Flipped Classroom)

Sinh viên cần tìm hiểu các điểm lý thuyết sau qua tài liệu giảng viên cung cấp:

- **Khái niệm Semantic Web:** Tại sao trình duyệt và các bộ máy tìm kiếm (Google) cần hiểu ý nghĩa của các thẻ?
- **Các thẻ nội dung phổ biến:** Phân biệt mục đích sử dụng của `<section>` vs `<article>`, `<ul>` vs `<ol>`.
- **Cấu trúc một file HTML5 hoàn chỉnh:** Các thành phần trong thẻ `<head>` (meta tags, title) giúp website hiển thị tốt trên mạng xã hội.

BẢNG TỔNG HỢP CÁC THẺ HTML SEMANTIC

Nhóm thẻ	Tên thẻ	Ý nghĩa & Vai trò	Link tham khảo (W3Schools)
Bố cục chính	<header>	Chứa nội dung giới thiệu, logo, hoặc thanh điều hướng đầu trang.	Xem tại đây

Nhóm thẻ	Tên thẻ	Ý nghĩa & Vai trò	Link tham khảo (W3Schools)
	<nav>	Định nghĩa một tập hợp các liên kết điều hướng (Menu).	Xem tại đây
	<main>	Chứa nội dung chính và duy nhất của văn bản (không lặp lại ở trang khác).	Xem tại đây
	<footer>	Chứa thông tin chân trang như bản quyền, liên hệ, sơ đồ trang.	Xem tại đây
Phân đoạn nội dung	<section>	Định nghĩa một phần nội dung chung có cùng chủ đề trong văn bản.	Xem tại đây
	<article>	Chứa nội dung độc lập, tự có ý nghĩa (như bài blog, tin tức, bình luận).	Xem tại đây
	<aside>	Chứa nội dung bên lề, liên quan gián tiếp đến nội dung chính (Sidebar).	Xem tại đây
	<details>	Định nghĩa một phần nội dung bổ sung mà người dùng có thể ẩn/hiện.	Xem tại đây
	<summary>	Định nghĩa tiêu đề cho thẻ <details> khi hiển thị.	Xem tại đây
Đồ họa & Đa phương tiện	<figure>	Chứa nội dung độc lập như ảnh, sơ đồ, minh họa.	Xem tại đây
	<figcaption>	Định nghĩa chú thích cho nội dung bên trong thẻ <figure>.	Xem tại đây
Thời gian & Khác	<time>	Định nghĩa ngày tháng hoặc thời gian mà máy tính có thể đọc được.	Xem tại đây

Nhóm thẻ	Tên thẻ	Ý nghĩa & Vai trò	Link tham khảo (W3Schools)
	<mark>	Đánh dấu hoặc làm nổi bật một đoạn văn bản cụ thể.	Xem tại đây

Lời khuyên cho sinh viên khi làm bài Tuần 2:

- **Nguyên tắc "No Div" bừa bãi:** Trước khi dùng thẻ , hãy tự hỏi xem nội dung đó có thể thay bằng <section>, <article> hay <aside> được không.
- **Cấu trúc phân tầng:** Một thẻ <main> nên chứa nhiều thẻ <section>, và trong mỗi <section> có thể chứa các thẻ <article> để phân rã thông tin rõ ràng hơn.
- **Kết hợp AI (Vibe Code):** Khi yêu cầu AI sinh code, hãy dùng các từ khóa trong bảng này vào Prompt (ví dụ: *"Generate a semantic HTML5 layout with a header, a main containing two sections, and a footer"*).

4. Phần thực hành tại lớp (In-class Activities)

Thời lượng: 3 tiết (135 phút)

Hoạt động 1: "X-Ray Vision" - Soi cấu trúc Web (30 phút)

- **Nhiệm vụ:** Tập nhìn xuyên qua hình ảnh để thấy "bộ xương" HTML.
- **Thực hiện:** Giảng viên đưa ra 3 ảnh chụp màn hình (Landing page, Blog, Dashboard). Sinh viên dùng AI hoặc công cụ vẽ để phác thảo sơ đồ cây (Tree structure) của các thẻ Semantic cần dùng.
- **Thảo luận:** Tại sao vùng này nên dùng `<aside>` mà không phải `<div>`?

Hoạt động 2: Prompting & Coding - Bài thực hành số 1 (60 phút)

Đây là bài tập thực hành chính thức đầu tiên: **Cấu trúc hóa bản sắc cá nhân**.

- **Nội dung:** Sinh viên sử dụng Vibe Code để khởi tạo file `index.html` cho dự án Portfolio của mình.
- **Yêu cầu kỹ thuật:**
  - Phải có đầy đủ các vùng: `header` (chứa nav), `main` (chứa ít nhất 3 section), và `footer`.
  - Sử dụng AI để sinh nội dung mẫu (Lorem Ipsum) phù hợp với chủ đề cá nhân.
  - Mã nguồn phải được AI giải thích (comment) rõ vai trò của từng khối Semantic.

Hoạt động 3: Đồng bộ hóa dự án nhóm (45 phút)

- **Nhiệm vụ:** Đưa mã nguồn cá nhân/nhóm lên GitHub theo quy trình chuyên nghiệp.
- **Thực hiện:**
  1. Mỗi thành viên tạo nhánh `feature/html-structure`.
  2. Viết code, commit với message: `[HTML] Setup semantic structure for landing page.`
  3. Tạo Pull Request trên GitHub để trưởng nhóm Review.
  4. Trưởng nhóm kiểm tra xem code có dùng thẻ `<div>` bừa bãi không trước khi Merge.

5. Đánh giá & Checklist hoàn thành

Cuối buổi, giảng viên kiểm tra link GitHub của các nhóm dựa trên:

1. **Tính hợp lệ:** File HTML có báo lỗi cú pháp không?
2. **Tính ngữ nghĩa:** Các thẻ Semantic được dùng đúng vị trí chưa?
3. **Tính kỷ luật:** Lịch sử Commit có thể hiện đúng tiến độ của từng thành viên không?

Câu hỏi gợi ý thảo luận cuối buổi: *"Nếu hôm nay AI sinh ra một đoạn code toàn thẻ , các em sẽ yêu cầu nó sửa lại như thế nào bằng một câu Prompt duy nhất?*

THÁCH THỨC TẠI NHÀ: "KIẾN TRÚC SƯ SEMANTIC"

THÁCH THỨC TẠI NHÀ: "KIẾN TRÚC SƯ SEMANTIC" (TUẦN 2)

🎯 1. Mục tiêu của thách thức

- **Vận dụng:** Chuyển đổi ý tưởng dự án đã bốc thăm tại Tuần 1 thành cấu trúc mã nguồn HTML5 hoàn chỉnh bằng các thẻ ngữ nghĩa.
- **Cộng tác:** Thực hành quy trình làm việc nhóm trên GitHub thông qua các nhánh tính năng (Feature branches) và Pull Requests.
- **Làm chủ AI:** Sử dụng Vibe Code để sinh mã cấu trúc dựa trên các mô tả chi tiết (Prompting).

📋 2. Yêu cầu cụ thể

A. Về cấu trúc kỹ thuật (Semantic HTML)

Các nhóm phải xây dựng file `index.html` cho trang chủ của dự án nhóm với các yêu cầu sau:

- **Sử dụng ít nhất 7 loại thẻ Semantic khác nhau** từ danh sách đã học (ví dụ: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`, `<figure>`, `<time>`).
- **Tối ưu Metadata:** Trong thẻ `<head>`, phải có đầy đủ thẻ `meta description` và `title` chuẩn SEO phù hợp với chủ đề dự án.
- **Tính hợp lệ:** Mã nguồn phải vượt qua trình kiểm tra lỗi (Validator) và không có các thẻ định dạng kiểu cũ (như `<center>`, `<font>`).

B. Về quy trình nhóm (Git Workflow)

- **Phân chia Task:** Trưởng nhóm phân chia mỗi thành viên phụ trách một `<section>` hoặc một vùng nội dung cụ thể trong trang.
- **Làm việc trên nhánh:** Tuyệt đối không code trực tiếp trên nhánh `main`. Mỗi thành viên phải tạo nhánh riêng (ví dụ: `feature/header-nav`, `feature/main-content`).
- **Review chéo:** Ít nhất 1 thành viên khác phải vào nhận xét (Comment) trong Pull Request của bạn mình trước khi Trưởng nhóm nhấn **Merge** vào nhánh `develop`.

📝 3. Hướng dẫn nộp bài

Mỗi nhóm chỉ nộp **01 đại diện** (Trưởng nhóm) thông qua link quản lý của Giảng viên.

Hồ sơ nộp bài gồm:

1. **Link GitHub Repository:** Chứa mã nguồn đã được gộp (Merge) hoàn chỉnh vào nhánh `develop`.
2. **File `prompt_log.md`:** Ghi lại các câu lệnh AI đã dùng để sinh cấu trúc cho từng phần.
3. **Ảnh chụp màn hình (Screenshot):** Chứng minh đã có sự trao đổi/Review trong phần Pull Request trên GitHub.

Gợi ý thực hiện (Tips)

- **Họp nhóm Online:** Nên có một buổi gọi video ngắn (15-20 phút) để thống nhất cấu trúc thư mục chung trước khi mỗi người bắt đầu code nhánh riêng.
- **Giao tiếp với AI:** Nếu AI tạo ra quá nhiều thẻ `<div>`, hãy yêu cầu nó: *"Rewrite the following code using Semantic HTML5 tags like and to improve SEO"*.
- **Kiểm tra:** Các bạn có thể sử dụng công cụ **W3C Markup Validation Service** để tự kiểm tra lỗi cú pháp trước khi nộp.

TUẦN 3: CSS LAYOUT SYSTEM (FLEXBOX & GRID)

1. Mục tiêu bài học (Lesson Objectives)

Sau khi kết thúc tuần này, sinh viên sẽ:

- **Về kiến thức**: Hiểu rõ sự khác biệt và trường hợp sử dụng của Flexbox (1 chiều) và CSS Grid (2 chiều). Nắm vững mô hình Box Model để kiểm soát kích thước thành phần.
- **Về kỹ năng**: Sử dụng Vibe Code để tạo ra các bố cục dàn trang chuyên nghiệp, hiện đại mà không cần dùng đến các kỹ thuật cũ như `float` hay `table`.
- **Về thái độ**: Rèn luyện tính tỉ mỉ trong việc căn chỉnh giao diện và tư duy tối ưu hóa mã nguồn CSS.

2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO)

Sau tuần học này, sinh viên có khả năng:

- **LLO 2.1.1**: Vận dụng Flexbox để căn chỉnh các thành phần bên trong (như Navigation bar, Card content) một cách linh hoạt.
- **LLO 2.1.2**: Thiết lập cấu trúc lưới tổng thể cho website bằng CSS Grid (Header, Sidebar, Main, Footer).
- **LLO 2.1.3**: Giải thích và điều chỉnh các thuộc tính trong Box Model (Padding, Margin, Border, Content) để xử lý khoảng cách giao diện.
- **LLO 2.1.4**: Viết Prompt yêu cầu AI tạo ra các Layout phức tạp và thực hiện tinh chỉnh trực tiếp trên trình duyệt (Inspect Element).

3. Nội dung chuẩn bị (Ở nhà - Flipped Classroom)

Sinh viên cần nghiên cứu các khái niệm sau:

- **Mô hình Box Model**: Cách tính toán kích thước thực tế của một phần tử trên Web.
- **Flexbox Guide**: Các thuộc tính quan trọng như `flex-direction`, `justify-content`, `align-items`.
- **CSS Grid Guide**: Khái niệm về `grid-template-columns`, `grid-template-rows` và `gap`.
- **Link tham khảo**:
  - CSS Flexbox - W3Schools
  - CSS Grid - W3Schools

4. Phần thực hành tại lớp (In-class Activities)

Thời lượng: 3 tiết (135 phút)

Hoạt động 1: "Flex & Grid Battle" (30 phút)

- **Nhiệm vụ**: Giải quyết các bài toán căn chỉnh nhanh.
- **Thực hiện**: Giảng viên đưa ra các thử thách nhỏ (ví dụ: căn giữa một vật thể, tạo menu dàn đều, tạo lưới ảnh 3 cột). Sinh viên dùng AI để giải quyết và giải thích tại sao chọn Flexbox hay Grid cho trường hợp đó.

Hoạt động 2: Bài thực hành số 2 - Xây dựng hệ thống lưới (60 phút)

Đây là bài thực hành tiếp nối từ file HTML tuần trước:

- **Nội dung**: Sử dụng Vibe Code để định dạng CSS cho Portfolio cá nhân.
- **Yêu cầu kỹ thuật**:
  - Sử dụng **CSS Grid** để chia bố cục tổng thể (Main và Sidebar).
  - Sử dụng **Flexbox** cho phần danh sách "Kỹ năng" (Skills) để các thẻ tự động xuống dòng khi thiếu không gian.
  - Thiết lập khoảng cách đồng nhất giữa các khối nội dung bằng thuộc tính `gap`.

Hoạt động 3: Debug & Refactor mã nguồn (45 phút)

- **Nhiệm vụ**: Tối ưu hóa CSS do AI sinh ra.
- **Thực hiện**: Sinh viên so sánh mã nguồn CSS của các nhóm khác nhau. Tìm kiếm các thuộc tính dư thừa mà AI thường thêm vào và tiến hành lược bỏ để mã nguồn gọn nhẹ hơn.

HƯỚNG DẪN THỰC HÀNH CHI TIẾT

HƯỚNG DẪN THỰC HÀNH CHI TIẾT (STEP-BY-STEP)

HOẠT ĐỘNG 1: "FLEX & GRID BATTLE" (CĂN CHỈNH NHANH)

1. Phân tích yêu cầu (Analysis)

- **Mục tiêu:** Căn giữa một hộp nội dung (box) vào chính giữa màn hình cả theo chiều dọc và chiều ngang.
- **Đặc điểm:** Sử dụng Flexbox cho các đối tượng đơn lẻ và CSS Grid cho các bố cục dạng bảng.

2. Viết Prompt thực hiện (Prompting)

- **Prompt mẫu (Flexbox):** *"Hãy viết mã CSS sử dụng Flexbox để căn giữa một thẻ có class là 'container' vào chính giữa trang web theo cả chiều dọc và chiều ngang. Đảm bảo 'container' có chiều cao toàn màn hình (100vh)."*
- **Prompt mẫu (Grid):** *"Tạo một bố cục lưới (CSS Grid) gồm 3 cột bằng nhau, khoảng cách giữa các cột là 20px. Mỗi cột chứa một thẻ có màu nền xám nhạt và bo góc 10px."*

3. Đánh giá & Kiểm tra (Checklist)

- [ ] Đối tượng đã nằm chính giữa màn hình chưa?
- [ ] Khi thay đổi kích thước trình duyệt, các cột có tự co giãn không?
- [ ] Mã nguồn có sử dụng các thuộc tính hiện đại như gap thay vì margin truyền thống không?

HOẠT ĐỘNG 2: XÂY DỰNG HỆ THỐNG LƯỚI (PORTFOLIO CÁ NHÂN)

1. Phân tích yêu cầu (Analysis)

- **Cấu trúc tổng thể:** Sử dụng **CSS Grid** để chia trang thành: Header (trên cùng), Sidebar (bên trái - 25%), Main Content (bên phải - 75%), và Footer (dưới cùng).
- **Khu vực kỹ năng:** Dùng **Flexbox** để các thẻ kỹ năng (HTML, CSS, JS...) tự động sắp xếp và xuống dòng khi màn hình nhỏ (wrap).

2. Viết Prompt thực hiện (Prompting)

- **Prompt 1 (Layout tổng):** *"Dựa trên cấu trúc HTML5 đã có, hãy viết CSS Grid để dàn trang: Header chiếm toàn bộ chiều ngang, bên dưới là Sidebar rộng 250px và Main content chiếm phần còn lại, cuối cùng là Footer. Sử dụng thuộc tính grid-template-areas để quản lý."*
- **Prompt 2 (Skills Section):** *"Hãy viết CSS cho danh mục 'Skills'. Sử dụng Flexbox để các item hiển thị dạng hàng ngang, tự động xuống dòng (flex-wrap), khoảng cách giữa các item là 15px và căn giữa các item."*

3. Đánh giá kết quả (Evaluation)

- Kiểm tra mô hình **Box Model**: Sử dụng Inspect Element (F12) để xem padding và margin có bị chồng chéo không.
- Kiểm tra tính logic: AI có sinh ra các mã thừa như float: left hay không (nếu có, yêu cầu xóa bỏ).

HOẠT ĐỘNG 3: DEBUG & REFACTOR (TỐI ƯU MÃ NGUỒN)

1. Phân tích yêu cầu (Analysis)

- Tìm kiếm các thuộc tính CSS trùng lặp.
- Thay thế các giá trị cứng (ví dụ: 20px) bằng các biến CSS (var(--spacing-unit)) đã học ở các tuần trước (nếu có).

2. Viết Prompt thực hiện (Prompting)

- **Prompt:** *"Đây là đoạn mã CSS do AI sinh ra. Hãy tối ưu hóa nó bằng cách loại bỏ các thuộc tính mặc định không cần thiết, gộp các class có thuộc tính giống nhau và sử dụng Shorthand properties (ví dụ: gộp margin-top/bottom/left/right thành margin)."*

CÁCH KIỂM TRA CÔNG VIỆC (Final Checklist)

STT	Tiêu chí kiểm tra	Kết quả (Đạt/Không)
1	Sử dụng **CSS Grid** cho bố cục chính của trang.	
2	Sử dụng **Flexbox** cho các thành phần con (Menu, Card).	
3	Không còn sử dụng kỹ thuật float để dàn trang.	
4	Khoảng cách giữa các thành phần được quản lý bằng gap.	
5	Mã nguồn CSS sạch, có comment giải thích các khối Grid/Flex.	
6	Đã Push mã nguồn lên nhánh feature/css-layout trên GitHub.	

Mẹo cho sinh viên:

Khi làm việc với **CSS Grid**, hãy sử dụng công cụ **Grid Inspector** trong trình duyệt Chrome/Edge để nhìn thấy các đường kẻ lưới (grid lines), giúp việc căn chỉnh trực quan hơn rất nhiều. Nếu AI sinh code lỗi, hãy chụp màn hình lỗi và gửi cho AI kèm theo yêu cầu: *"Layout đang bị lệch như ảnh, hãy sửa lại thuộc tính grid-area"*.

PHẦN MỞ RỘNG

BÀI TẬP 1: THỬ THÁCH "HOLY GRAIL LAYOUT" (GRID MASTER)

Yêu cầu:

- Xây dựng cấu trúc website kinh điển gồm 5 phần: Header (trên cùng), Sidebar trái (Menu), Main Content (Nội dung chính), Sidebar phải (Quảng cáo) và Footer (dưới cùng).

- Sử dụng **CSS Grid** với thuộc tính `grid-template-areas` để định vị.
- Yêu cầu AI tạo nội dung giả (Placeholder) sao cho khi nội dung ở Main Content dài ra, hai Sidebar vẫn giữ nguyên tỉ lệ chiều cao.

BÀI TẬP 2: THIẾT KẾ "BẢNG GIÁ DỊCH VỤ" (FLEXBOX CARDS)

Yêu cầu:

- Tạo một khu vực gồm 3 bảng giá (Basic, Pro, Enterprise) nằm ngang hàng nhau.
- Sử dụng **Flexbox** để căn giữa các thành phần bên trong mỗi bảng giá (Nút "Mua ngay" luôn nằm sát đáy bảng giá dù nội dung mô tả dài ngắn khác nhau).
- Hiệu ứng: Khi di chuột (hover) vào một bảng giá, bảng đó sẽ phóng to nhẹ (`transform: scale`) và thay đổi màu sắc nổi bật.

BÀI TẬP 3: LƯỚI ẢNH TỰ DO (CSS GRID MASONRY-LIKE)

Yêu cầu:

- Xây dựng một thư viện ảnh (Photo Gallery) với ít nhất 8 ảnh có kích thước khác nhau.
- Sử dụng **CSS Grid** với thuộc tính `grid-column: span` và `grid-row: span` để tạo các ô ảnh to nhỏ đan xen nhau một cách nghệ thuật (giống giao diện Pinterest hoặc Instagram Explore).
- Đảm bảo khoảng cách giữa các ảnh (`gap`) luôn đồng nhất là 15px.

BÀI TẬP 4: THANH ĐIỀU HƯỚNG PHỨC HỢP (COMPLEX NAV BAR)

Yêu cầu:

- Thiết kế một Header gồm: Logo (bên trái), Menu chính (chính giữa), và các nút Action như "Đăng nhập/Đăng ký" (bên phải).
- Sử dụng **Flexbox** và thuộc tính `justify-content: space-between` để phân bổ 3 nhóm phần tử này.
- Menu chính phải có tính năng thả xuống (Dropdown) khi di chuột vào, các mục trong Dropdown cũng phải được dàn trang bằng Flexbox.

BÀI TẬP 5: TRANG CHI TIẾT SẢN PHẨM (PRODUCT PAGE LAYOUT)

Yêu cầu:

- Xây dựng giao diện trang chi tiết sản phẩm thương mại điện tử.
- Kết hợp **CSS Grid** cho bố cục lớn: Ảnh sản phẩm bên trái (60%) và Thông tin mua hàng bên phải (40%).
- Sử dụng **Flexbox** bên trong phần thông tin để dàn hàng các lựa chọn: Size (S, M, L), Màu sắc (các ô tròn màu), và bộ nút "Tăng/Giảm số lượng".

BÀI TẬP 6: DASHBOARD QUẢN TRỊ (ADMIN PANEL UI)

Yêu cầu:

- Thiết kế một giao diện Dashboard đơn giản gồm: Sidebar cố định bên trái (Fixed Sidebar) và vùng nội dung cuộn bên phải.

- Vùng nội dung bên phải sử dụng **CSS Grid** để hiển thị 4 thẻ thống kê (Stats Cards) ở trên cùng, bên dưới là một bảng dữ liệu (Data Table) chiếm toàn bộ chiều ngang.
- Yêu cầu AI tối ưu hóa mã nguồn sao cho Sidebar không bị mất khi người dùng cuộn trang.

YÊU CẦU NỘP BÀI (DÀNH CHO TẤT CẢ CÁC BÀI)

1. **Mã nguồn:** Đẩy toàn bộ các bài tập vào một Repository GitHub có tên `LHU_WebDesign_Practice_Week3`.
2. **Cấu trúc:** Mỗi bài tập nằm trong một thư mục riêng biệt (Ví dụ: `Ex01_HolyGrail`, `Ex02_PricingTable`).
3. **Prompt Log:** Trong mỗi thư mục bài tập, kèm theo file `prompts.md` lưu lại các câu lệnh bạn đã dùng để "ra lệnh" cho AI thực hiện Layout đó

THÁCH THỨC TẠI NHÀ (NHÓM): "THE GRID MASTER"

THÁCH THỨC TẠI NHÀ (NHÓM): "THE GRID MASTER"

> **Mục tiêu**: Hoàn thiện bộ khung Layout cho dự án nhóm đã bốc thăm.

> **Yêu cầu cụ thể**:

1. **Dàn trang tổng thể**: Áp dụng CSS Grid để tạo cấu trúc trang chủ hoàn chỉnh dựa trên file HTML của Tuần 2.
2. **Khu vực dự án (Project Gallery)**: Tạo lưới hiển thị các dự án (ít nhất 6 item) bằng CSS Grid sao cho các item có kích thước bằng nhau.
3. **Quy trình nhóm**:
   - `Tạo nhánh feature/css-layout.`
   - Thành viên A làm CSS cho Header/Footer.
   - Thành viên B làm CSS cho phần nội dung chính.
   - Thực hiện Pull Request và giải quyết xung đột (nếu có) khi gộp mã nguồn CSS.

Hướng dẫn nộp bài:

- Link GitHub Repo đã cập nhật file `style.css`.
- File `prompt_log.md` ghi lại cách bạn yêu cầu AI giải quyết các lỗi dàn trang khó.
- **Hạn nộp**: Trước buổi học Tuần 4 là 24 giờ.

> 💡 **Gợi ý cho sinh viên**: Nếu Layout bị vỡ, hãy thử dùng câu lệnh: *"Check this CSS Grid code for any overlap issues and fix the responsive column sizing"*

TUẦN 4: DESIGN SYSTEM & TƯ DUY RESPONSIVE

TUẦN 4: DESIGN SYSTEM & TƯ DUY RESPONSIVE (MOBILE FIRST)

1. Mục tiêu bài học (Lesson Objectives)

- **Về kiến thức:** Hiểu khái niệm Design System và tầm quan trọng của việc nhất quán giao diện. Nắm vững nguyên tắc Mobile First trong thiết kế hiện đại.
- **Về kỹ năng:** Sử dụng thành thạo Biến CSS (--variable-name) để quản lý màu sắc, font chữ toàn hệ thống. Vận dụng Media Queries để tối ưu hóa hiển thị trên di động.
- **Về thái độ:** Rèn luyện tư duy lập trình có kế hoạch, tránh viết mã "cứng" (hard-code) gây khó khăn khi bảo trì.

2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO)

- **LLO 2.2.1:** Thiết lập bộ khung Design System cơ bản sử dụng CSS Variables cho ít nhất 5 thuộc tính (màu sắc, khoảng cách, font-size).
- **LLO 2.2.2:** Vận dụng nguyên tắc Mobilefirst để xây dựng giao diện hiển thị chuẩn trên màn hình nhỏ trước khi mở rộng sang Desktop.
- **LLO 2.2.3:** Sử dụng AI để sinh mã Media Queries và thực hiện kiểm tra (Inspect) trên nhiều kích thước màn hình thiết bị giả lập.
- **LLO 2.2.4:** Phân tích và sửa lỗi "vỡ khung" khi thay đổi kích thước trình duyệt thông qua quy trình Debug với AI.

3. Nội dung chuẩn bị (Ở nhà - Flipped Classroom)

Sinh viên cần nghiên cứu các khái niệm trọng tâm sau:

- **CSS Variables (Custom Properties):** Cách khai báo tại :root và sử dụng hàm var().
- **Viewport & Meta Tag:** Tại sao cần <meta name="viewport" ...> để hiển thị đúng trên điện thoại.
- **Media Queries:** Cú pháp @media screen and (max-width: ...).
- **Mobile First vs Desktop First:** Tại sao xu hướng hiện nay ưu tiên Mobile First?

BẢNG HỆ THỐNG LÝ THUYẾT

STT	Chủ đề chính	Nội dung chi tiết	Link tài liệu tham khảo (W3Schools/MDN)
1	**CSS Variables**	Cách khai báo biến tại :root để dùng chung cho toàn dự án. Cú pháp var(--variable-name).	CSS Variables - W3Schools
2	**Mobile First Design**	Triết lý thiết kế ưu tiên màn hình nhỏ trước, sau đó dùng Media Queries để mở rộng cho Desktop.	Responsive Web Design Intro - W3Schools
3	**The Viewport Tag**	Tầm quan trọng của thẻ <meta name="viewport" ...> trong việc kiểm soát hiển thị trên thiết bị di động.	RWD Viewport - W3Schools

STT	Chủ đề chính	Nội dung chi tiết	Link tài liệu tham khảo (W3Schools/MDN)
4	**Media Queries**	Cú pháp @media và các điểm gãy (Breakpoints) phổ biến để thay đổi giao diện theo kích thước màn hình.	CSS3 Media Queries - W3Schools
5	**Responsive Images**	Sử dụng thuộc tính max-width: 100% và height: auto để ảnh không bị tràn khung trên điện thoại.	RWD Images - W3Schools
6	**Responsive Grid**	Kết hợp CSS Grid/Flexbox với Media Queries để thay đổi số lượng cột tùy theo chiều rộng màn hình.	RWD Grid View - W3Schools
7	**Typography System**	Cách thiết lập thang đo kích thước chữ (Font-size scale) linh hoạt cho các loại màn hình khác nhau.	RWD Text - W3Schools

💡 *GỢI Ý PHƯƠNG PHÁP HỌC TẬP TẠI NHÀ (FLIPPED CLASSROOM)*

Để chuẩn bị tốt nhất cho buổi thực hành, bạn hãy yêu cầu sinh viên thực hiện 3 bước sau với bảng nội dung trên:

1. **Bước 1 (Đọc & Hiểu):** Truy cập các link trong bảng để nắm vững cú pháp. Sinh viên cần đặc biệt lưu ý sự khác biệt giữa min-width (dùng cho Mobile First) và max-width (dùng cho Desktop First).
2. **Bước 2 (Thử nghiệm với AI):** Copy các đoạn mã mẫu từ W3Schools vào VS Code, sau đó yêu cầu AI (Vibe Code) giải thích: *"Tại sao đoạn code Media Query này lại giúp website hiển thị tốt trên iPhone?"*.
3. **Bước 3 (Thực hành nhỏ):** Tự tạo một file HTML đơn giản có 1 khối div, sau đó viết CSS sao cho trên Mobile khối đó có màu đỏ, trên Desktop khối đó chuyển sang màu xanh.

🛠 *CÔNG CỤ BỔ SUNG*

- **Công cụ kiểm tra:** Hướng dẫn sinh viên dùng **Chrome DevTools (F12)** để giả lập các thiết bị như iPhone, Samsung Galaxy, iPad.
- **Tư duy Design System:** Nhấn mạnh rằng Design System không chỉ là màu sắc, mà còn là sự thống nhất về khoảng cách (Spacing), kiểu chữ (Typography) và biểu tượng (Icons).

4. Phần thực hành tại lớp (In-class Activities)

Hoạt động 1: Xây dựng "Bảng mã màu thông minh" (30 phút)

- **Phân tích:** Thay vì dùng mã màu HEX trực tiếp cho từng thẻ, ta sẽ định nghĩa chúng ở một nơi duy nhất.
- **Viết Prompt:** *"Hãy tạo một bộ CSS Variables tại :root bao gồm: 2 màu chủ đạo (primary/secondary), 3 mức màu xám cho text, và 1 màu nền. Sau đó áp dụng các biến này vào cấu trúc Portfolio đã làm ở tuần 3."*
- **Kiểm tra:** Sinh viên thay đổi giá trị màu tại :root và quan sát toàn bộ trang web đổi màu đồng bộ.

Hoạt động 2: Bài thực hành số 3 - Responsive Portfolio (60 phút)

- **Phân tích:** Biến đổi bố cục đa cột trên Desktop thành 1 cột trên Mobile.
- **Viết Prompt:** *"Sử dụng Media Queries để tối ưu trang Portfolio. Trên màn hình < 768px, hãy chuyển CSS Grid từ 2 cột (sidebar/main) thành 1 cột duy nhất, ẩn sidebar hoặc chuyển nó lên đầu trang."*
- **Kiểm tra:** Sử dụng phím **F12** -> Biểu tượng thiết bị di động để kiểm tra các điểm gãy (breakpoints).

Hoạt động 3: "Responsive Breakpoint Challenge" (30 phút)

> **Mục tiêu:** Giúp sinh viên hiểu cách điều chỉnh bố cục linh hoạt tại các điểm gãy (breakpoints) khác nhau.

Hướng dẫn thực hiện:

1. **Phân tích yêu cầu:** Thiết kế một khu vực "Dịch vụ" (Services) gồm 4 cột trên Desktop, 2 cột trên Tablet và 1 cột trên Mobile.
2. **Viết Prompt:** *"Sử dụng CSS Grid và Media Queries, hãy tạo một bố cục cho class '.services-container'. Yêu cầu: Màn hình > 1024px có 4 cột, màn hình 768px - 1024px có 2 cột, và màn hình < 768px có 1 cột. Sử dụng 'gap: 20px'."*.
3. **Kiểm tra (Checklist):**
    - [ ] Khi thu nhỏ trình duyệt, số lượng cột có tự động thay đổi không?
    - [ ] Khoảng cách `gap` có được duy trì ổn định không?

Hoạt động 4: "Design System Refactoring" (30 phút)

> **Mục tiêu:** Chuyển đổi một đoạn code "cứng" thành một hệ thống có thể quản lý bằng biến.

Hướng dẫn thực hiện:

1. **Phân tích yêu cầu:** Giảng viên đưa ra một đoạn CSS có nhiều mã màu và thông số pixel lặp lại.
2. **Viết Prompt:** *"Hãy trích xuất tất cả các mã màu và thông số khoảng cách (margin/padding) trong đoạn code này vào các biến CSS tại :root. Sau đó, thay thế các giá trị trong code gốc bằng hàm var()."*.
3. **Kiểm tra (Checklist):**
    - [ ] Tất cả các mã màu cũ đã được thay bằng biến chưa?
    - [ ] Khi thay đổi giá trị một biến tại `:root`, các phần tử liên quan có thay đổi theo không?

Bài tập về nhà tuần 4

BÀI TẬP TỰ RÈN LUYỆN TẠI NHÀ (HOMEWORK)

Mỗi bài tập dưới đây yêu cầu sinh viên phải áp dụng triệt để **CSS Variables** và **Mobile First**.

Bài 1: Responsive Typography Scale

- **Yêu cầu:** Thiết lập một hệ thống cỡ chữ (h1, h2, h3, p) sử dụng biến CSS. Sử dụng Media Queries để tự động giảm kích thước chữ khi màn hình nhỏ hơn 600px để tránh tràn văn bản.
- **Tiêu chí:** Chữ trên Mobile phải dễ đọc, không quá to so với màn hình.

Bài 2: Adaptive Card Layout

- **Yêu cầu:** Tạo một "Card" giới thiệu sản phẩm gồm ảnh, tiêu đề và mô tả. Trên Desktop, ảnh nằm bên trái và chữ nằm bên phải. Trên Mobile, ảnh nằm trên và chữ nằm dưới.
- **Tiêu chí:** Sử dụng Flexbox `flex-direction` để thay đổi hướng hiển thị.

Bài 3: Smart Navigation Bar

- **Yêu cầu:** Xây dựng thanh menu. Trên màn hình lớn, menu hiển thị đầy đủ các liên kết. Trên màn hình dưới 768px, ẩn toàn bộ menu và chỉ hiển thị một nút "Menu" (hoặc biểu tượng Hamburger).
- **Tiêu chí:** Sử dụng thuộc tính `display: none` và Media Queries.

Bài 4: Dark/Light Mode Theme System

- **Yêu cầu:** Khai báo 2 bộ biến màu (ví dụ: `--bg-color` và `--text-color`). Tạo một class `.dark-theme` để ghi đè các giá trị biến này. Thử nghiệm việc thay đổi giao diện bằng cách thêm class vào thẻ `<body>`.
- **Tiêu chí:** Giao diện chuyển đổi màu sắc mượt mà.

Bài 5: Responsive Image Gallery with CSS Grid

- **Yêu cầu:** Tạo một lưới ảnh sử dụng `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr))`.
- **Tiêu chí:** Lưới ảnh phải tự động xuống dòng và co giãn số lượng cột dựa trên chiều rộng màn hình mà không cần viết quá nhiều Media Queries.

Bài 6: Mobile-First Contact Form

- **Yêu cầu:** Thiết kế một form liên hệ (Họ tên, Email, Nội dung, Nút gửi). Trên Mobile, các ô nhập liệu chiếm 100% chiều rộng. Trên Desktop, form được giới hạn chiều rộng và căn giữa trang.
- **Tiêu chí:** Form trông chuyên nghiệp trên cả điện thoại và máy tính.

YÊU CẦU NỘP BÀI

- **GitHub:** Mỗi bài tập lưu trong một thư mục riêng (ví dụ: `Exercise-01`, `Exercise-02`).
- **Prompt Log:** Kèm theo file `ai_prompts.md` ghi lại các câu lệnh quan trọng mà bạn đã dùng để AI hỗ trợ viết CSS.
- **Hạn nộp:** Trước buổi học Tuần 5

Thực hành tuần 4 - Multi-Device News Portal

Chủ đề bài tập: "Multi-Device News Portal"

(Thiết kế trang tin tức đa thiết bị)

1. Mục tiêu bài học

- **Tư duy Mobile First:** Thiết kế cho màn hình nhỏ nhất trước, sau đó mới mở rộng cho màn hình lớn.
- **Kỹ thuật Media Queries:** Sử dụng @media để thay đổi Layout.
- **Flexible Grids:** Sử dụng đơn vị %, vw, vh và fr thay vì px cố định.
- **AI Prompting:** Học cách yêu cầu AI viết mã CSS linh hoạt (Fluid Design).

  Công nghệ: Visual code + Github copilot chatbot or Gemini code assist. | Google Antigravity

2. Mô tả yêu cầu Layout (Wireframe)

Sinh viên phải yêu cầu AI thực hiện 3 trạng thái giao diện khác nhau:

Thành phần	Mobile (< 600px)	Tablet (600px - 1024px)	Desktop (> 1024px)
Navigation	Menu "Hamburger" (ẩn)	Menu dạng ngang đơn giản	Menu đầy đủ + Search Bar
Hero Section	1 ảnh lớn chồng lên chữ	Ảnh bên trái, chữ bên phải	Ảnh nền full, chữ căn giữa
News Grid	**1 cột** (dọc)	**2 cột**	**3 hoặc 4 cột**
Sidebar	Ẩn hoặc đẩy xuống cuối	Ẩn	Hiện bên phải (Tiêu điểm)

3. Các bước thực hiện chi tiết (Step-by-Step)

Bước 1: Prompt cho cấu trúc Mobile First

> Yêu cầu sinh viên bắt đầu với màn hình nhỏ nhất để tránh code bị thừa.

Prompt 1: *"Hãy đóng vai chuyên gia UI/UX. Viết HTML5 và CSS3 theo tư duy **Mobile First** cho một trang tin tức. Ở chế độ mobile (mặc định), hãy thiết kế một cột duy nhất: Header có logo căn giữa, một ảnh đại diện bài viết lớn, và danh sách 5 bài viết xếp chồng lên nhau. Sử dụng các thẻ Semantic như <header>, <main>, <article>."*

Bước 2: Prompt cho điểm ngắt Tablet (Breakpoints)

Khi giao diện mobile đã ổn, sinh viên yêu cầu AI nâng cấp cho Tablet.

> **Prompt 2:** *"Bây giờ, hãy thêm một đoạn **Media Query** cho màn hình từ 600px trở lên. Lúc này, hãy dùng **CSS Grid** để chuyển danh sách bài viết từ 1 cột thành 2 cột đều nhau. Phần Header hãy căn logo sang trái và thêm một nút Menu bên phải."*

Bước 3: Prompt cho điểm ngắt Desktop (Layout phức tạp)
Cuối cùng là giao diện đầy đủ nhất.

> **Prompt 3:** *"Tiếp tục thêm Media Query cho màn hình từ 1024px trở lên. Hãy thay đổi layout thành 3 cột:
>
> 1. Cột chính (Main) chiếm 70% bên trái chứa lưới tin tức (3 cột nhỏ).
> 2. Cột phụ (Sidebar) chiếm 30% bên phải chứa danh sách 'Tin xem nhiều'.
> 3. Thay đổi menu Hamburger thành một thanh điều hướng ngang đầy đủ các mục: Trang chủ, Công nghệ, Giáo dục, Đời sống."*

4. Danh sách kiểm tra hoàn thành (To-do List cho sinh viên)

Sinh viên hãy tích vào các ô sau khi đã hoàn thành để đảm bảo chất lượng:

- [ ] **Kiểm tra Viewport:** Đã có thẻ <meta name="viewport" content="width=device-width, initial-scale=1.0"> trong HTML chưa?
- [ ] **Hình ảnh linh hoạt:** Đã sử dụng max-width: 100%; height: auto; cho các thẻ ảnh chưa? (Để ảnh không bị tràn màn hình).
- [ ] **Không có thanh cuộn ngang:** Kiểm tra trên Chrome DevTools ở các kích thước khác nhau, đảm bảo không xuất hiện thanh cuộn ngang (Horizontal Scroll).
- [ ] **Font chữ:** Font chữ trên mobile có đủ lớn để đọc không? (Khuyên dùng tối thiểu 16px).
- [ ] **Vùng chạm (Touch Targets):** Các nút bấm trên mobile có đủ lớn và cách xa nhau để không bấm nhầm không?

5. Lỗi Prompt cần tránh trong Responsive Design

Bạn hãy nhắc sinh viên tránh các câu lệnh sau để AI không sinh "code rác":

- **Đừng nói:** *"Làm cho nó chạy trên mọi màn hình."* (Quá chung chung).
- **Nên nói:** *"Sử dụng Media Queries với các điểm ngắt tại 600px và 1024px."*
- **Đừng dùng:** Cố định width: 1200px; cho các container chính.
- **Nên dùng:** max-width: 1200px; width: 90%; margin: 0 auto; để trang web tự co lại trên màn hình nhỏ.

6. Thử thách nâng cao (Bonus)

Sử dụng AI để mở rộng bài tập: *"Làm thế nào để ẩn một hình ảnh nặng trên Mobile nhưng lại hiện ra trên Desktop để tăng tốc độ tải trang?"* (Sử dụng display: none;).

Với bài tập này, sinh viên sẽ thấy được sức mạnh của việc **"Ra lệnh đúng"** để AI xử lý những phần CSS Media Query vốn rất dễ nhầm lẫn nếu viết tay hoàn toàn.

RESPONSIVE CSS CHEAT SHEET (2026 Edition)

1. Thẻ Meta bắt buộc (Trong file HTML)

AI đôi khi quên dòng này, khiến mọi code CSS Responsive đều vô nghĩa trên điện thoại:

HTML

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

2. Các điểm ngắt (Breakpoints) tiêu chuẩn

Thay vì viết số lộn xộn, hãy dạy sinh viên dùng các mốc chuẩn này:

CSS

/* 1. Mobile First (Mặc định cho điện thoại) */

```
body { font-size: 16px; }
```

/* 2. Tablet (Màn hình từ 768px trở lên) */

@media (min-width: 768px) {

```
    .container { width: 90%; }
    .sidebar { display: block; }
}
```

/* 3. Laptop/Desktop (Màn hình từ 1024px trở lên) */

@media (min-width: 1024px) {

```
    .container { max-width: 1200px; }
    .nav-menu { display: flex; }
}
```

3. Công thức "Vàng" cho Layout linh hoạt

A. Hình ảnh không bao giờ bị tràn (Responsive Images)

Dặn sinh viên luôn kiểm tra xem AI có dòng này không:

CSS

img {

max-width: 100%;

height: auto;

display: block;

```
}
```

B. Grid tự động chia cột (Không cần Media Query)

Đây là kỹ thuật "thượng thừa" mà sinh viên nên yêu cầu AI sử dụng:

CSS

```
.grid-container {
```

display: grid;

/* Tự động chia cột: tối thiểu 250px, tối đa chiếm hết chỗ trống */

grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));

gap: 20px;

```
}
```

C. Flexbox căn giữa đa thiết bị

CSS

```
.flex-center {
```

display: flex;

justify-content: center; /* Căn giữa ngang */

```
    align-items: center;     /* Căn giữa dọc */
    flex-wrap: wrap;         /* Tự xuống dòng trên màn hình nhỏ */
}
```

4. Bảng đối chiếu "Code Tốt" vs "Code Rác"

Sinh viên hãy đối chiếu kết quả AI trả về với bảng này:

Thành phần	☑ Code Tốt (Nên dùng)	✖ Code Rác (Nên sửa)
Chiều rộng	width: 90%; hoặc max-width: 1200px;	width: 1200px;
Đơn vị Font	rem hoặc em (ví dụ: 1.2rem)	px (ví dụ: 20px) cho mọi nơi
Chia cột	display: grid; hoặc flex;	float: left; hoặc display: table;
Căn lề	margin: 0 auto;	margin-left: 250px; (cố định)

---

5. Mẹo Prompt để AI xuất code chuẩn theo Cheat Sheet này:

Sinh viên có thể copy đoạn này dán vào AI:

> *"Hãy viết CSS cho Layout này theo phong cách **Mobile First**, sử dụng **đơn vị rem** thay vì px. Ưu tiên dùng **CSS Grid auto-fit** cho danh sách sản phẩm và đảm bảo không sử dụng float hay position tuyệt đối cho bố cục chính."*

Thách thức tự rèn luyện tại nhà

Thách thức tự rèn luyện tại nhà (Hand-on Challenges)

Thử thách 1: Dark Mode Switcher (Sơ khai)

> - **Yêu cầu:** Tạo thêm một bộ biến CSS cho giao diện tối (Dark Mode). Sử dụng AI để viết mã CSS sao cho khi thêm class .dark vào thẻ <body>, toàn bộ màu sắc của trang sẽ tự động thay đổi theo biến Dark Mode.

Thử thách 2: Responsive Navigation Bar

> - **Yêu cầu:** Thiết kế Menu ngang trên máy tính. Khi về màn hình điện thoại, Menu biến mất và thay bằng biểu tượng "3 gạch" (Hamburger Menu). Sử dụng AI để hỗ trợ viết CSS ẩn/hiện mà không cần dùng JavaScript quá phức tạp.

Thử thách 3: Typography System

> - **Yêu cầu:** Thiết lập hệ thống kích thước chữ (--fs-small, --fs-medium, --fs-large) bằng biến CSS. Đảm bảo khi xem trên điện thoại, các biến này tự động giảm kích thước xuống để không bị tràn chữ.

Checklist hoàn thành Tuần 4

STT	Tiêu chí	Kết quả
1	Mọi mã màu và khoảng cách đều sử dụng biến var().	
2	Website không bị xuất hiện thanh cuộn ngang (Horizontal scroll) trên di động.	
3	Hình ảnh và Card dự án tự động co giãn theo chiều rộng màn hình.	
4	Mã nguồn đã được cập nhật vào nhánh feature/responsive trên GitHub.	

Lời khuyên: Đừng cố gắng tạo ra quá nhiều điểm gãy (Breakpoints). Hãy tập trung vào 3 mốc chính: **Mobile (375px), Tablet (768px), và Desktop (1024px)**.

TUẦN 5: HIỆU ỨNG CHUYỂN ĐỘNG & THƯ VIỆN HIỆU ỨNG

TUẦN 5: HIỆU ỨNG CHUYỂN ĐỘNG & THƯ VIỆN HIỆU ỨNG (CSS ANIMATIONS)

1. Mục tiêu bài học (Lesson Objectives)

- **Về kiến thức:** Hiểu nguyên lý của chuyển động trong Web (Timing functions, Keyframes). Phân biệt giữa transition (biến đổi trạng thái) và animation (chuyển động lặp lại/phức tạp).
- **Về kỹ năng:** Sử dụng AI để tạo ra các hiệu ứng mượt mà (Smooth hover, Loading spinner, Scroll reveal). Tích hợp thành thạo các thư viện CSS bên thứ ba (Animate.css, AOS).
- **Về thái độ:** Rèn luyện tư duy "vừa đủ" trong thiết kế (Micro-interactions), tránh lạm dụng hiệu ứng gây rối mắt cho người dùng.

2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO)

- **LLO 2.3.1:** Vận dụng thuộc tính transition để tạo hiệu ứng tương tác cho nút bấm, menu và thẻ card.
- **LLO 2.3.2:** Xây dựng được các chuyển động tùy chỉnh bằng @keyframes (ví dụ: hiệu ứng trượt, mờ dần, hoặc xoay).
- **LLO 2.3.3:** Tích hợp thành công thư viện hiệu ứng bên ngoài vào dự án thông qua CDN.
- **LLO 2.3.4:** Sử dụng Prompt để yêu cầu AI tối ưu hóa hiệu năng chuyển động (sử dụng transform thay vì margin/left).

3. Nội dung lý thuyết (Flipped Classroom)

STT	Chủ đề chính	Nội dung chi tiết	Link tài liệu tham khảo
1	**CSS Transitions**	Cách tạo hiệu ứng thay đổi mượt mà giữa hai trạng thái (ví dụ: đổi màu khi hover).	CSS Transitions - W3Schools
2	**CSS Animations**	Sử dụng @keyframes để tạo các chuỗi chuyển động phức tạp và lặp lại.	CSS Animations - W3Schools
3	**2D/3D Transforms**	Các thuộc tính translate, rotate, scale, skew để biến đổi hình dạng phần tử.	CSS Transforms - W3Schools
4	**Animate.css**	Cách sử dụng thư viện CSS có sẵn để thêm hiệu ứng nhanh chóng.	Animate.css Documentation

4. Hoạt động thực hành trên lớp (In-class Activities)

Hoạt động 1: "Micro-interactions" - Nút bấm cảm xúc (30 phút)

- **Hướng dẫn:** Thiết kế một nút "Gửi tin nhắn". Khi hover, nút sẽ đổi màu, nhích lên trên một chút và có bóng đổ đổ xuống. Khi click, nút sẽ thu nhỏ lại tạo cảm giác "đã bấm".
- **Prompt mẫu:** *"Hãy viết mã CSS transition cho một button sao cho khi hover nó sẽ nâng lên 3px, thêm box-shadow đậm hơn, và sử dụng hàm timing 'cubic-bezier' để chuyển động trông tự nhiên hơn."*

Hoạt động 2: "Loading Master" - Tạo Spinner (30 phút)

- **Hướng dẫn:** Sử dụng @keyframes để tạo một vòng tròn xoay liên tục (Loading indicator) cho trang Portfolio.
- **Prompt mẫu:** *"Tạo một vòng tròn loading bằng CSS sử dụng animation xoay 360 độ vô hạn. Sử dụng border-top-color khác với các phần còn lại để tạo hiệu ứng xoay."*

Hoạt động 3: "AOS - Scroll Reveal" (30 phút)

- **Hướng dẫn:** Tích hợp thư viện AOS (Animate On Scroll) vào dự án nhóm. Khi người dùng cuộn trang, các khối nội dung sẽ hiện ra (fade-in, slide-up).
- **Prompt mẫu:** *"Hướng dẫn tôi cách tích hợp thư viện AOS vào file HTML thông qua CDN và cách thêm thuộc tính 'data-aos' vào các thẻ section để chúng hiện ra khi cuộn xuống."*

Hoạt động 4: "Refactoring Performance" (15 phút)

- **Hướng dẫn:** Giảng viên hướng dẫn cách thay đổi thuộc tính top/left thành transform: translate để AI sinh ra code có hiệu năng cao hơn (không gây giật lag trên điện thoại).

5. 6 Bài tập tự rèn luyện tại nhà (Homework)

- **Bài 1: Floating Action Button (FAB):** Tạo một nút tròn "Liên hệ" luôn nằm ở góc màn hình và có hiệu ứng "phập phồng" (pulse animation) để thu hút sự chú ý.
- **Bài 2: Card Flip Effect:** Thiết kế một thẻ nhân sự. Khi hover, thẻ sẽ lật 180 độ để hiện thông tin liên hệ ở mặt sau.
- **Bài 3: Typing Effect:** Sử dụng CSS (không dùng JS) để tạo hiệu ứng chữ đang tự gõ cho dòng giới thiệu: "Tôi là một Web Developer...".
- **Bài 4: Parallax Image:** Tạo một banner có hiệu ứng ảnh nền di chuyển chậm hơn nội dung khi cuộn trang (sử dụng background-attachment: fixed).
- **Bài 5: Modern Hamburger Menu:** Biến đổi biểu tượng 3 gạch ngang thành dấu "X" một cách mượt mà khi người dùng mở Menu (Sử dụng CSS Transitions).
- **Bài 6: Skill Bar Animation:** Khi trang web tải xong, các thanh kỹ năng (HTML 90%, CSS 85%...) sẽ chạy từ 0% đến giá trị đích.

YÊU CẦU NỘP BÀI

- **GitHub:** Cập nhật vào nhánh feature/animations.
- **Sản phẩm:** Trang Portfolio lúc này phải có ít nhất 3 loại hiệu ứng: Hover (tương tác), Keyframes (tự động), và Scroll (cuộn trang).
- **Prompt Log:** File week5_prompts.md ghi lại cách bạn yêu cầu AI tinh chỉnh tốc độ và độ mượt của chuyển động.

BÀI TẬP TỰ RÈN LUYỆN - TUẦN 5

BÀI TẬP TỰ RÈN LUYỆN (HOMEWORK) - TUẦN 5

Bài 1: Floating Action Button (FAB) - Nút tương tác nổi

- **Kịch bản:** Một nút tròn chứa biểu tượng "Zalo" hoặc "Messenger" luôn nằm ở góc dưới bên phải màn hình. Nút này liên tục co giãn nhẹ để thu hút sự chú ý của khách hàng.
- **Yêu cầu kỹ thuật:**
  - Sử dụng `position: fixed` để cố định vị trí.
  - Tạo một `@keyframes pulse` thay đổi `transform: scale()` từ 1 lên 1.1 và ngược lại.
  - Thêm `box-shadow` lan tỏa nhẹ khi nút co giãn.
- **Gợi ý Prompt:** *"Hãy tạo một nút tròn cố định ở góc dưới bên phải màn hình. Sử dụng CSS Animation để tạo hiệu ứng 'phập phồng' (pulse) co giãn liên tục và mượt mà. Nút có màu xanh, đổ bóng và biểu tượng nằm chính giữa."*

Bài 2: Card Flip Effect - Thẻ nhân sự lật mặt

- **Kịch bản:** Sinh viên tạo một thẻ giới thiệu bản thân. Mặt trước hiện ảnh và tên; khi di chuột vào, thẻ lật 180 độ lộ ra mặt sau chứa các liên kết mạng xã hội (Facebook, GitHub).
- **Yêu cầu kỹ thuật:**
  - Cấu trúc HTML gồm một container chính và hai thẻ con: `.card-front`, `.card-back`.
    - `Sử dụng thuộc tính transform-style: preserve-3d và backface-visibility: hidden.`
  - Dùng `transition: transform 0.6s` để hiệu ứng lật trông chuyên nghiệp.
- **Gợi ý Prompt:** *"Tạo hiệu ứng lật thẻ (Card Flip) bằng CSS. Khi hover vào thẻ, nó sẽ xoay 180 độ theo trục Y để hiện mặt sau. Đảm bảo hiệu ứng có chiều sâu 3D và mượt mà."*

Bài 3: Typing Effect - Hiệu ứng máy đánh chữ

- **Kịch bản:** Một dòng chữ giới thiệu trên Header (ví dụ: "Tôi là kỹ sư phần mềm") sẽ tự động chạy ra từng chữ như có người đang gõ.
- **Yêu cầu kỹ thuật:**
  - Sử dụng `@keyframes typing` để thay đổi chiều rộng (`width`) của văn bản từ 0 đến 100%.
  - Sử dụng `@keyframes blink` để tạo con trỏ nhấp nháy ở cuối dòng.
    - `Thiết lập thuộc tính white-space: nowrap và overflow: hidden.`
- **Gợi ý Prompt:** *"Hãy viết mã CSS tạo hiệu ứng máy đánh chữ (Typing effect) cho một thẻ . Chữ sẽ xuất hiện dần dần từ trái sang phải, kèm theo con trỏ nhấp nháy ở cuối dòng chữ. Không sử dụng JavaScript."*

Bài 4: Parallax Background - Ảnh nền chuyển động sâu

- **Kịch bản:** Khi người dùng cuộn trang web, hình ảnh nền của một section sẽ di chuyển chậm hơn so với nội dung chữ bên trên, tạo cảm giác không gian 3D.
- **Yêu cầu kỹ thuật:**
  - `Sử dụng thuộc tính background-attachment: fixed.`
  - `Căn giữa ảnh nền bằng background-position: center và background-size: cover.`
- **Gợi ý Prompt:** *"Hướng dẫn cách tạo hiệu ứng Parallax đơn giản cho một Section bằng CSS. Khi cuộn trang, ảnh nền phải giữ cố định hoặc di chuyển chậm hơn nội dung phía trên. Hãy đảm bảo ảnh hiển thị tốt trên cả Desktop và Mobile."*

Bài 5: Modern Hamburger Menu - Biến đổi biểu tượng

- **Kịch bản:** Nút Menu trên di động gồm 3 gạch ngang. Khi bấm vào, gạch giữa biến mất, gạch trên và gạch dưới xoay lại tạo thành dấu "X".
- **Yêu cầu kỹ thuật:**
  - Mỗi gạch ngang là một thẻ `<span>` bên trong nút.
  - Sử dụng `transition` cho các thuộc tính `transform` và `opacity`.
  - Khi nút có thêm class `.active`, các gạch sẽ chuyển đổi vị trí và góc xoay.
- **Gợi ý Prompt:** *"Tạo hiệu ứng chuyển đổi biểu tượng Hamburger (3 gạch) thành dấu X bằng CSS Transition. Khi tôi thêm class 'open' vào container, thanh trên cùng xoay 45 độ, thanh giữa biến mất, và thanh dưới xoay -45 độ."*

Bài 6: Skill Bar Animation - Thanh kỹ năng tự động chạy

- **Kịch bản:** Khi trang Portfolio vừa load, các thanh đo mức độ kỹ năng (ví dụ: HTML 90%) sẽ bắt đầu chạy từ mức 0% đến mức mục tiêu.
- **Yêu cầu kỹ thuật:**
  - Tạo một class cha làm khung nền và class con làm thanh màu hiển thị phần trăm.
  - Sử dụng `@keyframes fillBar` thay đổi `width` từ 0 đến giá trị mong muốn.
  - Thiết lập `animation-fill-mode: forwards` để thanh giữ nguyên vị trí sau khi chạy xong.
- **Gợi ý Prompt:** *"Viết CSS cho các thanh tiến trình (Progress bars) hiển thị kỹ năng. Khi trang được tải, các thanh này sẽ chạy từ 0% đến mức phần trăm cụ thể (ví dụ 80%) trong vòng 2 giây với hiệu ứng mượt mà."*

Checklist nộp bài dành cho sinh viên:

- [ ] **Cấu trúc:** Có đầy đủ 6 thư mục bài tập trong Repository GitHub.
- [ ] **Hiệu năng:** Các hiệu ứng không gây giật lag (sử dụng GPU Acceleration qua `transform`).
- [ ] **Prompt Log:** Ghi chú lại những lần bạn yêu cầu AI thay đổi thời gian (`duration`) hoặc kiểu chuyển động (`ease-in-out`) để đạt kết quả tốt nhất.

THÁCH THỨC TẠI NHÀ: "THE INTERACTIVE SHOWCASE"

THÁCH THỨC TẠI NHÀ: "THE INTERACTIVE SHOWCASE" (TUẦN 5)

🎯 1. Mục tiêu của thách thức

- **Tích hợp:** Hợp nhất tất cả các kiến thức từ Tuần 1 đến Tuần 5 (Semantic HTML, Flex/Grid, Responsive, Design System và Animations) vào một sản phẩm duy nhất.
- **Trải nghiệm người dùng (UX):** Sử dụng các hiệu ứng chuyển động để dẫn dắt hành vi người dùng, tăng tính thẩm mỹ nhưng không làm chậm hiệu suất trang web.
- **Làm việc nhóm:** Thực hiện quy trình Review mã nguồn nghiêm ngặt để đảm bảo các hiệu ứng của các thành viên không xung đột với nhau.

📋 2. Yêu cầu cụ thể

A. Về nội dung sản phẩm

Nhóm phải nâng cấp trang chủ dự án đã bốc thăm thành một phiên bản "Smart Interface" bao gồm:

1. **Intro Animation:** Khi vừa vào trang, các thành phần Header và Hero Section phải xuất hiện theo các hướng khác nhau (Fade-in, Slide-up).
2. **Interactive Portfolio/Menu:** Các thẻ Card giới thiệu sản phẩm/thành viên phải có hiệu ứng tương tác (Hover effect) hoặc lật (Flip) để hiển thị thông tin chi tiết.
3. **Scroll Revelation:** Sử dụng thư viện AOS (Animate On Scroll) để tạo hiệu ứng các Section xuất hiện mượt mà khi người dùng cuộn chuột.
4. **Micro-interactions:** Các nút bấm (Call-to-Action) phải có trạng thái phản hồi khi di chuột và khi click (Active state).

B. Về kỹ thuật & AI (Vibe Coding)

- **Tối ưu hiệu năng:** Tuyệt đối không dùng các thuộc tính gây lag như top, left để tạo chuyển động; phải dùng transform: translate() hoặc opacity.
- **Prompt Engineering:** Nhóm phải có ít nhất 1 câu Prompt "bậc thầy" để điều chỉnh các thông số cubic-bezier cho chuyển động trông tự nhiên và chuyên nghiệp.

📝 3. Hướng dẫn và Yêu cầu nộp bài

Hồ sơ nộp bài bao gồm:

1. **Link GitHub Repository:** Toàn bộ mã nguồn đã hoàn thiện trên nhánh main.
2. **File showcase.gif hoặc video demo:** Một đoạn quay màn hình ngắn (30-60s) thể hiện tất cả các hiệu ứng đã cài đặt.
3. **File prompt_logic.md:**
    - Liệt kê các Prompt đã dùng để xử lý lỗi hiệu ứng.
    - Giải thích lý do tại sao nhóm chọn hiệu ứng đó cho từng Section (Tư duy UX).

Quy trình nộp:

- Trưởng nhóm đại diện nộp link Repo và Video Demo lên hệ thống quản lý học tập (LMS).
- **Hạn nộp:** Trước buổi học Tuần 6 là 24 giờ.

4. Tiêu chí chấm điểm (Rubric)

Tiêu chí	Trọng số	Yêu cầu đạt
Tính tương tác	40%	Hiệu ứng mượt mà, không giật lag, mang lại cảm giác cao cấp (Premium feel).
Tính nhất quán	20%	Các hiệu ứng đồng bộ về tốc độ và phong cách trên toàn trang.
Kỹ thuật Git	20%	Lịch sử commit rõ ràng, có PR review cho phần animation.
Tối ưu Mobile	20%	Hiệu ứng vẫn hoạt động tốt trên giả lập di động mà không gây tràn khung.

Gợi ý cho các nhóm: Các bạn có thể tham khảo các trang web như *Awwwards* hoặc *Lapa.ninja* để lấy ý tưởng về các chuyển động hiện đại, sau đó mô tả lại cho AI thực hiện. Đừng quên sử dụng biến CSS cho thời gian (--transition-speed: 0.3s) để dễ dàng điều chỉnh toàn bộ trang web nhé!

TUẦN 6: HOÀN THIỆN GIAO DIỆN & TỐI ƯU HÓA SẢN PHẨM

TUẦN 6: HOÀN THIỆN GIAO DIỆN & TỐI ƯU HÓA SẢN PHẨM (REFACTORING & UI/UX FINALIZATION)

1. Mục tiêu bài học (Lesson Objectives)

- **Về kiến thức:** Hiểu quy trình kiểm thử giao diện (UI Testing) và các tiêu chuẩn tối ưu hóa mã nguồn CSS (Clean Code).
- **Về kỹ năng:** Sử dụng AI để rà soát lỗi giao diện, tối ưu hóa tốc độ tải trang thông qua việc nén ảnh và tinh gọn mã nguồn.
- **Về thái độ:** Hình thành tính chuyên nghiệp, tỉ mỉ trong việc trau chuốt từng chi tiết nhỏ của sản phẩm trước khi bàn giao/trình bày.

2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO)

- **LLO 2.4.1:** Thực hiện được việc kiểm tra độ tương thích giao diện trên ít nhất 5 loại thiết bị phổ biến bằng công cụ DevTools.
- **LLO 2.4.2:** Sử dụng AI để tái cấu trúc (Refactor) mã nguồn CSS từ dạng lộn xộn thành dạng có cấu trúc theo Design System.
- **LLO 2.4.3:** Hoàn thiện file index.html và style.css đạt chuẩn W3C Validation.
- **LLO 2.4.4:** Trình bày được ý tưởng và cấu trúc kỹ thuật của sản phẩm nhóm trước tập thể.

3. Nội dung lý thuyết tự học (Flipped Classroom)

STT	Chủ đề chính	Nội dung chi tiết	Link tài liệu tham khảo
1	**CSS Refactoring**	Cách dọn dẹp mã nguồn CSS, loại bỏ các thuộc tính thừa và tối ưu selector.	CSS Refactoring Tips
2	**Web Performance**	Kỹ thuật nén ảnh (WebP), nén file CSS để tăng tốc độ tải trang.	Web Performance - MDN
3	**UI Checklist**	Các tiêu chuẩn cần kiểm tra trước khi hoàn thiện một giao diện Web.	Web Design Checklist

4. Hoạt động thực hành trên lớp (In-class Activities)

Hoạt động 1: "The Bug Hunter" - Săn lỗi giao diện (30 phút)

- **Hướng dẫn:** Sinh viên đổi máy tính cho nhau trong nhóm hoặc giữa các nhóm. Sử dụng DevTools để tìm ra ít nhất 3 lỗi giao diện (vỡ khung, màu chữ khó đọc, ảnh bị méo) trên máy bạn.
- **Sử dụng AI:** Chụp màn hình lỗi hoặc copy đoạn code lỗi gửi cho AI với Prompt: *"Giao diện đang bị lỗi hiển thị như ảnh trên iPhone 12, hãy phân tích nguyên nhân và đề xuất mã CSS sửa lỗi."*

Hoạt động 2: "Speed Up!" - Tối ưu hóa hiệu năng (30 phút)

- **Hướng dẫn:** Sử dụng các công cụ nén ảnh trực tuyến để chuyển đổi toàn bộ ảnh sang định dạng .webp.

- **Sử dụng AI:** Yêu cầu AI rà soát file CSS để gộp các class có chung thuộc tính nhằm giảm dung lượng file.

Hoạt động 3: Buổi Workshop "Showcase & Peer Review" (75 phút)

- **Hướng dẫn:** Mỗi nhóm có 5 phút để trình bày sản phẩm Portfolio/Landing Page của nhóm mình.
- **Nội dung:** Giới thiệu về Design System đã chọn, các hiệu ứng chuyển động ấn tượng nhất và cách nhóm đã phối hợp qua GitHub.
- **Phản biện:** Các nhóm khác đưa ra nhận xét về trải nghiệm người dùng (UX).

6. Danh mục Checklist hoàn thiện Tuần 6

STT	Tiêu chí đánh giá	Kết quả
1	File HTML đạt chuẩn W3C (không lỗi cú pháp).	
2	Website hiển thị tốt trên cả iPhone (Safari) và Android (Chrome).	
3	Tốc độ phản hồi hiệu ứng (Transition) mượt mà dưới 0.3s.	
4	Lịch sử Git có ít nhất 10 commit từ tất cả các thành viên nhóm.	
5	Có file README.md chuyên nghiệp hướng dẫn cách xem sản phẩm.	

HƯỚNG DẪN HOẠT ĐỘNG TẠI LỚP (IN-CLASS)

Hoạt động 1: "The Bug Hunter" (Săn lỗi giao diện)

- **Mục tiêu:** Rèn luyện tư duy phản biện và khả năng kiểm thử giao diện (UI Testing).
- **Cách thực hiện:**
  1. **Trao đổi:** Các nhóm hoán đổi máy tính hoặc gửi link GitHub cho nhau.
  2. **Kiểm tra:** Sử dụng phím **F12 (DevTools)**, chuyển đổi qua lại giữa các dòng điện thoại (iPhone, Pixel, Samsung) để tìm lỗi.
  3. **Vibe Coding:** Chụp màn hình lỗi hoặc copy đoạn code nghi vấn, gửi cho AI với Prompt: *"Phân tích tại sao layout này bị tràn khung trên màn hình 375px và đề xuất cách sửa bằng Flexbox/Grid"*.
- **Kết quả:** Danh sách các lỗi đã tìm thấy và mã nguồn đã sửa.

Hoạt động 2: "Speed Up!" (Tối ưu hóa hiệu năng)

- **Mục tiêu:** Tăng tốc độ tải trang và làm gọn mã nguồn.
- **Cách thực hiện:**
  1. **Nén ảnh:** Chuyển đổi toàn bộ ảnh sang định dạng **.webp** để giảm dung lượng mà không mất chất lượng.
  2. **Refactor CSS:** Copy file CSS vào AI và sử dụng Prompt: *"Hãy rà soát file CSS này, gộp các class có thuộc tính giống nhau, loại bỏ mã thừa và sắp xếp lại theo thứ tự từ Header đến Footer"*.
- **Kết quả:** Một trang web có dung lượng nhẹ hơn và code "sạch" hơn.

Hoạt động 3: Workshop "Showcase & Peer Review"

- **Mục tiêu:** Trình bày sản phẩm và nhận góp ý từ cộng đồng.
- **Cách thực hiện:**
  1. **Thuyết trình:** Mỗi nhóm có 5 phút demo sản phẩm trực tiếp trên trình duyệt.
  2. **Nội dung:** Tập trung vào 3 điểm: Cấu trúc Semantic, Hệ thống Design System (Biến CSS) và các hiệu ứng Animation ấn tượng nhất.
  3. **Góp ý:** Các nhóm khác đóng vai trò khách hàng để nhận xét về trải nghiệm (UX).

🏠 CHUỖI BÀI TẬP TỰ RÈN LUYỆN (HOMEWORK)

Các bài tập này được thiết kế để chuẩn bị cho bài kiểm tra giữa kỳ vào Tuần 7.

Bài 1: Xây dựng Trang mục lục (Index Hub)

- **Yêu cầu:** Tạo một trang `index.html` tổng hợp, dẫn link tới toàn bộ 6 bài thực hành đã làm từ tuần 1 đến nay.
- **Yêu cầu kỹ thuật:** Sử dụng CSS Grid để tạo các "Card" bài tập. Mỗi Card phải có ảnh chụp màn hình bài tập đó.
- **Hiệu ứng:** Khi hover vào từng Card, ảnh sẽ phóng to nhẹ và hiện nút "Xem chi tiết".

Bài 2: Tối ưu hóa chuẩn W3C (Validation)

- **Yêu cầu:** Đưa link website của bạn vào công cụ **W3C Validator**.
- **Nhiệm vụ:** Sử dụng AI để giải thích các lỗi (nếu có) và sửa cho đến khi website đạt trạng thái "0 lỗi, 0 cảnh báo".

Bài 3: Responsive Navigation hoàn chỉnh

- **Yêu cầu:** Hoàn thiện thanh Menu sao cho:
  - Trên Desktop: Menu ngang, có hiệu ứng hover đổi màu mượt mà.
  - Trên Mobile: Menu biến thành Hamburger, khi bấm vào sẽ trượt ra từ bên cạnh (Sử dụng kỹ thuật CSS Checkbox Hack hoặc phối hợp Animation).

Bài 4: Tích hợp thư viện AOS chuyên sâu

- **Yêu cầu:** Áp dụng thư viện AOS vào ít nhất 5 thành phần khác nhau trên trang Portfolio nhóm.
- **Nhiệm vụ:** Tinh chỉnh các thuộc tính `data-aos-duration` và `data-aos-delay` để tạo ra một chuỗi chuyển động nhịp nhàng (không hiện ra cùng lúc).

Bài 5: Nâng cấp Footer chuyên nghiệp

- **Yêu cầu:** Thiết kế Footer gồm 3 cột (Thông tin nhóm, Liên kết nhanh, Form đăng ký nhận tin).
- **Yêu cầu kỹ thuật:** Sử dụng Flexbox để đảm bảo Footer luôn nằm ở dưới cùng của trang ngay cả khi nội dung chính ít (Sticky Footer).

Bài 6: Viết README.md chuyên nghiệp

- **Yêu cầu:** Viết file giới thiệu dự án trên GitHub.
- **Nội dung:**
  - Tên dự án và chủ đề.
  - Công nghệ sử dụng (HTML5, CSS3, Vibe Code...).
  - Danh sách các câu Prompt "đắt giá" nhất mà nhóm đã dùng để xử lý layout khó.

📋 CHECKLIST HOÀN THIỆN TRƯỚC TUẦN 7

- [ ] Toàn bộ bài tập đã được Push lên nhánh `main`.
- [ ] Mọi hình ảnh đều hiển thị đúng, không bị lỗi link (broken links).
- [ ] Đã kiểm tra website trên cả điện thoại thật và máy tính.
- [ ] File `style.css` không còn chứa mã màu trực tiếp (phải dùng biến CSS).

Thách thức tổng kết giai đoạn

Thách thức tổng kết giai đoạn (Hackathon Mini tại nhà)

Đây là bài tập lớn nhất để chuẩn bị cho cột mốc **Kiểm tra giữa kỳ ở Tuần 7**.

Chủ đề: "Hoàn thiện hệ thống giao diện đa tầng"

- **Yêu cầu:**
    1. Tất cả các bài tập từ Tuần 1 đến Tuần 5 phải được gộp chung vào một Repository duy nhất, có trang index.html làm mục lục dẫn đến các bài tập đó.
    2. Trang mục lục này phải có hiệu ứng Animation ấn tượng khi người dùng rê chuột vào từng bài tập.
    3. Mọi mã nguồn phải sử dụng **Biến CSS** tuyệt đối cho màu sắc và spacing.

TUẦN 7: ĐÁNH GIÁ GIỮA KỲ

TUẦN 7: ĐÁNH GIÁ GIỮA KỲ - "FRONT-END WARRIOR HACKATHON"

1. Mục tiêu buổi đánh giá (Assessment Objectives)

- **Tổng hợp:** Kiểm tra khả năng kết hợp Semantic HTML, CSS Layout (Grid/Flexbox), Responsive và Animation vào một sản phẩm cụ thể.
- **Kỹ năng thực chiến:** Đánh giá tốc độ xử lý vấn đề, khả năng Prompting để điều khiển AI và kỹ năng xử lý lỗi giao diện.
- **Sản phẩm:** Hoàn thiện một Landing Page đạt chuẩn chuyên nghiệp trong thời gian giới hạn.

2. Chuẩn đầu ra đánh giá (Assessment Learning Outcomes - ALO)

- **ALO 2.5.1:** Xây dựng hoàn chỉnh một trang Web từ yêu cầu thực tế trong 120-150 phút.
- **ALO 2.5.2:** Áp dụng đúng hệ thống Design System (Variables) và Responsive (Mobile First) cho sản phẩm.
- **ALO 2.5.3:** Giải trình được các câu lệnh Prompt đã sử dụng để tối ưu hóa mã nguồn.

3. Cấu trúc bài thi thực hành (Thời lượng: 150 phút)

Chủ đề thi: "AI Coffee Shop - Giao diện đặt hàng thông minh"

Phần 1: Xây dựng cấu trúc & Layout (60 phút)

- Sử dụng Semantic HTML5 để dựng khung trang web.
- Dàn trang bằng CSS Grid cho bố cục lớn và Flexbox cho các thành phần con (Menu, List sản phẩm).

> - **Yêu cầu:** Phải sử dụng CSS Variables cho ít nhất 3 màu chủ đạo và 2 thông số khoảng cách.

Phần 2: Responsive & Interaction (60 phút)

- Tối ưu hóa giao diện cho Mobile (dưới 768px).
- Thêm hiệu ứng Hover cho các thẻ sản phẩm và hiệu ứng Fade-in khi tải trang.
- Tích hợp nút "Đặt hàng" có hiệu ứng Micro-interaction khi nhấn.

Phần 3: Đóng gói & Giải trình (30 phút)

- Kiểm tra lỗi bằng DevTools trên nhiều thiết bị.
- Viết file REPORT.md tóm tắt: Ý tưởng thiết kế, 3 câu Prompt hiệu quả nhất đã dùng và cách xử lý 1 lỗi khó nhất gặp phải.
- Push toàn bộ mã nguồn lên một Repository mới tên là LHU_Midterm_YourName.

4. Tiêu chí chấm điểm (Rubric)

Tiêu chí	Trọng số	Mô tả mức đạt
Cấu trúc HTML	20%	Đúng chuẩn Semantic, không lỗi thẻ, chuẩn SEO cơ bản.
CSS & Layout	30%	Layout chính xác, sử dụng thành thạo Grid/Flexbox và Biến CSS.

Tiêu chí	Trọng số	Mô tả mức đạt
Responsive	20%	Hiển thị hoàn hảo trên Mobile, không bị tràn khung hay chồng chéo chữ.
Animation/UX	15%	Hiệu ứng mượt mà, tăng tính thẩm mỹ, không gây rối mắt.
Kỹ năng AI/Prompt	15%	Prompt thông minh, giải trình được cách điều khiển AI để sửa lỗi.

🚀 HƯỚNG DẪN ÔN TẬP CHO SINH VIÊN (DÀNH CHO TUẦN 7)

Để đạt kết quả cao, sinh viên cần thực hiện chuỗi bài tập "Pre-Midterm" tại nhà:

1. **Luyện tập tốc độ:** Thử dựng lại giao diện một trang Web bất kỳ (như Coffee House hoặc Highlands Coffee) trong vòng 60 phút.
2. **Lưu trữ "Prompt Library":** Tổng hợp sẵn các câu Prompt hay đã dùng từ Tuần 1-6 để có thể sử dụng nhanh trong bài thi.
3. **Kiểm tra môi trường:** Đảm bảo VS Code, Git và AI Assistant (Vibe Code) hoạt động ổn định, không gặp lỗi kết nối.

📥 HÀNH ĐỘNG DÀNH CHO GIẢNG VIÊN

- **Chuẩn bị đề thi:** Chuẩn bị sẵn **3-5 chủ đề** khác nhau để sinh viên bốc thăm, tránh việc sao chép mã nguồn.
- **Hỗ trợ kỹ thuật:** Chuẩn bị sẵn các đoạn mã mẫu (Starter Template) gồm cấu trúc thư mục chuẩn để sinh viên không mất thời gian tạo folder.
- **Quan sát:** Trong lúc thi, hãy đi vòng quanh để quan sát cách sinh viên đặt câu hỏi cho AI. Đây là dữ liệu quan trọng để đánh giá tư duy của các em.

Lời khuyên: Hãy nhấn mạnh với sinh viên rằng bài thi này không chỉ chấm "Kết quả cuối cùng" mà còn chấm cả "Quy trình làm việc" (Lịch sử Commit và file Prompt Log). Chúc các em sinh viên của bạn có một kỳ thi giữa kỳ bùng nổ sáng tạo!

Hỗ trợ giáo viên

DANH SÁCH ĐỀ THI

Mỗi đề thi được thiết kế để kiểm tra toàn diện các kỹ năng đã học nhưng với bối cảnh nội dung khác nhau.

Đề 1: Trang Giới thiệu Dự án Công nghệ (Tech Launchpad)

- **Mô tả:** Xây dựng Landing Page cho một sản phẩm AI mới.
- **Yêu cầu đặc thù:** Cần có bảng so sánh tính năng (CSS Grid) và hiệu ứng xuất hiện các khối công nghệ (AOS).
- **Điểm nhấn:** Phối màu hiện đại (Dark mode chủ đạo), sử dụng biến CSS cho mã màu Neon.

Đề 2: Trang Portfolio Nghệ thuật (Creative Studio)

- **Mô tả:** Xây dựng trang hồ sơ năng lực cho một nhiếp ảnh gia hoặc kiến trúc sư.
- **Yêu cầu đặc thù:** Lưới ảnh không đối xứng (Masonry-like) sử dụng CSS Grid.
- **Điểm nhấn:** Hiệu ứng lật thẻ (Card Flip) giới thiệu tác phẩm và chuyển động mượt mà khi cuộn trang.

Đề 3: Trang Đặt chỗ Nhà hàng/Quán Cà phê (Smart Bistro)

- **Mô tả:** Giao diện cho một quán cà phê hiện đại có chức năng xem menu và đặt bàn.
- **Yêu cầu đặc thù:** Menu phân loại theo tab (sử dụng Flexbox) và Form liên hệ Responsive chuẩn Mobile.
- **Điểm nhấn:** Sử dụng các hiệu ứng Micro-interaction cho các nút đặt hàng.

STT	Chủ đề	Mô tả nội dung	Yêu cầu kỹ thuật đặc thù
1	**Green Horizon**	Trang web về dự án môi trường, trồng rừng hoặc năng lượng sạch.	Sử dụng dải màu Gradient xanh lá; hiệu ứng AOS cho các số liệu thống kê.
2	**Cyber Gaming Gear**	Cửa hàng trực tuyến bán phụ kiện chơi game (chuột, bàn phím LED).	Giao diện tối (Dark mode) với hiệu ứng Neon; Card sản phẩm có hiệu ứng phát sáng khi hover.
3	**Zen Spa & Wellness**	Trang giới thiệu dịch vụ Spa, Yoga và chăm sóc sức khỏe.	Bố cục tối giản (Minimalism); sử dụng nhiều khoảng trắng (Whitespace) và font chữ mềm mại.

STT	Chủ đề	Mô tả nội dung	Yêu cầu kỹ thuật đặc thù
4	**Edu-Stream**	Nền tảng học trực tuyến dành cho trẻ em hoặc lập trình viên.	Lưới danh sách khóa học sử dụng CSS Grid; có các nhãn (tags) phân loại màu sắc rực rỡ.
5	**Space Odyssey**	Trang web khám phá vũ trụ hoặc đặt vé du lịch không gian.	Hiệu ứng ảnh nền Parallax; sử dụng các hình khối bo tròn và hiệu ứng mờ (Glassmorphism).
6	**Urban Foodie**	Blog đánh giá ẩm thực đường phố hoặc ứng dụng giao đồ ăn.	Thanh điều hướng (Nav) cố định khi cuộn; danh sách đánh giá sử dụng Flexbox để dàn hàng.
7	**FitTrack App**	Landing Page giới thiệu một ứng dụng theo dõi sức khỏe/tập gym.	Hiển thị giao diện điện thoại (Mockup) bên cạnh các tính năng chính; nút tải app nổi bật.
8	**Vintage Watch**	Trang web giới thiệu bộ sưu tập đồng hồ cổ điển hoặc đồ xa xỉ.	Sử dụng font chữ Serif (có chân) tạo cảm giác sang trọng; hiệu ứng lật ảnh (Card Flip).
9	**Pet Paradise**	Dịch vụ chăm sóc, khách sạn và spa dành cho thú cưng.	Các nút bấm có hình thù ngộ nghĩnh; sử dụng CSS Variables để quản lý các màu Pastel.
10	**Crypto Pulse**	Trang web theo dõi thị trường tiền điện tử hoặc ví số.	Bảng dữ liệu giá (Table) được thiết kế Responsive; các thẻ thống kê nhanh (Stat cards).

Để buổi thi diễn ra khách quan, thầy có thể áp dụng các hình thức sau:

- **Bốc thăm ngẫu nhiên:** Chuẩn bị các phiếu ghi tên chủ đề, sinh viên bốc thăm trước khi bắt đầu tính giờ làm bài.
- **Phân chia theo số thứ tự:**
    - Sinh viên số chẵn: Chọn các chủ đề về Công nghệ/Gaming/Tài chính (Đề 2, 5, 10).
    - Sinh viên số lẻ: Chọn các chủ đề về Đời sống/Nghệ thuật/Môi trường (Đề 1, 3, 8).

> - **Yêu cầu "Mức độ khó" (Bonus points):** Thầy có thể quy định một số chủ đề là "Nâng cao" (ví dụ Đề 5 hoặc Đề 8) với các yêu cầu phức tạp hơn về Animation để sinh viên khá giỏi thử sức và nhận điểm cộng

Yêu cầu tối thiểu:

1. **Header:** Logo, Menu điều hướng (về Mobile chuyển thành Hamburger).
2. **Hero Section:** Tiêu đề lớn (Headline), Mô tả ngắn và 01 nút kêu gọi hành động (CTA).
3. **Features/Services Section:** Lưới hiển thị các tính năng hoặc dịch vụ (Dùng CSS Grid).
4. **Gallery/Products:** Hình ảnh minh họa (Dùng hiệu ứng Hover hoặc AOS).
5. **Footer:** Thông tin liên hệ, Mạng xã hội và Bản quyền

🛠 2. ĐOẠN MÃ MẪU (STARTER TEMPLATE)

Thầy có thể gửi cho sinh viên một file nén (.zip) chứa cấu trúc thư mục chuẩn để các em bắt đầu làm bài ngay lập tức.

Cấu trúc thư mục (Folder Structure):

LHU_MIDTERM_PROJECT/

├── assets/

│ ├── css/

│ │ └── style.css (Chứa khai báo :root và các reset CSS)

│ ├── images/ (Chứa ảnh mẫu hoặc ảnh sinh viên tự chọn)

│ └── vendor/ (Chứa thư viện AOS hoặc Animate.css)

├── index.html (File HTML chuẩn Semantic)

├── prompt_log.md (File để sinh viên ghi lại các câu lệnh AI)

└── REPORT.md (File báo cáo cuối giờ)

Mã mẫu cơ bản trong style.css:

CSS

```
/* Khai báo Design System cơ bản */
:root {
  --primary-color: #2563eb;
  --secondary-color: #64748b;
  --text-dark: #1e293b;
  --bg-light: #f8fafc;
  --spacing-base: 15px;
  --transition-speed: 0.3s;
}
/* Reset CSS */
* {
  margin: 0;
  padding: 0;
```

```
  box-sizing: border-box;
}
body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  line-height: 1.6;
  color: var(--text-dark);
  background-color: var(--bg-light);
}
```

🔍 3. HƯỚNG DẪN QUAN SÁT & ĐÁNH GIÁ TƯ DUY (OBSERVATION GUIDE)

Thầy có thể sử dụng bảng sau để ghi chú nhanh trong quá trình đi vòng quanh lớp học:

Tiêu chí quan sát	Biểu hiện tích cực	Dấu hiệu cần hỗ trợ/Lưu ý
Cách đặt câu hỏi cho AI	Prompt mô tả chi tiết yêu cầu kỹ thuật (Ví dụ: "Tạo grid 3 cột, gap 20px, responsive về 1 cột").	Prompt quá chung chung (Ví dụ: "Làm hộ tôi cái web cà phê").
Xử lý lỗi (Debug)	Biết copy mã lỗi từ Console hoặc mô tả hiện tượng "vỡ layout" cho AI phân tích.	Chỉ copy-paste code mới liên tục mà không hiểu tại sao code cũ bị lỗi.
Tư duy hệ thống	Biết yêu cầu AI sử dụng biến CSS đã khai báo sẵn thay vì để AI tự sinh màu mới.	Để AI sinh ra hàng loạt file CSS rời rạc, không nhất quán về màu sắc.
Kỹ năng Git	Thực hiện commit theo từng giai đoạn hoàn thiện (HTML xong -> Commit).	Chỉ có duy nhất 1 commit cuối giờ hoặc không commit được bài.

📝 4. BIỂU MẪU GIẢI TRÌNH (REPORT.MD TEMPLATE)

> Yêu cầu sinh viên điền vào file này trước khi nộp bài để thầy đánh giá được phần "Vibe" (tư duy) đằng sau phần "Code":

1. **Ý tưởng thiết kế:** (Mô tả ngắn gọn về phong cách bạn chọn).
2. **Top 3 Prompts thành công nhất:**
    - *Prompt 1:* ... (Giải thích tại sao dùng câu lệnh này).
    - *Prompt 2:* ...
    - *Prompt 3:* ...
3. **Lỗi khó nhất đã gặp và cách sửa:** (Mô tả lại quá trình phối hợp với AI để xử lý lỗi).

TUẦN 8: JAVASCRIPT FUNDAMENTALS & TƯ DUY

TUẦN 8: JAVASCRIPT FUNDAMENTALS & TƯ DUY LOGIC AI-FIRST

1. Mục tiêu bài học (Lesson Objectives)

- **Về kiến thức:** Hiểu vai trò của JS trong trình duyệt. Nắm vững các khái niệm nền tảng: Biến (Variables), Kiểu dữ liệu (Data types), Toán tử và Cấu trúc điều kiện (If-else/Switch case).
- **Về kỹ năng:** Sử dụng AI để giải thích các khối code logic phức tạp và thực hiện viết các hàm xử lý tính toán cơ bản trực tiếp trên Console của trình duyệt.
- **Về thái độ:** Chuyển đổi tư duy từ "Thiết kế hình ảnh" sang "Thiết kế quy trình logic".

2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO)

- **LLO 3.1.1:** Khai báo và sử dụng đúng các loại biến (let, const) và phân biệt được các kiểu dữ liệu cơ bản (String, Number, Boolean).
- **LLO 3.1.2:** Viết được các cấu trúc điều kiện để xử lý logic (ví dụ: kiểm tra tuổi, kiểm tra đăng nhập).
- **LLO 3.1.3:** Sử dụng Prompt để yêu cầu AI chuyển đổi một yêu cầu bằng ngôn ngữ tự nhiên thành mã JavaScript tương ứng.
- **LLO 3.1.4:** Thực hiện Debug cơ bản bằng cách sử dụng console.log() để kiểm tra giá trị của biến.

3. Nội dung lý thuyết tự học (Flipped Classroom)

STT	Chủ đề chính	Nội dung chi tiết	Link tài liệu tham khảo
1	**JS Variables**	Cách lưu trữ dữ liệu với let và const. Tại sao không nên dùng var?	JS Variables - W3Schools
2	**Data Types**	Tìm hiểu về Chuỗi, Số, Đúng/Sai và mảng cơ bản.	JS Data Types - W3Schools
3	**If...Else**	Cách điều hướng chương trình dựa trên điều kiện.	JS If...Else - W3Schools
4	**JS Output**	Các cách hiển thị kết quả: console.log(), alert(), innerHTML.	JS Output - W3Schools

4. Hoạt động thực hành trên lớp (In-class Activities)

Hoạt động 1: "Hello JS Console" - Đối thoại với trình duyệt (30 phút)

- **Hướng dẫn:** Mở tab Console (F12). Sinh viên thực hiện khai báo tên, tuổi và thực hiện các phép toán cộng trừ nhân chia trực tiếp.
- **Vibe Coding:** Sử dụng AI để giải thích: *"Tại sao khi cộng '10' + 20 trong JS lại ra '1020' mà không phải 30?"*.

Hoạt động 2: "Logic Builder" - Máy tính mini (45 phút)

- **Hướng dẫn:** Viết một đoạn mã JS nhỏ yêu cầu người dùng nhập vào số điểm. Nếu điểm >= 5 thì hiện thông báo "Chúc mừng bạn đã đạt!", ngược lại hiện "Rất tiếc, hãy cố gắng hơn!".
- **Prompt mẫu:** *"Viết một đoạn mã JavaScript sử dụng prompt() để lấy điểm số từ người dùng, sau đó sử dụng if-else để kiểm tra và alert() kết quả xếp loại: Giỏi (>8), Khá (>6.5), Trung bình (>=5), Yếu (<5)."*

> **Hoạt động 3: "AI Logic Explainer" (30 phút)**

- **Hướng dẫn:** Giảng viên đưa ra một đoạn mã lỗi hoặc mã khó hiểu. Sinh viên dùng AI để "dịch" mã đó sang ngôn ngữ tự nhiên để hiểu luồng chạy của chương trình.

5. 6 Bài tập tự rèn luyện tại nhà (Homework)

Bài 1: Chuyển đổi tiền tệ đơn giản

> - **Yêu cầu:** Viết mã JS cho phép nhập số lượng USD và tính ra số tiền VND tương ứng (giả sử tỉ giá 25.000). Kết quả hiện ra console.

Bài 2: Kiểm tra số chẵn/lẻ

> - **Yêu cầu:** Nhập một số bất kỳ từ bàn phím. Thông báo cho người dùng biết đó là số chẵn hay số lẻ.

Bài 3: Tính chỉ số BMI cơ bản

> - **Yêu cầu:** Nhập chiều cao (m) và cân nặng (kg). Tính BMI = cân nặng / (chiều cao * chiều cao) và in ra kết quả.

Bài 4: Logic Đăng nhập (Mô phỏng)

> - **Yêu cầu:** Tạo 2 biến username và password cố định. Viết cấu trúc if kiểm tra nếu người dùng nhập đúng cả 2 thì hiện "Đăng nhập thành công", nếu sai 1 trong 2 thì hiện "Thông tin không chính xác".

Bài 5: Máy tính giảm giá (Discount Calculator)

> - **Yêu cầu:** Nhập số tiền hóa đơn. Nếu số tiền > 500.000đ thì giảm giá 10%, ngược lại không giảm. Tính số tiền cuối cùng phải trả.

Bài 6: Phân loại tam giác

> - **Yêu cầu:** Nhập vào 3 cạnh của một tam giác. Kiểm tra xem đó có phải là tam giác đều không (3 cạnh bằng nhau).

> 💡 **Mẹo cho giảng viên:** Trong tuần đầu học JS, sinh viên rất dễ nản vì lỗi cú pháp (thiếu dấu ;, ngoặc {}). Hãy khuyến khích các em copy mã lỗi vào AI để nó giải thích lỗi sai thay vì sửa hộ ngay lập tức. Hãy tập trung vào **Tư duy thuật toán** hơn là thuộc lòng cú pháp.

🎯 THÁCH THỨC TUẦN 8: "THE LOGIC PUZZLE" (NHÓM)

🎯 THÁCH THỨC TUẦN 8: "THE LOGIC PUZZLE" (NHÓM)

- **Thử thách:** Hãy tích hợp một đoạn mã JS vào trang dự án giữa kỳ của nhóm. Cụ thể: Khi người dùng vào trang web, hãy hiện một thông báo chào mừng theo thời gian thực (Ví dụ: Nếu là buổi sáng thì hiện "Chào buổi sáng!", buổi chiều hiện "Chào buổi chiều!").

> - **Yêu cầu nộp:** Link GitHub và file logic_explanation.md giải thích cách các bạn dùng hàm new Date().getHours() để lấy thời gian.

HƯỚNG DẪN THỰC HÀNH CHI TIẾT (STEP-BY-STEP)

HƯỚNG DẪN THỰC HÀNH CHI TIẾT (STEP-BY-STEP) - TUẦN 8

HOẠT ĐỘNG 1: "HELLO JS CONSOLE" - ĐỐI THOẠI VỚI TRÌNH DUYỆT

1. Phân tích yêu cầu

- **Mục tiêu:** Làm quen với môi trường chạy mã JavaScript trực tiếp (Console) mà không cần tạo file.
- **Thao tác:** Sử dụng các lệnh cơ bản để lưu trữ dữ liệu và hiển thị kết quả.

2. Viết Prompt thực hiện (Thực hành cùng AI)

- **Prompt 1 (Giải thích):** *"Hãy giải thích cho tôi sự khác biệt giữa let và const trong JavaScript bằng ví dụ cực kỳ đơn giản."*
- **Prompt 2 (Thực hành):** *"Viết cho tôi các dòng code để thực hiện trên Console: khai báo một biến tên, một biến năm sinh, sau đó tính tuổi và in ra câu 'Chào [Tên], năm nay bạn [Số tuổi] tuổi'."*

3. Đánh giá & Kiểm tra (Checklist)

- [ ] Đã mở được tab Console bằng phím **F12** (hoặc chuột phải -> Inspect).
- [ ] Sử dụng đúng console.log() để in kết quả.
- [ ] Hiểu tại sao dùng const cho năm sinh và let cho tuổi (nếu tuổi có thay đổi).

HOẠT ĐỘNG 2: "LOGIC BUILDER" - MÁY TÍNH XẾP LOẠI MINI

1. Phân tích yêu cầu

- **Input:** Nhận một số điểm từ người dùng thông qua cửa sổ nhập liệu (prompt).
- **Logic:** Sử dụng cấu trúc if-else if-else để phân loại.
- **Output:** Hiển thị thông báo kết quả qua alert.

2. Viết Prompt thực hiện (Thực hành cùng AI)

- **Prompt 1 (Viết code):** *"Viết một đoạn mã JavaScript: Yêu cầu người dùng nhập điểm (0-10). Nếu điểm >= 8 in 'Giỏi', >= 6.5 in 'Khá', >= 5 in 'Trung bình', còn lại in 'Yếu'. Sử dụng câu lệnh prompt() và alert()."*
- **Prompt 2 (Tối ưu):** *"Làm thế nào để kiểm tra nếu người dùng nhập một giá trị không phải là số hoặc nhập điểm lớn hơn 10?"*

3. Đánh giá & Kiểm tra (Checklist)

- [ ] Cửa sổ prompt có xuất hiện khi tải lại trang không?
- [ ] Các điều kiện if-else có được đặt theo thứ tự từ cao xuống thấp không? (Tránh lỗi logic).
- [ ] Đã thử nhập các giá trị biên (như 4.9, 5.0, 8.0) để kiểm tra độ chính xác chưa?

HOẠT ĐỘNG 3: "AI LOGIC EXPLAINER" - GIẢI MÃ MÃ NGUỒN

1. Phân tích yêu cầu

- **Mục tiêu:** Đọc hiểu luồng chạy của chương trình (Tư duy thuật toán).
- **Tình huống:** Giảng viên đưa ra đoạn mã về cấu trúc Switch Case hoặc Toán tử logic (&&, ||).

2. Viết Prompt thực hiện (Thực hành cùng AI)

- **Prompt (Phân tích):** *"Hãy giải thích từng dòng của đoạn mã sau đây hoạt động như thế nào và cho biết kết quả cuối cùng là gì:
- JavaScript

```
let hour = 19;
```

if (hour < 12) {

```
    console.log('Chào buổi sáng');
} else if (hour < 18) {
    console.log('Chào buổi chiều');
} else {
    console.log('Chào buổi tối');
}
```

```"*

### 3. Cách kiểm tra công việc (Final Checklist)

| STT | Tiêu chí kiểm tra | Kết quả (Đạt/Không) |
|---|---|---|
| 1 | Khai báo biến thành công mà không bị lỗi SyntaxError. | |
| 2 | Hiểu cách sử dụng dấu ngoặc nhọn {} cho khối lệnh if. | |
| 3 | Sử dụng được console.log() để kiểm tra giá trị của biến (Debug). | |
| 4 | Giải thích được tại sao một đoạn code không chạy theo ý muốn. | |

---

### Gợi ý cho giảng viên:

Trong buổi học này, sinh viên thường gặp lỗi **"Lỗi cú pháp"** (Syntax Error). Thầy hãy khuyến khích các em:

1. **Đọc lỗi:** Nhìn vào dòng chữ đỏ trong Console để xem nó báo lỗi ở dòng mấy.
2. **Hỏi AI:** *"Dán đoạn code lỗi vào AI và hỏi: 'Tại sao đoạn code này báo lỗi Uncaught SyntaxError?'"*.
3. **Tư duy:** Luôn nhắc sinh viên rằng: *"Code là ngôn ngữ, logic là suy nghĩ. AI giúp bạn viết ngôn ngữ, nhưng bạn phải là người điều khiển suy nghĩ"*.
```

TUẦN 9: TỰ ĐỘNG HÓA VỚI VÒNG LẶP

TUẦN 9: TỰ ĐỘNG HÓA VỚI VÒNG LẶP & TƯ DUY ĐÓNG GÓI VỚI HÀM

1. Mục tiêu bài học (Lesson Objectives)

- **Về kiến thức:** Hiểu cách hoạt động của vòng lặp for và while. Nắm vững khái niệm hàm (Function), tham số (Parameters) và giá trị trả về (Return).
- **Về kỹ năng:** Vận dụng vòng lặp để xử lý danh sách dữ liệu. Biết cách đóng gói mã nguồn vào các hàm để tái sử dụng nhiều lần.
- **Về thái độ:** Hình thành tư duy "Don't Repeat Yourself" (DRY) – Đừng viết lại những gì đã có.

2. Chuẩn đầu ra bài học (Lesson Learning Outcomes - LLO)

- **LLO 3.2.1:** Sử dụng vòng lặp for để in ra dãy số hoặc duyệt qua các phần tử đơn giản.
- **LLO 3.2.2:** Khai báo và gọi hàm thành công, truyền được dữ liệu vào hàm qua tham số.
- **LLO 3.2.3:** Sử dụng Prompt để yêu cầu AI viết lại một đoạn code lặp lại thành một hàm tối ưu.
- **LLO 3.2.4:** Phân biệt được phạm vi biến (Scope) bên trong và bên ngoài hàm.

3. Nội dung lý thuyết tự học (Flipped Classroom)

STT	Chủ đề chính	Nội dung chi tiết	Link tài liệu tham khảo
1	**JS For Loop**	Cách lặp lại một khối mã với số lần xác định.	JS For Loop - W3Schools
2	**JS While Loop**	Lặp lại mã khi một điều kiện vẫn còn đúng.	JS While Loop - W3Schools
3	**JS Functions**	Cách định nghĩa và gọi một hàm xử lý công việc.	JS Functions - W3Schools
4	**Function Return**	Cách hàm trả về một kết quả sau khi tính toán.	JS Return Statement

4. Hoạt động thực hành trên lớp (In-class Activities)

Hoạt động 1: "The Counting Machine" - Máy đếm tự động (30 phút)

- **Hướng dẫn:** Sử dụng vòng lặp for để in ra dãy số từ 1 đến 100 trên Console. Sau đó tùy chỉnh để chỉ in ra các số chẵn.
- **Vibe Coding:** Sử dụng AI để đặt câu hỏi: *"Hãy giải thích cho tôi 3 thành phần bên trong ngoặc đơn của vòng lặp for (khởi tạo; điều kiện; bước nhảy) hoạt động như thế nào?"*

Hoạt động 2: "Function Factory" - Nhà máy xử lý dữ liệu (45 phút)

- **Hướng dẫn:** Viết một hàm tinhDienTich(dai, rong) để tính diện tích hình chữ nhật. Gọi hàm này 3 lần với các thông số khác nhau.

- **Prompt mẫu:** *"Viết một hàm JavaScript nhận vào tên người dùng và năm sinh, hàm này sẽ trả về chuỗi: 'Chào [Tên], bạn hiện tại [Số tuổi] tuổi'. Hãy hướng dẫn tôi cách gọi hàm và in kết quả ra console."*

Hoạt động 3: "Refactoring Challenge" (30 phút)

- **Hướng dẫn:** Giảng viên đưa ra một đoạn code tính toán lặp lại nhiều lần. Sinh viên sử dụng AI để yêu cầu: *"Hãy đóng gói đoạn code tính thuế thu nhập này thành một hàm để tôi có thể tái sử dụng cho nhiều mức lương khác nhau"*.

5. 6 Bài tập tự rèn luyện tại nhà (Homework)

Bài 1: Bảng cửu chương 7

- **Yêu cầu:** Sử dụng vòng lặp for để in ra bảng cửu chương 7 trên Console (7 x 1 = 7, ..., 7 x 10 = 70).

Bài 2: Tính tổng dãy số

- **Yêu cầu:** Viết một hàm nhận vào số n. Sử dụng vòng lặp để tính tổng các số từ 1 đến n và trả về kết quả.

Bài 3: Kiểm tra số nguyên tố cơ bản

- **Yêu cầu:** Viết hàm nhận vào một số. Kiểm tra xem số đó có chia hết cho số nào khác ngoài 1 và chính nó không (Sử dụng vòng lặp và toán tử chia lấy dư %).

Bài 4: Máy tính cá nhân (Calculator Function)

- **Yêu cầu:** Viết một hàm nhận vào 3 tham số: so1, so2 và phepTinh (+, -, *, /). Trả về kết quả tương ứng.

Bài 5: Tìm số lớn nhất trong 3 số

- **Yêu cầu:** Viết hàm timMax(a, b, c) sử dụng các câu lệnh if bên trong để trả về số lớn nhất.

Bài 6: Vòng lặp "Chào hỏi"

- **Yêu cầu:** Tạo một mảng chứa 5 tên thành viên trong nhóm. Sử dụng vòng lặp để in ra câu chào từng người trên Console.

THÁCH THỨC TUẦN 9: "THE SMART INTERFACE" (NHÓM)

- **Thử thách:** Quay lại dự án giữa kỳ. Hãy tạo một hàm JS để xử lý việc hiển thị danh sách sản phẩm/dịch vụ. Thay vì viết HTML cho từng sản phẩm, hãy dùng vòng lặp để tự động sinh ra giao diện từ một danh sách dữ liệu mẫu.
- **Yêu cầu nộp:** Link GitHub và file function_logic.md giải thích cách bạn truyền dữ liệu vào hàm để hiển thị thông tin.

Mẹo cho giảng viên: Tuần này sinh viên sẽ bắt đầu thấy khó ở khái niệm return (giá trị trả về). Thầy hãy lấy ví dụ thực tế: *"Hàm giống như một cái máy xay sinh tố, bạn đưa hoa quả (tham số)*

> *vào, máy xử lý và đưa lại cho bạn ly sinh tố (return). Nếu không có return, máy chỉ xay xong rồi để đó, bạn không lấy ra uống được."*

TUẦN 10: THAO TÁC VỚI DOM & XỬ LÝ SỰ KIỆN

TUẦN 10: THAO TÁC VỚI DOM & XỬ LÝ SỰ KIỆN (DOM MANIPULATION)

1. Mục tiêu bài học

- **Về kiến thức:** Hiểu cấu trúc cây DOM và cách trình duyệt quản lý các thẻ HTML. Nắm vững các phương thức truy xuất phần tử và lắng nghe sự kiện (Event Listening).
- **Về kỹ năng:** Có thể dùng JS để thay đổi nội dung, thuộc tính và CSS của một thẻ. Tạo được các tương tác cơ bản như bấm nút hiện thông báo, thay đổi ảnh khi click.
- **Về thái độ:** Chú trọng vào trải nghiệm người dùng (UX) thông qua các tương tác phản hồi nhanh.

---

2. Chuẩn đầu ra bài học (LLO)

- **LLO 3.3.1:** Sử dụng thành thạo querySelector và getElementById để chọn các phần tử trên trang.
- **LLO 3.3.2:** Thay đổi được nội dung văn bản (innerText, innerHTML) và thuộc tính (src, href) của phần tử.
- **LLO 3.3.3:** Viết được hàm xử lý sự kiện addEventListener cho các hành động click, input, submit.
- **LLO 3.3.4:** Sử dụng Prompt để yêu cầu AI tạo các đoạn mã tương tác phức tạp (ví dụ: đóng/mở Modal, Menu mobile).

---

3. Nội dung lý thuyết tự học (Flipped Classroom)

STT	Chủ đề chính	Nội dung chi tiết	Link tài liệu tham khảo
1	**What is DOM?**	Khái niệm về cây phân cấp các đối tượng trong tài liệu HTML.	JS DOM Intro - W3Schools
2	**DOM Elements**	Cách tìm các phần tử HTML bằng ID, Class, hoặc Tag name.	JS DOM Elements - W3Schools
3	**DOM HTML**	Thay đổi nội dung và thuộc tính (Attribute) của thẻ.	JS DOM HTML - W3Schools
4	**DOM Events**	Cách lắng nghe và phản hồi lại các hành động từ người dùng.	JS DOM Events - W3Schools

---

4. Hoạt động thực hành trên lớp (In-class Activities)

Hoạt động 1: "The Element Picker" - Truy tìm thẻ (30 phút)

- **Hướng dẫn:** Sinh viên sử dụng console.log(document.querySelector('.class-name')) để tìm các thành phần trong dự án của mình.

- **Vibe Coding:** Sử dụng AI để hỏi: *"Làm thế nào để chọn tất cả các thẻ bên trong một có id là 'menu' và đổi màu chữ của chúng sang màu đỏ?"*

> **Hoạt động 2: "Light Switch" - Bật/Tắt bóng đèn (45 phút)**

- **Hướng dẫn:** Tạo một nút bấm và một hình ảnh cái đèn. Khi bấm nút, hình ảnh đổi từ "đèn tắt" sang "đèn sáng" bằng cách thay đổi thuộc tính src.
- **Prompt mẫu:** *"Viết mã JS để khi tôi click vào một nút có id là 'toggle-btn', nó sẽ thay đổi màu nền của body từ trắng sang đen và ngược lại (Dark mode đơn giản)."*

> **Hoạt động 3: "Live Input" - Phản hồi tức thì (30 phút)**

- **Hướng dẫn:** Tạo một ô <input>. Khi người dùng gõ chữ vào ô đó, nội dung ngay lập tức xuất hiện ở một thẻ bên cạnh.
- **Prompt mẫu:** *"Sử dụng sự kiện 'input' trong JavaScript để lấy giá trị từ một ô text và hiển thị nó ngay lập tức vào một thẻ h2 bên dưới."*

---

5. 6 Bài tập tự rèn luyện tại nhà (Homework)

Bài 1: Thay đổi lời chào theo tên

> - **Yêu cầu:** Tạo 1 ô input và 1 nút bấm. Khi nhấn nút, hiển thị thông báo "Chào mừng [Tên người dùng]!" lên trang web thay vì dùng alert.

Bài 2: Bộ sưu tập ảnh (Simple Gallery)

> - **Yêu cầu:** Có 1 ảnh lớn và 3 ảnh nhỏ bên dưới. Khi click vào ảnh nhỏ nào, ảnh lớn sẽ thay đổi thành ảnh đó.

Bài 3: Danh sách việc cần làm (To-Do List cơ bản)

> - **Yêu cầu:** Cho phép người dùng nhập công việc vào ô input và nhấn "Thêm". Công việc mới sẽ xuất hiện dưới dạng một thẻ <li> mới trong danh sách.

Bài 4: Kiểm tra độ dài mật khẩu

> - **Yêu cầu:** Khi người dùng gõ vào ô mật khẩu, nếu độ dài < 8 ký tự thì hiện thông báo màu đỏ "Mật khẩu quá ngắn", nếu >= 8 ký tự thì hiện màu xanh "Mật khẩu hợp lệ".

Bài 5: Thay đổi kích thước văn bản

> - **Yêu cầu:** Tạo 2 nút "Phóng to" và "Thu nhỏ". Khi nhấn, kích thước chữ của đoạn văn bản tăng hoặc giảm 2px.

Bài 6: Ẩn/Hiện mật khẩu

> - **Yêu cầu:** Tạo một ô input mật khẩu và một nút "Hiện". Khi nhấn, chuyển type="password" thành type="text" và ngược lại.

🎯 THÁCH THỨC TUẦN 10: "THE SMART PROJECT" (NHÓM)

- **Thử thách:** Tích hợp logic vào trang dự án: Tạo một nút "Mua hàng" hoặc "Liên hệ". Khi nhấn vào, hãy hiển thị một **Modal (Hộp thoại)** chứa form đăng ký mà không làm tải lại trang. Sử dụng JS để đóng/mở Modal này.
- **Yêu cầu nộp:** Link GitHub và file dom_interaction.md mô tả các sự kiện (event) bạn đã sử dụng.

💡 **Gợi ý cho giảng viên:** Tuần này sinh viên sẽ rất hứng thú vì thấy trang web "động". Thầy hãy nhấn mạnh rằng: *"HTML là xương, CSS là da thịt, và DOM chính là hệ thần kinh giúp website phản ứng với môi trường."* Đừng quên nhắc các em kiểm tra kỹ các ID/Class để tránh lỗi null khi chọn phần tử.

TUẦN 11: XỬ LÝ FORM & KIỂM SOÁT DỮ LIỆU

TUẦN 11: XỬ LÝ FORM & KIỂM SOÁT DỮ LIỆU (FORM HANDLING & VALIDATION)

1. Mục tiêu bài học

- **Về kiến thức:** Hiểu cách JavaScript can thiệp vào quy trình gửi dữ liệu của Form. Nắm vững các kỹ thuật kiểm tra tính hợp lệ của dữ liệu (Validation) trước khi gửi đi.
- **Về kỹ năng:** Ngăn chặn được hành vi tải lại trang mặc định của Form. Xây dựng được các thông báo lỗi thông minh và trực quan cho người dùng.
- **Về thái độ:** Đề cao tính bảo mật và trải nghiệm người dùng (tránh để người dùng nhập sai định dạng dữ liệu).

2. Chuẩn đầu ra bài học (LLO)

- **LLO 3.4.1:** Sử dụng event.preventDefault() để kiểm soát quy trình submit Form.
- **LLO 3.4.2:** Truy xuất được dữ liệu từ các loại input khác nhau (text, checkbox, radio, select).
- **LLO 3.4.3:** Viết được logic kiểm tra dữ liệu: không để trống, đúng định dạng email, độ dài mật khẩu.
- **LLO 3.4.4:** Sử dụng Prompt để yêu cầu AI tạo các biểu thức chính quy (Regex) cơ bản để kiểm tra số điện thoại hoặc email.

---

3. Nội dung lý thuyết tự học (Flipped Classroom)

STT	Chủ đề chính	Nội dung chi tiết	Link tài liệu tham khảo
1	**JS Forms**	Cách truy cập các thành phần của Form qua DOM.	JS Forms - W3Schools
2	**Validation API**	Các thuộc tính kiểm tra có sẵn của HTML5 (required, min, max).	Constraint Validation API
3	**Event: Submit**	Cách lắng nghe sự kiện gửi form và xử lý dữ liệu.	JS Form Events

---

4. Hoạt động thực hành trên lớp (In-class Activities)

Hoạt động 1: "Stop the Page!" - Chặn tải lại trang (30 phút)

- **Hướng dẫn:** Tạo một Form đơn giản. Khi nhấn "Gửi", trang web thường bị load lại. Sinh viên dùng event.preventDefault() để chặn hành vi này và in dữ liệu ra console.log.
- **Vibe Coding:** Sử dụng AI để hỏi: *"Tại sao tôi cần dùng preventDefault() khi xử lý Form bằng JavaScript?"*.

Hoạt động 2: "Smart Validator" - Bộ kiểm tra thông minh (45 phút)

- **Hướng dẫn:** Xây dựng Form đăng ký. Nếu ô Email không có chữ "@" hoặc ô Mật khẩu trống, hãy hiển thị thông báo lỗi màu đỏ ngay dưới ô input đó.
- **Prompt mẫu:** *"Viết mã JS để kiểm tra một Form: Nếu ô 'username' ít hơn 5 ký tự, hãy hiển thị lỗi 'Tên quá ngắn' vào một thẻ span có class 'error-msg' và đổi border của input sang màu đỏ."*

Hoạt động 3: "Data Collector" - Thu thập dữ liệu (30 phút)

- **Hướng dẫn:** Thu thập dữ liệu từ Checkbox (Sở thích) và Radio button (Giới tính) để tạo thành một đối tượng (Object) dữ liệu người dùng hoàn chỉnh.

---

5. 6 Bài tập tự rèn luyện tại nhà (Homework)

- **Bài 1: Form liên hệ cơ bản**: Tạo form gồm Tên, Email, Nội dung. Chỉ cho phép gửi khi tất cả các trường đã được điền.
- **Bài 2: Kiểm tra mật khẩu khớp nhau**: Tạo 2 ô "Mật khẩu" và "Nhập lại mật khẩu". Hiển thị thông báo "Mật khẩu không khớp" nếu dữ liệu ở 2 ô khác nhau.
- **Bài 3: Bộ chọn màu nền (Color Picker)**: Tạo một ô input type="color". Khi người dùng chọn màu, màu nền của trang web sẽ thay đổi theo.
- **Bài 4: Tính tổng đơn hàng**: Tạo form gồm các checkbox sản phẩm kèm giá tiền. Khi người dùng tick vào, tổng số tiền sẽ tự động cập nhật bên dưới.
- **Bài 5: Regex Email Checker**: Sử dụng AI để lấy một đoạn mã Regex kiểm tra định dạng email và áp dụng vào Form của bạn.
- **Bài 6: Khóa nút Submit**: Nút "Gửi" sẽ bị mờ (disabled) cho đến khi người dùng tick vào ô "Tôi đồng ý với điều khoản dịch vụ".

🎯 THÁCH THỨC TUẦN 11: "THE SECURE FORM" (NHÓM)

- **Thử thách:** Hoàn thiện trang "Liên hệ" hoặc "Đăng ký" trong dự án nhóm. Form phải có:
  1. Kiểm tra dữ liệu trực tiếp khi người dùng đang gõ (Real-time validation).
  2. Thông báo thành công mượt mà (dùng setTimeout để ẩn thông báo sau 3 giây).
- **Yêu cầu nộp:** Link GitHub và file form_logic.md mô tả các bước kiểm tra dữ liệu bạn đã thiết lập.

💡 **Mẹo cho giảng viên:** Tuần này rất quan trọng về mặt tư duy logic. Thầy hãy nhắc sinh viên: *"Đừng bao giờ tin tưởng dữ liệu người dùng nhập vào. Hãy luôn kiểm tra (Validate) thật kỹ trước khi làm bất cứ điều gì tiếp theo"*. Đây là bước đầu tiên để các em hiểu về bảo mật web cơ bản.

TUẦN 12: QUẢN LÝ DỮ LIỆU VỚI MẢNG NÂNG CAO

TUẦN 12: QUẢN LÝ DỮ LIỆU VỚI MẢNG NÂNG CAO & ĐỐI TƯỢNG (DATA MANAGEMENT)

1. Mục tiêu bài học

- **Về kiến thức:** Hiểu cách lưu trữ dữ liệu phức tạp dưới dạng Object (thuộc tính và phương thức). Nắm vững các hàm duyệt mảng hiện đại (`forEach`, `map`, `filter`).
- **Về kỹ năng:** Có khả năng tổ chức dữ liệu cho một danh sách sản phẩm (tên, giá, hình ảnh) và lọc dữ liệu theo yêu cầu (ví dụ: tìm sản phẩm giá dưới 500k).
- **Về thái độ:** Rèn luyện tư duy quản lý dữ liệu hệ thống, chuẩn bị cho việc làm việc với API ở các cấp độ cao hơn.

2. Chuẩn đầu ra bài học (LLO)

- **LLO 3.5.1:** Khai báo và truy xuất dữ liệu từ Object (Đối tượng) thành thạo.
- **LLO 3.5.2:** Sử dụng `filter()` để lọc danh sách dữ liệu và `map()` để hiển thị dữ liệu ra giao diện HTML.
- **LLO 3.5.3:** Kết hợp kiến thức Tuần 10 (DOM) để đổ dữ liệu từ một mảng Object lên trang web một cách tự động.
- **LLO 3.5.4:** Sử dụng Prompt để yêu cầu AI tạo cấu trúc dữ liệu JSON mẫu cho các chủ đề dự án khác nhau.

---

3. Nội dung lý thuyết tự học (Flipped Classroom)

STT	Chủ đề chính	Nội dung chi tiết	Link tài liệu tham khảo
1	JS Objects	Cách định nghĩa đối tượng, thuộc tính và phương thức.	JS Objects - W3Schools
2	Array Iteration	Các hàm duyệt mảng hiện đại: `forEach`, `map`, `filter`.	JS Array Iteration - W3Schools
3	JSON Intro	Làm quen với định dạng dữ liệu phổ biến nhất hiện nay.	JS JSON - W3Schools

---

4. Hoạt động thực hành trên lớp (In-class Activities)

Hoạt động 1: "Object Identity" - Định danh đối tượng (30 phút)

- **Hướng dẫn:** Mỗi sinh viên tạo một Object `user` chứa thông tin bản thân (tên, tuổi, chuyên ngành, kỹ năng). Sau đó dùng `console.log` để in ra một câu giới thiệu từ Object đó.
- **Vibe Coding:** Sử dụng AI để hỏi: *"Làm thế nào để thêm một phương thức (function) vào bên trong Object để nó tự tính năm sinh dựa trên thuộc tính tuổi?"*.

Hoạt động 2: "The Product Filter" - Bộ lọc sản phẩm (45 phút)

- **Hướng dẫn:** Giảng viên cung cấp một mảng chứa 10 Object sản phẩm (mỗi sản phẩm có tên và giá). Sinh viên dùng hàm `filter()` để tạo ra một mảng mới chỉ chứa các sản phẩm "Đang giảm giá".
- **Prompt mẫu:** *"Tôi có một mảng danh sách sản phẩm, hãy viết mã JS dùng hàm filter để lọc ra các sản phẩm có giá lớn hơn 1 triệu đồng và dùng map để in tên của chúng ra màn hình."*.

Hoạt động 3: "Dynamic UI Render" - Đổ dữ liệu tự động (30 phút)

- **Hướng dẫn:** Thay vì viết HTML thủ công cho từng sản phẩm, sinh viên dùng vòng lặp `map()` hoặc `forEach()` để tự động tạo ra các thẻ `div.card` trên trang web từ mảng dữ liệu.

---

5. 6 Bài tập tự rèn luyện tại nhà (Homework)

- **Bài 1: Danh bạ điện thoại**: Tạo mảng các Object chứa tên và số điện thoại. Viết hàm tìm kiếm số điện thoại khi nhập tên.
- **Bài 2: Tính tổng giỏ hàng**: Cho một mảng các sản phẩm trong giỏ hàng (tên, giá, số lượng). Tính tổng số tiền mà người dùng phải trả.
- **Bài 3: Sắp xếp danh sách**: Sử dụng hàm `sort()` để sắp xếp mảng sản phẩm theo giá từ thấp đến cao.
- **Bài 4: Quản lý sinh viên**: Tạo Object `sinhVien` có thuộc tính điểm số. Viết hàm kiểm tra nếu điểm > 5 thì thêm thuộc tính `ketQua: "Đậu"`, ngược lại là "Rớt".
- **Bài 5: Biến đổi mảng**: Sử dụng `map()` để biến một mảng tên thành một mảng các thẻ `<li>Tên</li>`.
- **Bài 6: Xóa phần tử**: Viết logic để khi click vào một nút "Xóa" cạnh sản phẩm, sản phẩm đó sẽ biến mất khỏi danh sách hiển thị.

🎯 THÁCH THỨC TUẦN 12: "THE SMART CATALOG" (NHÓM)

- **Thử thách:** Nâng cấp dự án nhóm bằng cách quản lý dữ liệu tập trung.
    1. Tạo một file `data.js` chứa mảng các Object đại diện cho dịch vụ/sản phẩm của nhóm.
    2. Viết mã JS để tự động hiển thị toàn bộ dữ liệu này lên trang chủ.
    3. Thêm một ô "Tìm kiếm" để lọc nhanh sản phẩm theo tên ngay trên giao diện.
- **Yêu cầu nộp:** Link GitHub và file `data_logic.md` giải thích cấu trúc Object mà nhóm đã thiết kế.

💡 **Mẹo cho giảng viên:** Đây là tuần "kết tinh" logic. Hãy giúp sinh viên hiểu rằng: **Data (Dữ liệu) + Logic (JavaScript) = Giao diện (UI)**. Nếu quản lý dữ liệu tốt, việc thay đổi giao diện sau này sẽ cực kỳ dễ dàng mà không cần động vào HTML.

PBL - Agile (Scrum)

TUẦN 13: KHỞI ĐỘNG DỰ ÁN CUỐI KHÓA (PBL - PHASE 1: PLANNING & PROTOTYPING)

1. Mục tiêu bài học (Lesson Objectives)

- **Về kiến thức:** Hiểu quy trình phát triển dự án phần mềm theo mô hình Agile/Scrum đơn giản.
- **Về kỹ năng:** Lập kế hoạch dự án, phân chia công việc (Task management), và xây dựng luồng người dùng (User Flow) hoàn chỉnh.
- **Về thái độ:** Rèn luyện kỹ năng làm việc nhóm, trách nhiệm cá nhân và tư duy giải quyết vấn đề thực tế.

2. Chuẩn đầu ra bài học (LLO)

- **LLO 4.1.1:** Xác định được bài toán thực tế và đề xuất giải pháp web tương ứng (E-commerce, Portfolio, Education, v.v.).
- **LLO 4.1.2:** Sử dụng AI để liệt kê các chức năng chính (User Stories) và xây dựng cấu trúc thư mục dự án chuyên nghiệp.
- **LLO 4.1.3:** Phân chia nhiệm vụ trên GitHub Projects hoặc Trello để theo dõi tiến độ.
- **LLO 4.1.4:** Thiết kế được sơ đồ luồng dữ liệu (Data Flow) cơ bản cho ứng dụng.

3. Quy trình triển khai PBL (Project Lifecycle)

Giai đoạn	Hoạt động chính	Vai trò của AI (Vibe Coding)
Phân tích	Xác định đối tượng mục tiêu và các tính năng "phải có".	Prompt: *"Phân tích các chức năng cần thiết cho một web bán đồ điện tử dành cho sinh viên."*
Thiết kế	Vẽ Wireframe, xác định luồng đi của người dùng (User Flow).	Prompt: *"Mô tả luồng người dùng từ lúc vào trang chủ đến khi hoàn tất thanh toán."*
Kỹ thuật	Quyết định cấu trúc Data Object và các hàm xử lý cốt lõi.	Prompt: *"Thiết kế cấu trúc JSON cho danh sách sản phẩm gồm: id, tên, giá, ảnh, đánh giá."*

4. Hoạt động thực hành trên lớp (In-class Activities)

Hoạt động 1: "Project Pitching" - Bảo vệ ý tưởng (45 phút)

- **Hướng dẫn:** Mỗi nhóm có 5 phút để thuyết trình về dự án mình sẽ thực hiện.
- **Nội dung:** Tên dự án, lý do chọn đề tài, các tính năng sử dụng JavaScript (ví dụ: giỏ hàng, lọc tìm kiếm, validation form).
- **Mentor phản hồi:** Thầy sẽ đánh giá tính khả thi và gợi ý điều chỉnh phạm vi (Scope) để đảm bảo hoàn thành đúng hạn.

Hoạt động 2: "The Blueprint" - Thiết kế hệ thống (45 phút)

- **Hướng dẫn:** Nhóm sử dụng AI để xây dựng cấu trúc dữ liệu và sơ đồ xử lý.

- **Prompt mẫu:** *"Dựa trên đề tài 'Web đặt lịch Spa', hãy liệt kê các hàm JavaScript cần viết để xử lý việc chọn dịch vụ và tính tổng tiền."*

Hoạt động 3: "GitHub Workflow" (30 phút)

- **Hướng dẫn:** Thiết lập Repository nhóm, mời thành viên và tạo các "Issue" (nhiệm vụ) đầu tiên trên GitHub.

5. Thách thức dành cho dự án (Project Challenges)

Để dự án đạt chuẩn "Vibe & Code", thầy yêu cầu mỗi nhóm phải vượt qua 3 thử thách kỹ thuật:

1. **Thử thách Dynamic:** Website tuyệt đối không dùng HTML tĩnh cho danh sách sản phẩm/dịch vụ. Toàn bộ phải được render từ mảng Object (Tuần 12).
2. **Thử thách UX:** Phải có ít nhất một chức năng "Real-time" (ví dụ: tìm kiếm đến đâu hiện kết quả đến đó, hoặc thông báo lỗi ngay khi gõ form).
3. **Thử thách Clean Code:** Mã nguồn phải được tổ chức thành các Module rõ ràng: data.js (dữ liệu), ui.js (xử lý hiển thị), app.js (logic chính).

6. Danh mục Checklist cho sinh viên (PBL Milestone 1)

STT	Tiêu chí hoàn thiện	Trạng thái
1	Có file PLAN.md mô tả chi tiết các chức năng sẽ làm.	
2	Cấu trúc dữ liệu Object đã hoàn thiện và được AI kiểm tra logic.	
3	Phân chia công việc rõ ràng trên GitHub (Ai làm Header, ai làm Logic giỏ hàng...).	
4	Có ít nhất 01 bản vẽ phác thảo (Wireframe) bằng giấy hoặc công cụ online.	

Lời khuyên cho thầy (The Mentor's Tip):

Trong phương pháp PBL, sinh viên sẽ hỏi thầy rất nhiều về "Làm sao để làm tính năng X?". Thay vì trả lời ngay, thầy hãy hướng dẫn các em cách đặt câu hỏi cho AI: *"Em hãy thử mô tả logic bước-bằng-bước cho AI và xem nó gợi ý thuật toán gì, sau đó thầy sẽ giúp em tối ưu đoạn mã đó"*. Điều này giúp sinh viên hình thành tư duy tự học bền vững.

TUẦN 13: KHỞI ĐỘNG DỰ ÁN & SPRINT 1 (PLANNING & CORE UI)

Mục tiêu PBL: Xác định vấn đề và xây dựng kế hoạch giải quyết. **Mục tiêu Agile:** Thiết lập Product Backlog và hoàn thành bộ khung giao diện.

1. Lý thuyết (45 phút): Agile Scrum & Vibe Coding Workflow

- **Tư duy Agile:** Tập trung vào việc tạo ra sản phẩm có thể chạy được (MVP - Minimum Viable Product) thay vì đợi hoàn hảo mới trình làng.
- **Scrum Roles:**

    - **Scrum Master (Trưởng nhóm):** Điều phối, gỡ bỏ các rào cản kỹ thuật.
    - **Product Owner (Chủ dự án):** Quyết định tính năng nào quan trọng nhất dựa trên nhu cầu người dùng.
- **Scrum Artifacts:**
    - **Product Backlog:** Danh sách "việc cần làm" được sắp xếp theo ưu tiên.
    - **Sprint Backlog:** Những việc cụ thể sẽ làm trong tuần 13.

2. Thực hành PBL - Giai đoạn "Khám phá & Thiết kế"

- **Hoạt động 1: Sprint Planning (Lập kế hoạch):**
    - Nhóm thảo luận và chia nhỏ dự án thành các **Issues** trên GitHub Projects.
    - Phân chia vai trò: Ai làm UI, ai làm Logic dữ liệu.
- **Hoạt động 2: Vibe Coding - Xây dựng "Skeleton":**
    - Thiết lập cấu trúc thư mục chuẩn: `assets/`, `css/`, `js/`, `data/`.
    - Sử dụng AI để tạo nhanh khung HTML Semantic và Layout CSS (Grid/Flexbox).
- **Prompt mẫu:** *"Dựa trên mô hình Scrum, hãy giúp nhóm tôi chia dự án 'Web bán giày' thành 5 User Stories quan trọng nhất để làm trong Sprint 1 này."*.

🧮 TUẦN 14: SPRINT 2 - LOGIC THÔNG MINH & KIỂM THỬ (FINAL PUSH)

Mục tiêu PBL: Thực thi giải pháp và đánh giá chất lượng sản phẩm. **Mục tiêu Agile:** Hoàn thiện các tính năng JavaScript và chuẩn bị cho buổi Demo (Hackathon).

1. Lý thuyết (30 phút): Agile Review & Refactoring

- **Sprint Review:** Tự đánh giá những gì đã làm được ở tuần 13.
- **Definition of Done (DoD):** Một tính năng được gọi là "Xong" khi nó: Chạy đúng logic, không lỗi Console, hiển thị tốt trên Mobile và Code sạch (đã qua AI Refactor).

2. Thực hành PBL - Giai đoạn "Thực thi & Tinh chỉnh"

- **Hoạt động 1: Daily Stand-up (Họp nhanh đầu giờ):**
    - Mỗi thành viên trả lời 3 câu hỏi: *Hôm qua làm gì? Hôm nay làm gì? Có gặp khó khăn gì không?*.
- **Hoạt động 2: JavaScript Vibe Logic:**
    - Tập trung vào tính năng "đắt giá": Đổ dữ liệu từ Object JSON, Form Validation, bộ lọc tìm kiếm.
    - Sử dụng AI để xử lý các đoạn mã phức tạp và tối ưu hóa hiệu năng.
- **Hoạt động 3: Đóng gói sản phẩm:**
    - Hoàn thiện file `README.md` chuyên nghiệp (có danh sách Prompt đã dùng).
    - Đẩy sản phẩm lên **GitHub Pages** để chạy Live.

🏆 TIÊU CHÍ "VIBE CODE" DÀNH CHO GIẢNG VIÊN (MENTOR)

Để sinh viên thực sự trải nghiệm Agile, thầy cần quan sát và chấm điểm dựa trên quy trình phối hợp:

1. **Tính liên tục (Git Flow):** Kiểm tra xem các em có commit code đều đặn không, hay đến phút cuối mới nộp.

2. **Sự linh hoạt (Prompting):** Cách sinh viên mô tả lỗi cho AI để tìm giải pháp (thay vì yêu cầu AI viết lại toàn bộ).
3. **Tư duy sản phẩm (PBL):** Sản phẩm có giải quyết được vấn đề thực tế của chủ đề đã chọn không?.

> **Checklist nộp bài trước Hackathon (Tuần 15):**

- [ ] Repository GitHub có đầy đủ các nhiệm vụ (Tasks) đã hoàn thành.
- [ ] File `prompt_log.md` ghi lại hành trình "đối thoại" với AI.
- [ ] Một bản Pitching ngắn gọn thể hiện được giá trị của dự án.

Với thiết kế này, sinh viên sẽ không cảm thấy mình đang "làm bài tập", mà đang thực sự **phát triển một dự án công nghệ** theo đúng tiêu chuẩn công nghiệp hiện đại.

Phương pháp giảng dạy Project base Learning

PHƯƠNG PHÁP GIẢNG DẠY

PROJECT BASE LEARNING

1. DÀNH CHO GIÁO VIÊN (VAI TRÒ MENTOR & PRODUCT OWNER)

Giáo viên trong PBL không còn là người "đọc - chép" mà là người định hướng, đặt ra các tiêu chuẩn và kiểm soát chất lượng đầu ra.

1.1. Quy trình chuẩn bị dự án

- **Xác định vấn đề (Driving Question):** Thay vì nói "Hãy làm một trang web bán hàng", hãy đặt vấn đề: *"Làm thế nào để xây dựng một trải nghiệm mua sắm trực tuyến mượt mà giúp các hộ kinh doanh tại địa phương tiếp cận khách hàng hiệu quả?"*
- **Thiết lập Rubric (Tiêu chí đánh giá):** Cần rõ ràng về trọng số cho: Kỹ thuật (HTML/CSS/JS), Quy trình (Git/Agile), và Tư duy (Vibe Coding/AI Usage).

1.2. Các giai đoạn hướng dẫn trong PBL

- Giai đoạn 1: Khám phá: Hướng dẫn sinh viên nghiên cứu thực tế, phân tích đối thủ và người dùng mục tiêu.
- Giai đoạn 2: Phản biện (Feedback & Revision): Tổ chức các buổi thuyết trình ngắn giữa kỳ để các nhóm nhận xét lẫn nhau.

1.3. Tích hợp Agile/Scrum

- Đóng vai là **Product Owner** để duyệt danh sách tính năng (Backlog) của sinh viên.
- Khuyến khích sinh viên sử dụng các công cụ quản lý dự án hiện đại như Trello hoặc GitHub Projects.

2. DÀNH CHO SINH VIÊN (VAI TRÒ WEB DEVELOPER & SCRUM TEAM)

Các bạn sinh viên sẽ đóng vai là một đội ngũ phát triển phần mềm, làm chủ dự án từ ý tưởng đến sản phẩm thực tế.

2.1. 7 Yếu tố cốt lõi của một dự án PBL thành công

- **Nội dung cốt lõi:** Dự án phải vận dụng được kiến thức JavaScript, DOM, và Form đã học.
- **Kỹ năng thế kỷ 21:** Làm việc nhóm, giao tiếp, và sử dụng AI thông minh.
- **Câu hỏi định hướng:** Luôn bám sát vấn đề dự án đang giải quyết.
- **Nhu cầu cần biết:** Tự tìm tòi kiến thức mới (ví dụ: thư viện AOS, Regex) để phục vụ tính năng dự án.
- **Quyền tự chủ:** Nhóm tự quyết định công nghệ và phong cách thiết kế.
- **Phê bình và sửa chữa:** Sẵn sàng thay đổi mã nguồn dựa trên góp ý của giảng viên.
- **Sản phẩm công khai:** Website phải được "Live" trên internet (Vercel, GitHub Pages).

2.2. Quy trình "Vibe Coding" trong Agile

- **Bước 1 - Phân tích:** Dùng AI thảo luận về cấu trúc dữ liệu JSON.
- **Bước 2 - Thực thi:** Viết code cốt lõi, phối hợp qua Git.

- **Bước 3 - Tinh chỉnh:** Dùng AI để tối ưu hóa (Refactor) code giúp mã nguồn sạch và chuyên nghiệp.

3. SỰ KẾT HỢP GIỮA PBL VÀ QUY TRÌNH AGILE (HÀNH ĐỘNG CỤ THỂ)

Giai đoạn PBL	Hoạt động Agile tương ứng	Kết quả (Deliverables)
Phát biểu vấn đề	**Sprint Planning**	Danh sách tính năng (Backlog) trên GitHub.
Thiết kế giải pháp	**Vibe Design**	Sơ đồ luồng (User Flow) và JSON Data mẫu.
Thực hiện dự án	**Sprint Execution**	Các file HTML/CSS/JS được commit đều đặn.
Đánh giá & Chỉnh sửa	**Sprint Review/Retrospective**	File prompt_log.md và README.md hoàn thiện.
Công bố sản phẩm	**Demo Day (Hackathon)**	Link Website hoạt động thực tế.

Agile & Scrum Framework

1. AGILE LÀ GÌ? (TƯ DUY LINH HOẠT)

Agile không phải là một phương pháp cụ thể, mà là một **triết lý** hoặc **tư duy** trong phát triển phần mềm. Thay vì lập kế hoạch cứng nhắc ngay từ đầu (như mô hình Thác nước - Waterfall), Agile tập trung vào việc:

- **Phát triển cuốn chiếu:** Chia nhỏ dự án thành nhiều giai đoạn.
- **Thích nghi nhanh:** Sẵn sàng thay đổi yêu cầu dựa trên phản hồi của khách hàng ở bất kỳ giai đoạn nào.
- **Giá trị thực:** Mục tiêu cuối cùng là tạo ra sản phẩm "chạy được" và mang lại giá trị thực tế sau mỗi chu kỳ ngắn.

2. SCRUM FRAMEWORK (KHUNG LÀM VIỆC NHÓM)

Scrum là khung làm việc phổ biến nhất để thực thi tư duy Agile. Nó chia nhóm làm việc thành các đơn vị nhỏ với vai trò và quy trình rõ ràng.

2.1. 3 Vai trò chính (Roles)

Trong một nhóm Scrum (Scrum Team), không có khái niệm cấp bậc "Sếp - Nhân viên", mà chỉ có sự phối hợp giữa:

- **Product Owner (PO - Chủ sản phẩm):** Người giữ tầm nhìn dự án, quyết định tính năng nào quan trọng nhất để làm trước.
- **Scrum Master (Người điều phối):** Không phải quản lý dự án, mà là người hỗ trợ nhóm hiểu đúng Scrum, loại bỏ các vật cản để nhóm làm việc hiệu quả.
- **Development Team (Nhóm phát triển):** Những người trực tiếp thiết kế, viết code và kiểm thử sản phẩm (Sinh viên đóng vai trò này).

2.2. 3 Tạo vật quan trọng (Artifacts)

Đây là các công cụ để quản lý công việc minh bạch:

- **Product Backlog:** Danh sách tổng thể tất cả các tính năng, yêu cầu cần có của trang web.
- **Sprint Backlog:** Danh sách các đầu việc cụ thể mà nhóm cam kết hoàn thành trong một Sprint (chu kỳ).
- **Increment (Phần tăng trưởng):** Sản phẩm hoàn thiện có thể chạy được sau mỗi Sprint (Ví dụ: Tuần 13 xong Header/Footer, Tuần 14 xong Giỏ hàng).

2.3. C4 Sự kiện chính (Events)

Scrum vận hành thông qua các chu kỳ gọi là **Sprint** (thường kéo dài 1-4 tuần):

- **Sprint Planning (Lập kế hoạch):** Nhóm họp để chọn việc từ Product Backlog đưa vào Sprint Backlog.
- **Daily Scrum (Họp đứng hàng ngày):** Cuộc họp 15 phút để mỗi người trả lời: *Hôm qua làm gì? Hôm nay làm gì? Có khó khăn gì không?*.
- **Sprint Review (Sơ kết):** Trình diễn những gì đã làm được cho PO (Giảng viên) xem.
- **Sprint Retrospective (Cải tiến):** Nhóm tự nhìn lại cách làm việc để cải thiện trong Sprint tiếp theo.

3. TẠI SAO PBL NÊN KẾT HỢP VỚI SCRUM?

Khi áp dụng phương pháp **PBL (Project-Based Learning)**, Scrum giúp sinh viên:

- **Tránh "nước đến chân mới nhảy":** Công việc được chia nhỏ và hoàn thành theo từng tuần.
- **Tăng tính trách nhiệm:** Ai cũng có nhiệm vụ rõ ràng trên bảng Kanban (To-do, Doing, Done).
- **Phối hợp tốt với AI (Vibe Coding):** AI giúp viết code nhanh (Development), nhưng Scrum giúp nhóm quản lý các đoạn code đó khớp với nhau thành một hệ thống hoàn chỉnh.

TÓM TẮT CHO GIẢNG VIÊN & SINH VIÊN

- **Giảng viên (PO):** Đừng cầm tay chỉ việc, hãy đưa ra yêu cầu về tính năng và nghiệm thu sản phẩm sau mỗi Sprint.
- **Sinh viên (Dev Team):** Chủ động chia việc, hỗ trợ nhau và sử dụng AI để giải quyết các phần code khó một cách linh hoạt.

Dự án mẫu

1. SPRINT 1 – THIẾT LẬP CẤU TRÚC & GIAO DIỆN CỐT LÕI

1.1. Vai trò trong nhóm (Scrum Roles):

- **Product Owner (Giảng viên):** Yêu cầu hệ thống phải có trang Dashboard xem điểm và trang Đăng ký môn học.
- **Scrum Master:** Theo dõi bảng Trello/GitHub Projects.
- **Dev Team:** Thiết kế giao diện và cấu trúc dữ liệu.

1.2. 2. Hoạt động Vibe Coding (Làm việc với AI):

- **Thiết kế dữ liệu:** Sinh viên dùng Prompt để AI tạo file data.js chứa danh sách môn học, điểm số mẫu dưới dạng JSON.
  - **Prompt**: "Hãy tạo một mảng JSON gồm 10 môn học với các thuộc tính: mã môn, tên môn, số tín chỉ, điểm chuyên cần, điểm thi và trạng thái (Đã đạt/Chưa đạt)."
- **Dựng khung UI:** Chia Layout bằng CSS Grid (Sidebar chứa Menu, Content chứa bảng điểm).

1.3. Kết quả Sprint 1: Giao diện tĩnh (HTML/CSS) hoàn thiện, cấu trúc dữ liệu JSON sẵn sàng.

2. SPRINT 2 – JAVASCRIPT LOGIC & TƯƠNG TÁC THÔNG MINH

2.1. Quy trình Agile (Daily Stand-up):

- "Hôm qua mình đã làm xong phần hiển thị danh sách môn học."
- "Hôm nay mình sẽ làm chức năng tính điểm trung bình (GPA) tự động bằng JS."

2.2. Hoạt động Vibe Coding (Làm việc với AI):

- Đổ dữ liệu động: Viết hàm dùng map() để render danh sách điểm từ file data.js lên bảng HTML.
- Logic tính toán: Viết hàm tinhGPA() nhận vào mảng điểm và trả về điểm trung bình hệ 4.
  - Prompt: "Viết hàm JavaScript tính điểm trung bình tích lũy dựa trên mảng môn học đã có, lưu ý quy đổi từ thang điểm 10 sang thang điểm 4."
- Bộ lọc (Filter): Thêm ô tìm kiếm môn học hoặc lọc môn học theo học kỳ.

2.3. Kết quả

Website đã có thể tính điểm tự động, tìm kiếm môn học và hiển thị thông báo "Cảnh báo học vụ" nếu điểm GPA < 2.0 (sử dụng cấu trúc If-Else).

3. HACKATHON – CÔNG BỐ & TRÌNH DIỄN (DEMO DAY)

3.1. Hoạt động chính:

- Pitching (5 phút): Nhóm trình bày: "Hệ thống của chúng em giúp sinh viên biết chính xác mình cần bao nhiêu điểm ở kỳ tới để không bị cảnh báo học vụ."
- Demo Live: Thực hiện thao tác "Đăng ký môn học" ngay trên web – khi nhấn nút "Đăng ký", môn học đó tự động chuyển vào danh sách "Môn học đã chọn" và cập nhật tổng số tín chỉ.

3.2. Tiêu chí đánh giá "Vibe Code":

- UI/UX: Giao diện có giống các trang Portal hiện đại không? (Màu sắc xanh - trắng LHU).

- Clean Code: Code JavaScript được chia thành các hàm xử lý riêng biệt hay viết lộn xộn?
- AI Collaboration: File prompt_log.md có cho thấy sinh viên biết dùng AI để giải quyết các phần khó (như thuật toán tính điểm) không?

BẢNG PHÂN CHIA CÔNG VIỆC VÍ DỤ (BACKLOG)

Công việc (Task)	Thành viên phụ trách	Công cụ AI hỗ trợ	Trạng thái
Thiết kế Header & Sidebar	Nguyễn Văn A	Tạo Layout CSS Flexbox	Done
Tạo file JSON dữ liệu điểm	Trần Thị B	Sinh dữ liệu mẫu thực tế	Done
Hàm JS tính GPA tổng kết	Lê Văn C	Thuật toán quy đổi điểm	Doing
Form liên hệ hỗ trợ học vụ	Phạm Văn D	Regex kiểm tra MSSV	To-do

Các bạn không chỉ đang làm một bài tập web, các bạn đang xây dựng một công cụ có thể giúp hàng ngàn sinh viên LHU quản lý tương lai của mình. Hãy để AI là cộng sự, và Agile là kim chỉ nam!

Phân tích chi tiết theo SCRUM

DỰ ÁN: LHU STUDENT PORTAL (HỆ THỐNG QUẢN LÝ HỌC VỤ)

Bài toán (Driving Question): Làm thế nào để xây dựng một giao diện web giúp sinh viên LHU có thể tự quản lý lộ trình học tập, xem điểm số và đăng ký môn học một cách trực quan, hiện đại?

1. SPRINT 1 – THIẾT LẬP CẤU TRÚC & GIAO DIỆN CỐT LÕI

1.1. Vai trò trong nhóm (Scrum Roles):

- Product Owner (Giảng viên): Yêu cầu hệ thống phải có trang Dashboard xem điểm và trang Đăng ký môn học.
- Scrum Master: Theo dõi bảng Trello/GitHub Projects.
- Dev Team: Thiết kế giao diện và cấu trúc dữ liệu.

1.2. Hoạt động Vibe Coding (Làm việc với AI):

- Thiết kế dữ liệu: Sinh viên dùng Prompt để AI tạo file data.js chứa danh sách môn học, điểm số mẫu dưới dạng JSON.
- Prompt: "Hãy tạo một mảng JSON gồm 10 môn học với các thuộc tính: mã môn, tên môn, số tín chỉ, điểm chuyên cần, điểm thi và trạng thái (Đã đạt/Chưa đạt)."
- Dựng khung UI: Chia Layout bằng CSS Grid (Sidebar chứa Menu, Content chứa bảng điểm).

1.3. Kết quả cuối tuần 13: Giao diện tĩnh (HTML/CSS) hoàn thiện, cấu trúc dữ liệu JSON sẵn sàng

2. SPRINT 2 – JAVASCRIPT LOGIC & TƯƠNG TÁC THÔNG MINH

2.1. Quy trình Agile (Daily Stand-up):

- "Hôm qua mình đã làm xong phần hiển thị danh sách môn học."
- "Hôm nay mình sẽ làm chức năng tính điểm trung bình (GPA) tự động bằng JS."

2.2. Hoạt động Vibe Coding (Làm việc với AI):

- Đổ dữ liệu động: Viết hàm dùng map() để render danh sách điểm từ file data.js lên bảng HTML.
- Logic tính toán: Viết hàm tinhGPA() nhận vào mảng điểm và trả về điểm trung bình hệ 4.
- Prompt: "Viết hàm JavaScript tính điểm trung bình tích lũy dựa trên mảng môn học đã có, lưu ý quy đổi từ thang điểm 10 sang thang điểm 4."
- Bộ lọc (Filter): Thêm ô tìm kiếm môn học hoặc lọc môn học theo học kỳ.

2.3. Kết quả cuối tuần 14: Website đã có thể tính điểm tự động, tìm kiếm môn học và hiển thị thông báo "Cảnh báo học vụ" nếu điểm GPA < 2.0 (sử dụng cấu trúc If-Else).

3. HACKATHON – CÔNG BỐ & TRÌNH DIỄN (DEMO DAY)

3.1. Hoạt động chính:

- Pitching (5 phút): Nhóm trình bày: "Hệ thống của chúng em giúp sinh viên biết chính xác mình cần bao nhiêu điểm ở kỳ tới để không bị cảnh báo học vụ."
- Demo Live: Thực hiện thao tác "Đăng ký môn học" ngay trên web – khi nhấn nút "Đăng ký", môn học đó tự động chuyển vào danh sách "Môn học đã chọn" và cập nhật tổng số tín chỉ.

3.2. Tiêu chí đánh giá "Vibe Code":

- UI/UX: Giao diện có giống các trang Portal hiện đại không? (Màu sắc xanh - trắng LHU).
- Clean Code: Code JavaScript được chia thành các hàm xử lý riêng biệt hay viết lộn xộn?
- AI Collaboration: File prompt_log.md có cho thấy sinh viên biết dùng AI để giải quyết các phần khó (như thuật toán tính điểm) không?

BẢNG PHÂN CHIA CÔNG VIỆC VÍ DỤ (BACKLOG)

Công việc (Task)	Thành viên phụ trách	Công cụ AI hỗ trợ	Trạng thái
Thiết kế Header & Sidebar	Nguyễn Văn A	Tạo Layout CSS Flexbox	Done
Tạo file JSON dữ liệu điểm	Trần Thị B	Sinh dữ liệu mẫu thực tế	Done
Hàm JS tính GPA tổng kết	Lê Văn C	Thuật toán quy đổi điểm	Doing
Form liên hệ hỗ trợ học vụ	Phạm Văn D	Regex kiểm tra MSSV	To-do

Lời nhắn cho sinh viên: "Các bạn không chỉ đang làm một bài tập web, các bạn đang xây dựng một công cụ có thể giúp hàng ngàn sinh viên LHU quản lý tương lai của mình. Hãy để AI là cộng sự, và Agile là kim chỉ nam!"

Phương phap phân chia vai trò

PHƯƠNG PHÁP PHÂN CHIA VAI TRÒ TRONG NHÓM SCRUM

Để sinh viên có thể hiểu được và biết cách chia việc theo phương pháp **Module-Based (Chia theo khối chức năng độc lập)** kết hợp với **Shared Data Contract (Hợp đồng dữ liệu chung)**.

Hãy thực hiện theo hướng dẫn dưới đây:

1. CHIẾN LƯỢC CHIA VIỆC ĐỂ TRÁNH CHỒNG CHÉO

1.1. Thống nhất "Hợp đồng dữ liệu" (Data Contract) - Quan trọng nhất

Trước khi bắt đầu "vibe", cả nhóm phải ngồi lại để AI tạo ra một file dữ liệu mẫu duy nhất (data.js). Tất cả các thành viên đều phải dùng chung file này.

- **Mục đích:** Đảm bảo khi bạn A làm chức năng "Tính GPA" và bạn B làm chức năng "Lọc môn học", cả hai đều gọi đúng tên thuộc tính (ví dụ: subjectName thay vì người dùng tenMon, người dùng name).

1.2. Chia theo lớp (Layers) và Module

Thay vì chia theo kiểu "Bạn A làm trang chủ, bạn B làm trang điểm" (dễ bị trùng CSS/Header), hãy chia theo **Tính năng độc lập trong cùng một hệ thống**:

Vai trò (Role)	Nhiệm vụ cụ thể (Module)	Phạm vi file (Scope)
UI/UX Designer (Dev 1)	Thiết kế Layout tổng thể, Sidebar, Navbar và các biến CSS dùng chung (màu sắc, font).	style.css, index.html
Data Logic (Dev 2)	Viết các hàm xử lý dữ liệu thuần túy (Tính GPA, xếp loại, tính tổng tín chỉ).	logic.js
Feature Master (Dev 3)	Xử lý các tương tác DOM (Bấm nút Đăng ký môn thì hiện vào danh sách, tìm kiếm môn học).	features.js

2. VÍ DỤ CỤ THỂ: DỰ ÁN QUẢN LÝ HỌC VỤ

Hãy giả sử nhóm có 3 sinh viên, Các em "vibe" như sau để không bao giờ đè code lên nhau:

2.1. Bước 1: Thiết lập cấu trúc "Xương cá" (Tuần 13 - Sprint 1)

- Nhóm trưởng yêu cầu AI tạo ra cấu trúc thư mục và các file trống:
- index.html: Chỉ chứa khung chính.
- styles.css: Chứa các biến CSS :root.
- data.js: Chứa mảng const STUDENTS_DATA = [...].

2.2. Bước 2: Phân chia Task theo "Luồng AI" (Tuần 14 - Sprint 2)

- **Sinh viên A (Làm Giao diện tĩnh):**
  - **Prompt:** "Dựa trên Layout chuẩn của LHU, hãy viết CSS cho Sidebar và bảng hiển thị điểm. Lưu ý sử dụng các biến màu sắc đã định nghĩa trong :root."
  - **Tránh chồng chéo:** Bạn này chỉ tập trung vào CSS, không viết logic JS.
- **Sinh viên B (Làm Logic tính toán):**
  - **Prompt: "**Tôi có mảng dữ liệu sinh viên trong data.js. Hãy viết các hàm JavaScript (Export) để tính điểm trung bình hệ 4 và đếm số môn chưa đạt. Yêu cầu không đụng vào DOM, chỉ trả về kết quả số."

    - **Trình chồng chéo:** Bạn này viết hàm "sạch" (Pure Functions), dễ dàng kiểm tra mà không làm hỏng giao diện.
- **Sinh viên C (Làm Tương tác - DOM):**
    - **Prompt**: "Hãy viết code lắng nghe sự kiện khi người dùng gõ vào ô tìm kiếm. Khi có kết quả, gọi hàm hiển thị danh sách từ bạn B để cập nhật lại bảng điểm."
    - **Tránh chồng chéo**: Bạn này đóng vai trò "người kết nối", chỉ gọi các hàm đã có sẵn.

3. QUY TẮC "VÀNG" ĐỂ GIÁO VIÊN HƯỚNG DẪN SINH VIÊN

Để các Role hoạt động tốt nhất mà AI không làm "loạn" code, thầy hãy đưa ra 3 quy tắc này:

- **Quy tắc "Hộp cát" (Sandboxing):** Mỗi sinh viên viết code vào một file JS riêng biệt. AI khi được yêu cầu hỗ trợ cũng chỉ được đọc file đó.
- **Quy tắc "Tiền tố" (Prefixing):** Khuyên sinh viên đặt tên Class CSS có tiền tố theo tên mình hoặc chức năng để tránh trùng tên (ví dụ: .log-gpa-button thay vì .button).
- **Quy tắc "Check-in" (Git Flow):** Trước khi gộp code vào main, nhóm trưởng phải dùng AI để "Review":
    - **Prompt cho Nhóm trưởng:** "Đây là code của bạn A và bạn B. Hãy kiểm tra xem có đoạn nào xung đột (conflict) về tên biến hay không trước khi tôi gộp chúng lại."

4. BẢNG PHÂN CHIA CÔNG VIỆC VÍ DỤ (BACKLOG)

Công việc (Task)	Thành viên phụ trách	Công cụ AI hỗ trợ	Trạng thái
Thiết kế Header & Sidebar	Nguyễn Văn A	Tạo Layout CSS Flexbox	Done
Tạo file JSON dữ liệu điểm	Trần Thị B	Sinh dữ liệu mẫu thực tế	Done
Hàm JS tính GPA tổng kết	Lê Văn C	Thuật toán quy đổi điểm	Doing
Form liên hệ hỗ trợ học vụ	Phạm Văn D	Regex kiểm tra MSSV	To-do

PHỤ LỤC

Phụ lục

Phụ lục 1 - Danh sách đề tài

PHỤ LỤC 1

DANH SÁCH CHỦ ĐỀ

1. THÔNG TIN CHUNG VỀ CHỦ ĐỀ

1.1. Yêu cầu chung

Mỗi nhóm (3-5 sinh viên) lựa chọn một vấn đề thực tế (tại nơi làm việc, tại trường ĐH Lạc Hồng, hoặc trong đời sống) và xây dựng một **Website** hoàn chỉnh để giải quyết vấn đề đó. Sinh viên được khuyến khích sử dụng AI tối đa trong tất cả các khâu từ lên ý tưởng, viết code đến kiểm thử.

1.2. Các nhóm đề tài gợi ý

Sinh viên có thể chọn 1 trong các hướng sau:

- **Hướng 1 (LHU Campus):** Xây dựng trang Landing Page giới thiệu ngành học, câu lạc bộ hoặc hệ thống tra cứu nhanh quy định học vụ cho sinh viên mới.
- **Hướng 2 (Kinh doanh):** Xây dựng trang web giới thiệu sản phẩm/dịch vụ cho một cửa hàng thực tế (Ví dụ: Web đặt lịch tiệm tóc, Web giới thiệu thực đơn quán cafe, Web bán đặc sản Đồng Nai).
- **Hướng 3 (Hỗ trợ công việc):** Xây dựng công cụ web đơn giản phục vụ công việc hiện tại của các thành viên trong nhóm (Ví dụ: Trang quản lý danh sách thiết bị, trang báo cáo tiến độ công việc nội bộ).

1.3. Yêu cầu kỹ thuật (Sản phẩm cuối cùng)

- Giao diện (Frontend): Sử dụng HTML5, CSS3.. Giao diện phải Responsive (chạy tốt trên cả máy tính và điện thoại).
- Tương tác (Javascript): Có các tính năng tương tác như Slide ảnh, kiểm tra Form, tính toán dữ liệu hoặc lấy dữ liệu từ một API mở.
- AI Integration: Thể hiện rõ việc dùng AI để sinh mã nguồn, sửa lỗi hoặc tối ưu hóa nội dung/hình ảnh trên trang web.

1.4. Các trang tối thiểu cần có trong dự án

Mỗi dự án dù chọn phân hệ nào cũng phải đảm bảo ít nhất 03 trang sau để đạt yêu cầu về cấu trúc điều hướng và quản lý dữ liệu:

1.4.1. Trang Chủ (Dashboard/Landing Page)

- Mục tiêu: Hiển thị cái nhìn tổng quan và các phím tắt điều hướng nhanh.
- Yêu cầu kỹ thuật: Sử dụng CSS Grid/Flexbox để bố cục các thẻ (Cards) tóm tắt thông tin quan trọng (ví dụ: GPA hiện tại, môn học sắp tới, hoặc deadline gần nhất).
- Javascript: Tự động đổ dữ liệu tổng quát từ file JSON lên các widget.

1.4.2. Trang Chi Tiết Chức năng (Functional Page)

- Mục tiêu: Đây là trang trọng tâm để sinh viên thực hiện các yêu cầu đặc thù của từng chủ đề.
- Ví dụ:
  - Phân hệ Xem điểm: Trang danh sách điểm chi tiết tất cả các học kỳ.
  - Phân hệ Đăng ký: Trang danh sách môn học kèm chức năng lọc và chọn môn.

- Phân hệ Tài chính: Trang kê khai chi tiết các khoản phí và lịch sử đóng tiền.
- Javascript: Thực hiện các bộ lọc (Filter), tìm kiếm (Search) và tính toán dữ liệu trực tiếp trên trang này.

1.4.3. Trang Quản lý Hồ sơ/Cấu hình (Profile & Settings Page)

- Mục tiêu: Quản lý thông tin cá nhân và thiết lập ứng dụng.
- Yêu cầu kỹ thuật: Sử dụng HTML5 Form với đầy đủ các loại Input để sinh viên thực hành kiểm soát dữ liệu (Validation).
- Javascript: Xử lý sự kiện chỉnh sửa thông tin, kiểm tra định dạng email/số điện thoại bằng Regex và cập nhật hiển thị ngay lập tức khi người dùng nhấn "Lưu".

1.5. Hồ sơ nộp bài (Checklist)

Ngoài mã nguồn website (Trên github.com), mỗi nhóm phải nộp một **"Nhật ký đồng hành cùng AI"** (File PDF) bao gồm:

- **Bảng danh sách Prompt:** Tổng hợp ít nhất 5-10 câu lệnh quan trọng nhất mà nhóm đã dùng để điều khiển AI tạo ra Website.
- **Minh chứng sửa lỗi:** Ảnh chụp màn hình một đoạn code bị lỗi và cách AI đã hướng dẫn sửa lỗi đó.
- **Video Demo (3-5 phút):** Quay màn hình giới thiệu các tính năng chính của Website.

Chú ý: Tất cả các file nộp được tổ chức theo cấu trúc kho lưu trữ trong phụ lục 3

2. DANH SÁCH CÁC CHỦ ĐỀ

2.1. DANH SÁCH 5 CHỦ ĐỀ: HỆ THỐNG QUẢN LÝ HỌC VỤ (STUDENT PORTAL)

2.1.1. Phân hệ: Dashboard Cá nhân & Lộ trình Học tập (The Learning Path)

- Tầm nhìn dài hạn: Trở thành hệ thống AI gợi ý lộ trình học tập và dự báo kết quả dựa trên dữ liệu lịch sử.
- Yêu cầu cụ thể cho học phần này:
  - **HTML5**: Sử dụng các thẻ Semantic (<section>, <article>, <aside>) để bố cục trang Dashboard.
  - **CSS3**: Thiết kế các Progress Bar (thanh tiến độ) và biểu đồ cột bằng CSS thuần để thể hiện phần trăm hoàn thành chương trình.
  - **JavaScript**: Viết các hàm xử lý mảng dữ liệu điểm (JSON) để tự động tính điểm GPA hệ 4, xếp loại học lực và thay đổi màu sắc trạng thái (Ví dụ: GPA < 2.0 hiển thị màu đỏ cảnh báo).

2.1.2. Phân hệ: Cổng Đăng ký Môn học Thông minh (Smart Enrollment)

- **Tầm nhìn dài hạn:** Tích hợp thanh toán trực tuyến, kiểm tra điều kiện môn học tiên quyết và tối ưu hóa thời khóa biểu tự động.
- **Yêu cầu cụ thể cho học phần này:**
  - **HTML5**: Xây dựng Form tìm kiếm và danh sách sản phẩm (môn học) sử dụng các thuộc tính data-* để lưu trữ thông tin tín chỉ.
  - **CSS3**: Tạo hiệu ứng Hover cho các thẻ môn học, thiết kế bảng danh sách môn học đã chọn chuyên nghiệp và Responsive.
  - **JavaScript:** Xử lý logic "Giỏ hàng môn học": Thêm/Xóa môn học khỏi danh sách chọn, tính tổng số tín chỉ theo thời gian thực và ngăn chặn nếu tổng tín chỉ vượt quá 25.

2.1.3. Phân hệ: Quản lý Thời khóa biểu & Chuyên cần (Attendance Tracker)

- **Tầm nhìn dài hạn:** Tích hợp định vị GPS/QR Code để điểm danh tự động và kết nối với thông báo đẩy trên điện thoại.

- **Yêu cầu cụ thể cho học phần này:**
  - HTML5: Sử dụng thẻ <table> hoặc CSS Grid để dựng lịch học theo tuần.
  - CSS3: Sử dụng Flexbox để căn chỉnh các ca học. Áp dụng hiệu ứng chuyển động khi người dùng chuyển đổi giữa các tuần/tháng.
  - JavaScript: Viết hàm lắng nghe sự kiện click để "đánh dấu" buổi học. Tính toán tỷ lệ phần trăm đi học và hiện thông báo nhắc nhở nếu sinh viên nghỉ quá 20% số buổi.

2.1.4. Phân hệ: Quản lý Đồ án & Tiến độ Báo cáo (Project Milestone)

- Tầm nhìn dài hạn: Hệ thống quản lý phiên bản đồ án, kết nối giảng viên - sinh viên để chấm điểm và nhận xét trực tuyến.
- **Yêu cầu cụ thể cho học phần này:**
  - HTML5: Xây dựng cấu trúc các thẻ lồng nhau để tạo dòng thời gian (Timeline) các mốc deadline báo cáo.
  - CSS3: Sử dụng CSS Pseudo-elements (::before, ::after) để vẽ đường nối timeline. Thiết kế các Badge (nhãn) trạng thái: "Đã nộp", "Chưa nộp", "Trễ hạn".
  - JavaScript: Xử lý sự kiện nộp link đồ án thông qua Form, kiểm tra định dạng URL hợp lệ và hiển thị đồng hồ đếm ngược (Countdown) đến ngày báo cáo.

2.1.5. Phân hệ: Tra cứu Học phí & Học bổng (Financial Hub)

- Tầm nhìn dài hạn: Hệ thống quản lý tài chính sinh viên toàn diện, tích hợp ví điện tử và xét duyệt hồ sơ học bổng tự động.
- Yêu cầu cụ thể cho học phần này:
  - HTML5: Sử dụng các kiểu dữ liệu Input phù hợp (number, date) để giả lập bảng kê khai học phí.
  - CSS3: Thiết kế giao diện dạng các thẻ (Cards) hiển thị mức học bổng đạt được dựa trên các điều kiện điểm số.
  - JavaScript: Viết logic tính toán học phí còn nợ sau khi trừ đi học bổng. Sử dụng sự kiện change trên các checkbox môn học để cập nhật tổng số tiền cần đóng ngay lập tức.

2.2. DANH SÁCH 5 CHỦ ĐỀ: THƯƠNG MẠI ĐIỆN TỬ (E-COMMERCE)

2.2.1. 1. Phân hệ: Hệ thống Mua sắm & Giỏ hàng thông minh (Smart Shopping Cart)

- Tầm nhìn dài hạn: Tích hợp thanh toán đa kênh, gợi ý sản phẩm dựa trên AI và quản lý kho hàng thời gian thực.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Sử dụng Grid/Flexbox để dựng lưới sản phẩm (Product Grid). Áp dụng hiệu ứng Hover và Badge (nhãn) "Hot/New/Sale" cho từng sản phẩm.
  - JavaScript: Xử lý logic thêm sản phẩm vào giỏ hàng mà không load lại trang. Tính toán tổng tiền, thuế, và phí vận chuyển tự động trong giỏ hàng khi người dùng thay đổi số lượng.
- Các trang tối thiểu:
  - Trang chủ: Hiển thị danh mục và các sản phẩm nổi bật.
  - Trang cửa hàng: Danh sách tất cả sản phẩm kèm bộ lọc giá và loại hàng.
  - Trang thanh toán/Giỏ hàng: Hiển thị chi tiết đơn hàng và Form nhập thông tin nhận hàng.

2.2.2. *2. Phân hệ: Hệ thống Đánh giá & Phản hồi khách hàng (Review & Feedback System)*

- Tầm nhìn dài hạn: Tích hợp nhận diện cảm xúc qua văn bản, quản lý hình ảnh/video đánh giá và hệ thống tích điểm cho người dùng đóng góp.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế giao diện xếp hạng sao (Star Rating) bằng CSS thuần. Sử dụng Progress Bar để biểu diễn tỷ lệ % đánh giá tích cực/tiêu cực.
  - JavaScript: Cho phép người dùng chọn số sao và nhập bình luận. Khi nhấn "Gửi", đánh giá mới sẽ xuất hiện ngay lập tức trong danh sách mà không cần tải lại trang.
- Các trang tối thiểu:
  - Trang chủ: Các sản phẩm có đánh giá cao nhất.
  - Trang chi tiết sản phẩm: Hiển thị mô tả và toàn bộ danh sách đánh giá của khách hàng.
  - Trang hồ sơ cá nhân: Lịch sử các đánh giá mà sinh viên đó đã thực hiện.

2.2.3. *Phân hệ: Hệ thống Quản lý Khuyến mãi & Mã giảm giá (Discount & Voucher Hub)*

- Tầm nhìn dài hạn: Hệ thống cá nhân hóa ưu đãi theo hành vi mua sắm, tự động gửi mã giảm giá qua Email/SMS và tích hợp ví voucher.
- Yêu cầu cụ thể cho học phần này:
  - **HTML5 & CSS3**: Thiết kế giao diện Voucher dạng thẻ (Cards) có đường xé răng cưa bằng CSS. Sử dụng hiệu ứng Animation cho các banner khuyến mãi chạy ngang.
  - **JavaScript**: Xử lý sự kiện nhập mã giảm giá. Viết logic kiểm tra mã hợp lệ và trừ tiền trực tiếp vào hóa đơn nếu mã đúng (ví dụ: mã "LHU2026" giảm 10%).
- **Các trang tối thiểu:**
  - Trang chủ: Hiển thị các Banner và Voucher đang hot.
  - Trang kho Voucher: Danh sách tất cả các mã đang có, phân loại theo ngành hàng.
  - Trang ví cá nhân: Lưu trữ các mã mà người dùng đã thu thập.

2.2.4. *Phân hệ: Hệ thống Theo dõi Đơn hàng & Vận chuyển (Order Tracking)*

- Tầm nhìn dài hạn: Tích hợp bản đồ thời gian thực (Google Maps API), thông báo trạng thái đơn hàng tự động và quản lý trả hàng/hoàn tiền.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Sử dụng CSS Pseudo-elements để vẽ sơ đồ Timeline trạng thái đơn hàng (Đã đặt -> Đã xác nhận -> Đang giao -> Thành công).
  - JavaScript: Xử lý sự kiện tra cứu đơn hàng bằng Mã đơn (Order ID). Hiển thị thông báo đếm ngược thời gian dự kiến nhận hàng.
- Các trang tối thiểu:
  - Trang chủ: Ô tìm kiếm đơn hàng nhanh.
  - Trang chi tiết vận chuyển: Hiển thị timeline và thông tin shipper.
  - Trang lịch sử mua hàng: Danh sách toàn bộ các đơn hàng đã thực hiện.

2.2.5. 5. Phân hệ: Hệ thống Đấu giá sản phẩm trực tuyến (Online Auction)

- Tầm nhìn dài hạn: Tích hợp WebSockets để cập nhật giá thời gian thực, hệ thống đấu giá tự động (Auto-bid) và thanh toán ký quỹ bảo mật.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế bảng đấu giá chuyên nghiệp, làm nổi bật số tiền hiện tại và người đang dẫn đầu.
  - JavaScript: Sử dụng setInterval để tạo đồng hồ đếm ngược thời gian kết thúc phiên đấu giá. Xử lý logic kiểm tra: giá người dùng nhập vào phải lớn hơn giá hiện tại mới được chấp nhận.
- Các trang tối thiểu:
  - Trang chủ: Danh sách các phiên đấu giá sắp kết thúc.
  - Trang đấu giá chi tiết: Nơi người dùng thực hiện nhập giá và xem lịch sử các lượt đấu giá.
  - Trang quản lý phiên: Danh sách các món hàng mà sinh viên đang tham gia đấu giá.

2.3. DANH SÁCH 5 CHỦ ĐỀ: QUẢN TRỊ DOANH NGHIỆP (ERP)

2.3.1. 1. Phân hệ: Quản lý Nhân sự & Tiền lương (HRM & Payroll)

- Tầm nhìn dài hạn: Tích hợp AI để đánh giá KPI, tự động hóa quy trình tuyển dụng và kết nối hệ thống bảo hiểm xã hội.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế bảng lương chi tiết và thẻ hồ sơ nhân viên (Employee Cards). Sử dụng Flexbox để tạo danh mục phòng ban.
  - JavaScript: Viết hàm tính lương dựa trên số ngày công, phụ cấp và thuế từ dữ liệu JSON. Tự động cập nhật bảng lương khi thay đổi số ngày nghỉ.
- Các trang tối thiểu:
  - Trang chủ (Dashboard): Thống kê tổng số nhân sự và biến động trong tháng.
  - Trang Danh sách nhân viên: Tìm kiếm và lọc nhân viên theo phòng ban/vị trí.
  - Trang Chi tiết lương: Hiển thị thu nhập và Form cập nhật ngày công.

2.3.2. 2. Phân hệ: Quản lý Kho & Kiểm kê vật tư (Inventory Management)

- Tầm nhìn dài hạn: Kết nối IoT (quét mã vạch), dự báo nhu cầu nhập hàng bằng Big Data và quản lý kho đa điểm.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Sử dụng các thuộc tính min, max cho input số lượng. Thiết kế nhãn trạng thái tồn kho (Hết hàng - Đỏ, Sắp hết - Vàng, Còn hàng - Xanh).
  - JavaScript: Xử lý logic nhập/xuất kho. Viết hàm kiểm tra nếu số lượng xuất vượt quá tồn kho thì hiện cảnh báo và chặn thao tác.
- Các trang tối thiểu:
  - Trang chủ: Danh sách mặt hàng sắp hết hoặc dưới định mức tối thiểu.
  - Trang Quản lý vật tư: Danh mục hàng hóa kèm chức năng tìm kiếm và cập nhật.
  - Trang Lịch sử: Dòng thời gian (Timeline) các lần nhập/xuất hàng.

2.3.3. *3. Phân hệ: Quản lý Quan hệ Khách hàng (CRM)*

- Tầm nhìn dài hạn: Tự động hóa chiến dịch Marketing, tích hợp Chatbot và phân tích giá trị vòng đời khách hàng (CLV).
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế Form ghi nhận thông tin khách hàng tiềm năng (Leads). Sử dụng CSS Transition cho các bảng khảo sát khách hàng.
  - JavaScript: Xử lý logic phân loại khách hàng (Đồng, Bạc, Vàng) tự động dựa trên tổng giá trị giao dịch trong dữ liệu mẫu.
- Các trang tối thiểu:
  - Trang chủ: Danh sách khách hàng cần liên hệ hoặc chăm sóc trong ngày.
  - Trang Danh bạ khách hàng: Hiển thị thông tin liên hệ và lịch sử tương tác.
  - Trang Form chăm sóc: Nhập nội dung phản hồi và ghi chú yêu cầu khách hàng.

2.3.4. *4. Phân hệ: Quản lý Dự án & Công việc (Project Task Management)*

- Tầm nhìn dài hạn: Tích hợp biểu đồ Gantt tương tác, quản lý tài nguyên (Resource allocation) và tự động nhắc việc qua ứng dụng di động.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Xây dựng bảng Kanban (To-do, Doing, Done) bằng CSS Grid. Thiết kế thẻ công việc kèm ảnh đại diện thành viên.
  - JavaScript: Xử lý kéo thả đơn giản (hoặc bấm nút chuyển trạng thái) để cập nhật tiến độ công việc. Tính % hoàn thành dự án dựa trên số task đã xong.
- Các trang tối thiểu:
  - Trang chủ: Tổng quan các dự án đang tham gia và tiến độ chung.
  - Trang Bảng công việc: Giao diện Kanban hoặc danh sách task chi tiết.
  - Trang Báo cáo tiến độ: Form nộp kết quả công việc và ghi chú thời gian làm (Time-tracking).

2.3.5. *5. Phân hệ: Quản lý Tài chính & Thu chi nội bộ (Internal Finance)*

- Tầm nhìn dài hạn: Kết nối ngân hàng (E-banking), tự động hóa báo cáo thuế và quản lý dòng tiền (Cash flow) thông minh.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế các biểu đồ tròn đơn giản bằng CSS (hoặc thẻ SVG) để thể hiện tỷ lệ thu chi. Sử dụng bảng số liệu có phân loại màu sắc.
  - JavaScript: Viết hàm tính tổng thu, tổng chi và số dư hiện tại. Xử lý logic lọc giao dịch theo ngày tháng hoặc theo loại chi phí.
- Các trang tối thiểu:
  - Trang chủ: Widget hiển thị số dư tức thời và các khoản chi lớn nhất.
  - Trang Nhật ký giao dịch: Danh sách chi tiết các khoản thu/chi kèm bộ lọc.
  - Trang Phiếu thu/chi: Form nhập dữ liệu mới với kiểm tra tính hợp lệ của số tiền.

2.4. DANH SÁCH 5 CHỦ ĐỀ: CÁ NHÂN HÓA ĐÀO TẠO (EDTECH)

2.4.1. *1. Phân hệ: Hệ thống Gợi ý Lộ trình học tập (Adaptive Learning Path)*

- Tầm nhìn dài hạn: Sử dụng AI để phân tích điểm mạnh/yếu của từng người học, từ đó tự động điều chỉnh độ khó và gợi ý bài học tiếp theo phù hợp nhất.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế sơ đồ lộ trình (Learning Roadmap) dưới dạng Timeline hoặc Mindmap bằng CSS Grid. Sử dụng các trạng thái màu sắc cho các bài học: "Đã xong", "Đang học", "Khóa".
  - JavaScript: Viết hàm xử lý logic "Mở khóa": bài học B chỉ hiện lên khi bài học A đạt trên 8.0 điểm. Tính toán phần trăm hoàn thành lộ trình dựa trên số bài đã đạt.
- Các trang tối thiểu:
  - Trang chủ (Dashboard): Hiển thị tổng quan lộ trình và bài học hiện tại.
  - Trang Lộ trình chi tiết: Bản đồ các bài học kèm điều kiện tiên quyết.
  - Trang Nội dung bài học: Hiển thị tài liệu và nút xác nhận hoàn thành.

2.4.2. *2. Phân hệ: Hệ thống Quản lý Mục tiêu & Thói quen học tập (Study Habit Tracker)*

- Tầm nhìn dài hạn: Tích hợp tính năng nhắc nhở thông minh qua di động, phân tích thời gian tập trung (Pomodoro) và thưởng (Gamification) để duy trì động lực.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế bảng theo dõi thói quen theo dạng lịch (Calendar Heatmap) giống GitHub. Sử dụng CSS Animation cho các huy hiệu (Badges) khi đạt mục tiêu.
  - JavaScript: Sử dụng setInterval để tạo đồng hồ đếm ngược thời gian học. Viết hàm lưu trữ "Chuỗi ngày học" (Streak) và hiển thị thông báo khích lệ.
- Các trang tối thiểu:
  - Trang chủ: Widget hiển thị Streak hiện tại và mục tiêu trong ngày.
  - Trang Thống kê: Biểu đồ thời gian học trong tuần (vẽ bằng CSS hoặc SVG).
  - Trang Cấu hình mục tiêu: Form cho phép người dùng tự đặt mục tiêu giờ học mỗi ngày.

2.4.3. *3. Phân hệ: Hệ thống Kho lưu trữ Tài liệu cá nhân (Personal Knowledge Base)*

- Tầm nhìn dài hạn: Xây dựng "Bộ não thứ hai" (Second Brain) với tính năng tự động tóm tắt tài liệu bằng AI và kết nối các ý tưởng bằng sơ đồ liên kết.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế giao diện quản lý file dạng thẻ (Card view) và danh sách (List view). Sử dụng thẻ <input type="search"> để tạo bộ lọc tìm kiếm nhanh.
  - JavaScript: Xử lý logic phân loại tài liệu theo Tag (nhãn). Viết hàm lọc tài liệu theo từ khóa hoặc định dạng file (PDF, Video, Link) mà không load lại trang.
- Các trang tối thiểu:
  - Trang chủ: Các tài liệu được xem nhiều nhất hoặc vừa mới thêm.

- Trang Kho tài liệu: Danh sách toàn bộ tài liệu kèm bộ lọc đa năng.
- Trang Chi tiết tài liệu: Nơi hiển thị nội dung và cho phép người dùng ghi chú (Note) nhanh.

2.4.4. *4. Phân hệ: Hệ thống Luyện tập & Kiểm tra thích ứng (Adaptive Quiz System)*

- Tầm nhìn dài hạn: Ngân hàng câu hỏi tự động sinh dựa trên trình độ người học, tích hợp chống gian lận và phân tích chi tiết lỗi sai để gợi ý kiến thức cần bù đắp.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế giao diện làm bài trắc nghiệm chuyên nghiệp với thanh tiến độ (Progress bar) và đồng hồ đếm ngược.
  - JavaScript: Xử lý logic chấm điểm ngay sau khi nộp bài. Hiển thị danh sách các câu sai kèm giải thích lấy từ dữ liệu JSON.
- Các trang tối thiểu:
  - Trang Danh sách đề thi: Phân loại theo độ khó hoặc chủ đề.
  - Trang Làm bài: Giao diện câu hỏi tương tác (Radio, Checkbox).
  - Trang Kết quả: Thống kê điểm số và phân tích các kỹ năng còn yếu.

2.4.5. *5. Phân hệ: Cổng kết nối Cố vấn & Học nhóm (Mentor-Mentee Connector)*

- Tầm nhìn dài hạn: Thuật toán ghép cặp (Matching) giữa người dạy và người học dựa trên kỹ năng và lịch rảnh, tích hợp phòng học trực tuyến và đánh giá 360 độ.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế danh sách Profile chuyên gia/Cố vấn với các nhãn kỹ năng (Skills Tags). Sử dụng Flexbox để tạo lịch rảnh (Availability slots).
  - JavaScript: Xử lý sự kiện "Đặt lịch hẹn": người dùng chọn giờ rảnh, hệ thống kiểm tra và thông báo đặt chỗ thành công (ghi nhận vào dữ liệu mẫu).
- Các trang tối thiểu:
  - Trang chủ: Tìm kiếm Cố vấn theo kỹ năng hoặc từ khóa.
  - Trang Hồ sơ Cố vấn: Thông tin chi tiết, đánh giá và lịch rảnh.
  - Trang Quản lý lịch hẹn: Danh sách các buổi gặp sắp tới của người học.

2.5. DANH SÁCH 5 CHỦ ĐỀ: PHÁT TRIỂN BỀN VỮNG

2.5.1. *1. Phân hệ: Hệ thống Theo dõi Dấu chân Carbon cá nhân (Carbon Footprint Tracker)*

- Tầm nhìn dài hạn: Sử dụng AI để phân tích thói quen tiêu dùng từ hóa đơn, tích hợp cảm biến IoT để đo lượng điện năng tiêu thụ và gợi ý các lộ trình giảm thiểu khí thải cá nhân hóa.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế giao diện Dashboard với các biểu đồ cột hoặc hình tròn (dùng CSS/SVG) để thể hiện tỷ lệ phát thải từ: Di chuyển, Ăn uống, Điện nước.
  - JavaScript: Viết hàm tính toán lượng khí thải dựa trên dữ liệu người dùng nhập vào (ví dụ: số km đi xe máy, số kWh điện). Tự động thay đổi màu sắc giao diện (Xanh/Vàng/Đỏ) tùy theo mức độ phát thải.
- Các trang tối thiểu:
  - Trang chủ (Overview): Hiển thị tổng lượng phát thải trong tháng và mục tiêu cắt giảm.

- Trang Nhật ký tiêu dùng: Form nhập liệu các hoạt động hàng ngày kèm chức năng lọc theo loại hoạt động.
- Trang Gợi ý sống xanh: Danh sách các mẹo giảm thiểu carbon dựa trên chỉ số cao nhất của người dùng.

2.5.2. *2. Phân hệ: Sàn trao đổi Đồ cũ & Tái chế (Circular Economy Exchange)*

- Tầm nhìn dài hạn: Tích hợp công nghệ Blockchain để truy xuất nguồn gốc sản phẩm, hệ thống định giá đồ cũ bằng AI và kết nối với các đơn vị vận chuyển xanh (xe điện).
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế lưới sản phẩm (Product Grid) chuyên nghiệp. Sử dụng nhãn (Badges) để phân loại tình trạng đồ cũ (Like new, Good, Fair).
  - JavaScript: Xử lý logic tìm kiếm và bộ lọc sản phẩm theo danh mục hoặc khoảng cách. Viết hàm tính toán "Lượng rác thải đã giảm thiểu" khi người dùng thực hiện một giao dịch thành công.
- Các trang tối thiểu:
  - Trang chủ: Hiển thị các mặt hàng mới đăng và thống kê tổng lượng rác thải cộng đồng đã giảm thiểu.
  - Trang Danh mục sản phẩm: Giao diện lọc và tìm kiếm đồ cũ theo nhiều tiêu chí.
  - Trang Đăng tin: Form cho phép người dùng nhập mô tả, tải ảnh (giả lập) và chọn loại hình trao đổi (Tặng/Bán/Đổi).

2.5.3. *3. Phân hệ: Hệ thống Quản lý & Tiết kiệm Năng lượng (Smart Energy Monitor)*

- Tầm nhìn dài hạn: Kết nối trực tiếp với công tơ điện thông minh, tự động điều khiển thiết bị điện để tối ưu hóa chi phí và tích hợp hệ thống điểm thưởng khi tiết kiệm năng lượng.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Sử dụng CSS Grid để dựng sơ đồ các phòng trong nhà. Thiết kế các nút gạt (Toggle Switch) trực quan để giả lập trạng thái bật/tắt thiết bị.
  - JavaScript: Sử dụng setInterval để giả lập dữ liệu điện năng tiêu thụ thay đổi theo thời gian thực. Viết logic tính toán tiền điện dự kiến cuối tháng dựa trên biểu giá bậc thang.
- Các trang tối thiểu:
  - Trang chủ: Sơ đồ trạng thái các thiết bị điện trong nhà.
  - Trang Thống kê chi tiết: Biểu đồ tiêu thụ điện theo giờ/ngày/tuần.
  - Trang Thiết lập định mức: Form đặt ngưỡng chi phí tối đa, hệ thống sẽ cảnh báo khi vượt ngưỡng.

2.5.4. *4. Phân hệ: Bản đồ Cứu trợ & Kết nối Cộng đồng (Community Rescue Map)*

- Tầm nhìn dài hạn: Tích hợp bản đồ thời gian thực để điều phối cứu trợ trong thiên tai, hệ thống xác minh thông tin bằng cộng đồng và kết nối tự động với các cơ quan chức năng.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế giao diện bản đồ (dùng ảnh nền hoặc khung chứa) với các điểm đánh dấu (Markers) có màu sắc khác nhau theo loại yêu cầu (Cần thực phẩm, Cần y tế, Cần nơi ở).

- JavaScript: Xử lý logic lọc các yêu cầu cứu trợ theo trạng thái "Đang chờ" hoặc "Đã được giúp". Viết hàm xử lý Form gửi yêu cầu cứu trợ với tính năng kiểm tra dữ liệu bắt buộc.

- Các trang tối thiểu:
  - Trang chủ (Bản đồ): Hiển thị các điểm nóng cần hỗ trợ xung quanh vị trí người dùng.
  - Trang Danh sách yêu cầu: Danh sách chi tiết các thông tin cứu trợ kèm số điện thoại liên hệ.
  - Trang Đăng ký tình nguyện viên: Form ghi nhận thông tin và khả năng hỗ trợ của cá nhân/tổ chức.

2.5.5. *5. Phân hệ: Hệ thống Quản lý Vườn phố & Nông nghiệp Đô thị (Urban Gardening)*

- Tầm nhìn dài hạn: Tích hợp cảm biến độ ẩm đất, ánh sáng để tự động hóa việc chăm sóc, hệ thống nhận diện sâu bệnh qua hình ảnh và sàn giao dịch nông sản sạch tại chỗ.
- Yêu cầu cụ thể cho học phần này:
  - HTML5 & CSS3: Thiết kế các "Thẻ cây trồng" (Plant Cards) hiển thị thông số môi trường lý tưởng. Sử dụng Progress Bar để thể hiện độ ẩm hoặc sức khỏe của cây.
  - JavaScript: Viết hàm nhắc lịch chăm sóc (tưới nước, bón phân) dựa trên loại cây. Xử lý sự kiện khi người dùng nhấn "Đã chăm sóc" để cập nhật lại thời gian trong dữ liệu JSON.
- Các trang tối thiểu:
  - Trang chủ (Khu vườn của tôi): Danh sách các loại cây đang trồng và trạng thái hiện tại.
  - Trang Thư viện cây trồng: Tra cứu kỹ thuật canh tác cho các loại rau/củ/quả đô thị.
  - Trang Nhật ký tăng trưởng: Nơi lưu trữ hình ảnh và ghi chú quá trình phát triển của cây theo thời gian.

Phụ lục 2 - Nhật ký đồng hành cùng AI

PHỤ LỤC 2

NHẬT KÝ ĐỒNG HÀNH CÙNG AI (PROMPT LOG)

Học phần: Thiết kế Web – Trường Đại học Lạc Hồng (LHU)

Tên nhóm: ..... **Lớp:** .....

Tên dự án: .....

1. Thông tin công cụ AI sử dụng

- **Công cụ chính:** (Ví dụ: ChatGPT-4o, Claude 3.5 Sonnet, Gemini...)
- **Công cụ hỗ trợ code (Extension):** (Ví dụ: GitHub Copilot, Codeium, Cursor...)
- **Công cụ tạo hình ảnh/nội dung:** (Ví dụ: Midjourney, Canva AI...)

2. Nhật ký quá trình thực hiện (Các Prompt quan trọng)

Giai đoạn	Mục tiêu (Bạn muốn AI làm gì?)	Nội dung Prompt (Câu lệnh bạn đã dùng)	Kết quả & Tùy chỉnh (AI có làm đúng không? Bạn có sửa lại gì không?)
Khởi tạo	Tạo khung HTML cho trang chủ.	*"Bạn là chuyên gia Front-end, hãy tạo file index.html chuẩn Semantic cho web bán Cafe, có Header, Banner, và danh sách 6 sản phẩm..."*	AI làm đúng 90%, em phải sửa lại link ảnh và thêm logo của LHU vào.
Giao diện	Chia cột cho danh sách sản phẩm.	*"Hãy viết CSS Flexbox để hiển thị 3 sản phẩm trên 1 hàng ở máy tính và 1 sản phẩm trên 1 hàng ở điện thoại."*	Kết quả tốt, giao diện đã tự co giãn được trên mobile.
Tính năng	Tạo hiệu ứng giỏ hàng đơn giản.	*"Viết Javascript để khi bấm vào nút 'Mua ngay', số lượng trên biểu tượng giỏ hàng sẽ tăng lên 1."*	AI viết code hơi phức tạp, em đã yêu cầu: *"Viết lại cách đơn giản nhất sử dụng onclick"* để dễ hiểu hơn.
Sửa lỗi	Sửa lỗi menu không hiển thị trên iPhone.	*"Menu của mình bị mất khi xem trên iPhone, đây là code của mình: [Dán code]. Hãy tìm lỗi giúp mình."*	AI phát hiện thiếu thẻ <meta name="viewport"> và em đã bổ sung.

3. Những bài học kinh nghiệm khi dùng AI

- **Câu lệnh (Prompt) hiệu quả nhất mà nhóm đúc kết được:**
  .....
- **Khó khăn khi làm việc với AI:** (Ví dụ: AI viết code quá dài, AI dùng thư viện cũ đã lỗi thời...)

- **Cách nhóm kiểm tra độ chính xác của code AI:** (Ví dụ: Chạy thử trên Chrome Inspect, hỏi AI giải thích từng dòng...)

4. Cam kết của nhóm

- [ ] Nhóm đã đọc và hiểu rõ ít nhất 80% mã nguồn do AI tạo ra.
- [ ] Nhóm đã tự tay chỉnh sửa lại giao diện (màu sắc, hình ảnh, nội dung) để phù hợp với thực tế.

5. Lưu ý

- **Sinh viên** chụp ảnh màn hình (Screenshot) những đoạn đối thoại hay với AI để chèn vào phần phụ lục của nhật ký này.
- **Kiểm tra nhanh:** Khi chấm bài, thầy sẽ chọn ngẫu nhiên một Prompt trong nhật ký của nhóm và hỏi: *"Tại sao em lại đặt câu hỏi như thế này mà không hỏi khác đi?"* để đánh giá tư duy của các em.

Phụ lục 3 - Cấu trúc kho lưu trữ tiêu chuẩn

PHỤ LỤC 3

CÁCH TỔ CHỨC CẤU TRÚC LƯU TRỮ TIÊU CHUẨN TRÊN GITHUB

1. CẤU TRÚC KHO LƯU TRỮ (REPOSITORY) TIÊU CHUẨN

Mỗi nhóm chỉ tạo **01 Repository duy nhất**. Nhóm trưởng (Scrum Master) khởi tạo và mời các thành viên khác vào làm Collaborators.

Sơ đồ thư mục đề xuất:

Plaintext

/du-an-nhom-x

│-- /assets # Chứa hình ảnh, icons, font

│-- /css # Các file styles (styles.css, layout.css)

│-- /js # Các file logic tách biệt (data.js, logic.js, main.js)

│-- /docs # Tài liệu thiết kế, wireframe

│-- index.html # File giao diện chính

│-- README.md # Giới thiệu dự án và danh sách Prompt

│-- prompt_log.md # Nhật ký làm việc với AI

Việc chia nhỏ file JS theo chức năng là cách tốt nhất để tránh việc 2 người cùng sửa một file, khiến AI tạo ra code chồng chéo.

2. QUY TRÌNH PHỐI HỢP (GIT WORKFLOW)

Để đảm bảo tính chuyên nghiệp của **Agile/Scrum**, Sinh viên tuân thủ quy trình sau:

- **Bước 1: Phân nhánh (Branching)**
  - **Nhánh `main`**: Chỉ chứa code đã hoàn thiện, không lỗi và có thể chạy Demo.
  - **Nhánh `feature-name`**: Mỗi thành viên khi làm một chức năng mới (ví dụ: `feature-login`, `feature-calculator`) phải tạo một nhánh riêng từ `main` để làm việc.
- **Bước 2: Viết Code & Commit**
  - Khi sử dụng AI để tạo code, sinh viên copy vào file tương ứng trên nhánh của mình.
  - **Commit Message chuẩn**: `[Tên-Tính-Năng] Mô tả ngắn gọn việc đã làm (Ví dụ: [UI] Hoàn thiện giao diện Sidebar).`
  - *Lưu ý: Thầy sẽ kiểm tra lịch sử commit để đánh giá sự tham gia của từng thành viên.*
- **Bước 3: Pull Request (PR) & Review**
  - Sau khi làm xong tính năng trên nhánh riêng, sinh viên gửi một **Pull Request** để gộp vào nhánh `main`.
  - Nhóm trưởng sử dụng AI để kiểm tra xem code mới có xung đột với code cũ không trước khi nhấn "Merge".

3. SỬ DỤNG GITHUB PROJECTS ĐỂ QUẢN LÝ AGILE

Thầy yêu cầu mỗi nhóm kích hoạt tính năng **Projects** trên GitHub để quản lý các đầu việc (Tasks):

1. **Cột To-do**: Danh sách các tính năng từ Backlog đã chọn cho Hackathon.
2. **Cột In Progress**: Những việc thành viên đang thực hiện.
3. **Cột Done**: Những việc đã hoàn thành và đã Merge vào `main`.

4. NỘI DUNG FILE README.MD (QUAN TRỌNG)

Đây là file dùng để thầy chấm điểm tiêu chí **Vibe Coding & Documentation**. Yêu cầu các nhóm phải có:

- **Tên dự án & Thành viên**: Tên nhóm và vai trò (Scrum Master, Dev Team).
- **Mô tả dự án**: Giải quyết vấn đề gì trong 5 chủ đề thầy đã giao.
- **Hướng dẫn chạy**: Link truy cập Live Demo (GitHub Pages/Vercel).
- **Prompt nổi bật**: Liệt kê 3-5 câu lệnh (Prompts) hiệu quả nhất mà nhóm đã dùng để điều khiển AI xử lý các logic khó.

"GitHub không chỉ là nơi nộp bài, nó là bằng chứng về cách các em phối hợp nhóm. Nhóm nào có lịch sử commit đa dạng từ tất cả thành viên và các nhánh được quản lý sạch sẽ sẽ nhận được điểm cộng lớn trong Rubric".

Phụ lục 4 Rubric chấm điểm học phần

PHỤ LỤC 4

RUBRIC CHẤM ĐIỂM CHI TIẾT (THANG ĐIỂM 10)

RUBRIC CHẤM ĐIỂM HỌC PHẦN THIẾT KẾ WEB

Tiêu chí (Trọng số)	Trọng số	Yếu (0 - 40%)	Trung bình (50 - 65%)	Khá (70 - 85%)	Giỏi (90 - 100%)
1. Giao diện UI/UX (20%)	20%	Giao diện sơ sài, chưa có bố cục rõ ràng. Không có tính Responsive (vỡ khung trên di động).	Bố cục cơ bản nhưng màu sắc/font chữ chưa hài hòa. Responsive còn lỗi nhẹ ở một số thành phần.	Giao diện sạch sẽ, đúng chuẩn LHU (màu sắc/logo). Responsive tốt trên các thiết bị phổ biến.	Giao diện hiện đại, sáng tạo. Trải nghiệm người dùng (UX) mượt mà, Responsive hoàn hảo trên mọi thiết bị.
2. Kỹ thuật JS/DOM (30%)	30%	Chưa sử dụng được hàm duyệt mảng. DOM cứng nhắc, nhiều lỗi logic khi tương tác.	Sử dụng được for hoặc forEach đơn giản. Thao tác DOM cơ bản, còn lỗi khi xử lý sự kiện phức tạp.	Sử dụng thành thạo map, filter. Thao tác DOM linh hoạt, xử lý sự kiện (click, input) ổn định.	Tối ưu mã nguồn với map, filter. Xử lý sự kiện nâng cao, không có lỗi console, code chia module khoa học.
3. Logic Dữ liệu (15%)	15%	Cấu trúc dữ liệu JSON lộn xộn hoặc sai định dạng. Các phép tính toán (GPA, tiền...) sai lệch lớn.	Có file JSON dữ liệu mẫu nhưng cấu trúc còn đơn giản. Tính toán đúng các trường hợp cơ bản.	Cấu trúc JSON khoa học, đầy đủ thuộc tính. Các hàm tính toán chính xác trong hầu hết các kịch bản.	Cấu trúc dữ liệu tối ưu, bao quát mọi trường hợp. Các hàm logic (GPA, học phí...) chính xác tuyệt đối.
4. Quy trình Agile/Git (15%)	15%	Không có bảng Kanban. Lịch sử Git chỉ có 1 thành viên commit (hoặc chỉ commit 1 lần cuối).	Có bảng Kanban nhưng cập nhật chưa đều. Git có ít nhất 2 thành viên tham gia đẩy code.	Bảng Kanban chia task rõ ràng. GitHub có lịch sử commit đều đặn từ đa số thành viên.	Sử dụng GitHub Projects/Kanban chuyên nghiệp. Lịch sử commit minh chứng sự phối hợp chặt chẽ của cả nhóm.
5. Vibe Coding - AI (10%)	10%	README sơ sài, không có danh sách Prompt. Không thể hiện được việc phối hợp với AI.	README có thông tin dự án nhưng các Prompt còn đơn giản, chưa giải quyết được vấn đề khó.	README rõ ràng, liệt kê được các Prompt giúp tạo ra cấu trúc khung hoặc CSS hiệu quả.	README chuyên nghiệp. Minh chứng được tư duy điều khiển AI qua các Prompt logic để xử lý logic khó.
6. Thuyết trình Pitching (10%)	10%	Trình bày ngập ngừng, không hiểu rõ code của nhóm mình. Demo gặp lỗi nghiêm trọng.	Trình bày đủ ý nhưng chưa lôi cuốn. Demo có lỗi nhẹ. Trả lời phản biện còn lúng túng.	Trình bày tự tin, đúng thời gian. Demo mượt mà. Hiểu và giải thích được các phần code quan trọng.	Thuyết trình lôi cuốn, demo hoàn hảo. Phản biện xuất sắc, làm rõ được cấu trúc và giải pháp kỹ thuật của nhóm.
Tổng Cộng	100 %				

Cách tính điểm định lượng cho từng tiêu chí:

- **Giỏi (9 - 10đ):** Đáp ứng đầy đủ tất cả các yêu cầu trong cột Giỏi.
- **Khá (7 - 8đ):** Đáp ứng phần lớn yêu cầu ở mức Giỏi nhưng còn một vài lỗi nhỏ về UX hoặc tối ưu code.
- **Trung bình (5 - 6đ):** Sản phẩm chạy được nhưng kỹ thuật còn cơ bản, quy trình nhóm chưa thật sự chặt chẽ.
- **Yếu (Dưới 5đ):** Sản phẩm không hoàn thiện, vi phạm các quy định về quy trình làm việc nhóm (Git/Agile).